

10장: 사용자 인증

이제 작동하는 사용자 모델이 있으므로 모든 웹사이트에 필요한 기능을 추가할 수 있습니다: 사용자가 가입하고, 로그인하고, 로그아웃할 수 있는 기능. 장고는 사용자가 로그인하고 로그아웃하는 데 필요한 모든 것을 제공하지만, 새로운 사용자가 가입할 수 있도록 자체 양식을 만들어야 합니다. 또한 사용자가 매번 URL을 손으로 입력하지 않도록 세 가지 기능에 대한 링크가 있는 기본 홈페이지를 구축할 것입니다.

템플릿

기본적으로 장고 템플릿 로더는 각 앱 내에서 중첩된 구조의 템플릿을 찾습니다. 예를 들어, accounts 앱 내의 home.html 템플릿을 위해서는 accounts/templates/accounts/home.html 구조가 필요합니다. 그러나 단일 프로젝트 수준의 템플릿 디렉토리 접근 방식이 더 깔끔하고 확장성이 좋으므로, 우리는 그것을 사용할 것입니다.

새로운 프로젝트 수준의 템플릿 디렉토리와 장고가 로그인 및 가입과 관련된 템플릿을 찾을 registration 디렉토리를 만듭니다.

셸

```
(.venv) $ mkdir templates  
(.venv) $ mkdir templates/registration
```

이 새로운 디렉토리에 대해 장고에 알려주기 위해 django_project/settings.py의 "DIRS" 구성을 업데이트해야 합니다.

Code

```
# django_project/settings.py  
TEMPLATES = [  
    {  
        ...  
        "DIRS": [BASE_DIR / "templates"],      # 새로 만들기  
        ...  
    }  
]
```

사이트에 로그인하거나 로그아웃할 때 어떤 일이 발생하는지 생각해보면, 사용자는 즉시 다음 페이지로 리디렉션됩니다. 우리는 각 경우에 사용자를 어디로 보낼지 Django에 알려야 합니다. LOGIN_REDIRECT_URL 및 LOGOUT_REDIRECT_URL 설정이 이를 수행합니다. 우리는 둘 다 홈페이지로 리디렉션하도록 구성할 것입니다.

named URL of 'home'. Remember that when we create our URL routes, we can add a name to each one. So when we make the homepage URL, we'll call it 'home'.

Add these two lines at the bottom of the `django_project/settings.py` file.

Code

```
# django_project/settings.py
LOGIN_REDIRECT_URL = "home" # new
LOGOUT_REDIRECT_URL = "home" # new
```

Now we can create four new templates within our text editor:

- `templates/base.html`
- `templates/home.html`
- `templates/registration/login.html`
- `templates/registration/signup.html`

Here's the HTML code for each file to use. The `base.html` will be inherited by every other template in our project. Using a block like `{% block content %}`, we can later override the content *just in this place* in other templates.

Code

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>{% block title %}Newspaper App{% endblock title %}</title>
</head>

<body>
  <main>
    {% block content %}
    {% endblock content %}
  </main>
</body>

</html>
```

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block title %}Home{% endblock title %}

{% block content %}
{% if user.is_authenticated %}
<p>Hi {{ user.username }}!</p>
```

'home'의 명명된 URL. URL 경로를 생성할 때 각 경로에 이름을 추가할 수 있다는 것을 기억하세요. 그래서 홈페이지 URL을 만들 때 우리는 그것을 'home'이라고 부를 것입니다.

django_project/settings.py 파일의 맨 아래에 이 두 줄을 추가하세요. 코드

```
# django_project/settings.py
LOGIN_REDIRECT_URL = "home" # 새
LOGOUT_REDIRECT_URL = "home" # 새
```

이제 텍스트 편집기 내에서 네 개의 새로운 템플릿을 만들 수 있습니다:

- templates/base.html
- templates/home.html
- templates/registration/login.html
- templates/registration/signup.html
-

각 파일에 사용할 HTML 코드입니다. base.html은 프로젝트의 다른 모든 템플릿에 의해 상속됩니다. {% block content %}와 같은 블록을 사용하여, 나중에 다른 템플릿에서 이 위치의 내용만 재정의할 수 있습니다.

코드

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>{% block title %}신문 앱{% endblock title %}</title></head>

<body>
  <main>
    <div>{% block content %}</div>
  </main>
</body>

</html>
```

코드

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block title %}홈{% endblock title %}

{% block content %}
{% if user.is_authenticated %}
  <p>안녕하세요 {{ user.username }}!</p>
{% else %}
  <p>로그인하세요</p>
{% endif %}
{% endblock content %}
```

```
<form action="{% url 'logout' %}" method="post">
    {% csrf_token %}
    <button type="submit">Log Out</button>
</form>
{% else %}
<p>You are not logged in</p>
<a href="{% url 'login' %}">Log In</a> |
<a href="{% url 'signup' %}">Sign Up</a>
{% endif %}
{% endblock content %}
```

Code

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}

{% block title %}Log In{% endblock title %}

{% block content %}
<h2>Log In</h2>
<form method="post">{% csrf_token %}
    {{ form }}
    <button type="submit">Log In</button>
</form>
{% endblock content %}
```

Code

```
<!-- templates/registration/signup.html -->
{% extends "base.html" %}

{% block title %}Sign Up{% endblock title %}

{% block content %}
<h2>Sign Up</h2>
<form method="post">{% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Sign Up</button>
</form>
{% endblock content %}
```

Our templates are now all set. The related URLs and views are still to go.

URLs

Let's start with the URL routes. In our `django_project/urls.py` file, we want our `home.html` template to appear as the homepage, but we don't want to build a dedicated pages app yet. We can use the shortcut of importing `TemplateView` and setting the `template_name` right in our URL pattern.

Next, we want to “include” the `accounts` app and the built-in `auth` app. The reason is that the built-in `auth` app already provides views and URLs for logging


```
<form action="{% url 'logout' %}" method="post">
    {% csrf_token %}
    <button type="submit">로그 아웃</button>
</form>
{% else %}
<p>로그인하지 않았습니다</p><a href="{% url
'login' %}">로그인</a> | <a href="{% url 'signup'
%}">회원가입</a>{% endif %}

{% endblock content %}
```

코드

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}

{% block title %}로그인{% endblock title %}{% block
content %}

<h2>로그인</h2>
<form method="post">{% csrf_token %}{% form
    %}
    <button type="submit">로그인</button>
</form>
{% endblock content %}
```

코드

```
<!-- templates/registration/signup.html -->
{% extends "base.html" %}

{% block title %}회원가입{% endblock title %}{% block
content %}

<h2>회원가입</h2>
<form method="post">{% csrf_token %}
    {% form.as_p %}
    <button type="submit">가입하기</button>
</form>
{% endblock content %}
```

우리의 템플릿은 이제 모두 설정되었습니다. 관련된 URL과 뷰는 아직 진행 중입니다.

URLs

URL 경로부터 시작하겠습니다. 우리의 `django_project/urls.py` 파일에서 `home.html` 템플릿이 홈페이지로 나타나기를 원하지만, 전용 페이지 앱을 아직 만들고 싶지 않습니다. 우리는 `TemplateView`를 가져오고 URL 패턴에서 `template_name`을 바로 설정하는 단축키를 사용할 수 있습니다.

다음으로, 우리는 계정 앱과 내장된 인증 앱을 "포함"하고 싶습니다. 그 이유는 내장된 인증 앱이 이미 로그인을 위한 뷰와 URL을 제공하기 때문입니다.

in and out. But to sign up, we must create our own view and URL. To ensure that our URL routes are consistent, we place them *both* at `accounts/` so the eventual URLs will be `/accounts/login`, `/accounts/logout`, and `/accounts/signup`.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include # new
from django.views.generic.base import TemplateView # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")), # new
    path("accounts/", include("django.contrib.auth.urls")), # new
    path("", TemplateView.as_view(template_name="home.html"),
         name="home"), # new
]
```

Now create a file with your text editor called `accounts/urls.py` and update it with the following code:

Code

```
# accounts/urls.py
from django.urls import path
from .views import SignUpView

urlpatterns = [
    path("signup/", SignUpView.as_view(), name="signup"),
]
```

The last step is our `views.py` file containing the logic for our signup form. We're using Django's generic `CreateView` here and telling it to use our `CustomUserCreationForm`, to redirect to login once a user signs up successfully and that our template is named `signup.html`.

Code

```
# accounts/views.py
from django.urls import reverse_lazy
from django.views.generic import CreateView

from .forms import CustomUserCreationForm

class SignUpView(CreateView):
    form_class = CustomUserCreationForm
    success_url = reverse_lazy("login")
    template_name = "registration/signup.html"
```

Ok, phew! We're done. Let's test things out. Start the server with `python manage.py runserver` and go to the homepage.

안팎으로. 하지만 가입하려면, 우리는 우리만의 뷰와 URL을 만들어야 합니다. 우리의 URL 경로가 일관되도록 하기 위해, 우리는 둘 다 accounts/에 배치하여 최종 URL이 /accounts/login, /accounts/logout, 및 /accounts/signup이 되도록 합니다.

코드

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include # new
from django.views.generic.base import TemplateView # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")), # new
    path("accounts/", include("django.contrib.auth.urls")), # new
    path("", TemplateView.as_view(template_name="home.html"),
         name="home"), # new
]
```

이제 텍스트 편집기로 accounts/urls.py라는 파일을 만들고 다음 코드를 업데이트하세요:

코드

```
# accounts/urls.py
from django.urls import path
from .views import SignUpView

urlpatterns = [
    path("signup/", SignUpView.as_view(), name="signup"), ]
```

마지막 단계는 우리의 signup 양식 로직을 포함하는 views.py 파일입니다. 우리는 Django의 일반 CreateView를 사용하고 있으며, 사용자가 성공적으로 가입하면 로그인으로 리디렉션하고 우리의 템플릿 이름이 signup.html이라고 알려주고 있습니다.

코드

```
# accounts/views.py
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .forms import CustomUserCreationForm

class SignUpView(CreateView):
    form_class = CustomUserCreationForm
    success_url = reverse_lazy("login")
    template_name = "registration/signup.html"
```

좋아, 후우! 끝났어. 테스트해보자. pythonmanage.py runserver로 서버를 시작하고 홈페이지로 가자.



Homepage Logged In

We logged in to the admin in the previous chapter, so you should see a personalized greeting here. Click on the “Log Out” link.



Homepage Logged Out

Now we’re on the logged-out homepage. Click on *Log In* link and use your **superuser** credentials. Upon successfully logging in, you’ll be redirected to the homepage and see the same personalized greeting. It works!

Now, use the “Log Out” link to return to the logged-out homepage, and this time, click on the “Sign Up” link. You’ll be redirected to our signup page. See that the age field is included!

Create a new user. Mine is called testuser, and I’ve set the age to 25.



홈페이지 로그인됨

이전 장에서 관리자에 로그인했으므로 여기에서 개인화된 인사말을 볼 수 있어야 합니다. “로그아웃” 링크를 클릭하세요.



홈페이지 로그아웃됨

이제 로그아웃된 홈페이지에 있습니다. 로그인 링크를 클릭하고 슈퍼유저 자격 증명을 사용하세요. 성공적으로 로그인하면 홈페이지로 리디렉션되고 동일한 개인화된 인사말을 보게 됩니다. 작동합니다!

이제 “로그아웃” 링크를 사용하여 로그아웃된 홈페이지로 돌아가고 이번에는 “가입” 링크를 클릭하세요. 가입 페이지로 리디렉션됩니다. 나이 필드가 포함되어 있는지 확인하세요!

새 사용자를 만드세요. 제 사용자는 testuser라고 하며, 나이를 25로 설정했습니다.

Sign Up

Username: Required. 150 characters or fewer. Letters, digits and @/./+/_ only.

Age:

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

Signup Page

After successfully submitting the form, you'll be redirected to the login page. Log in with your new user, and you'll be redirected again to the homepage with a personalized greeting for the new user. But since we have the new age field, let's add that to the `home.html` template. It is a field on the user model, so to display it, we only need to use `{{ user.age }}`.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block title %}Home{% endblock title %}

{% block content %}
{% if user.is_authenticated %}
<!-- new code here! -->
Hi {{ user.username }}! You are {{ user.age }} years old.
<!-- end of new code -->
<form action="{% url 'logout' %}" method="post">
  {% csrf_token %}
  <button type="submit">Log Out</button>
</form>
{% else %}
<p>You are not logged in</p>
<a href="{% url 'login' %}">Log In</a> |
<a href="{% url 'signup' %}">Sign Up</a>
{% endif %}
{% endblock content %}
```

Save the file and refresh the homepage.

Sign Up

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Age:

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

가입 페이지

양식을 성공적으로 제출한 후, 로그인 페이지로 리디렉션됩니다. 새 사용자로 로그인하면 개인화된 인사말과 함께 홈페이지로 다시 리디렉션됩니다. 하지만 새로운 나이 필드가 있으므로 home.html 템플릿에 추가합니다. 이는 사용자 모델의 필드이므로 표시하려면 `{{ user.age }}`를 사용하기만 하면 됩니다.

드

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block title %}홈{% endblock title %}

{% block content %}{% if
user.is_authenticated %}
<!-- 여기 새로운 코드! -->
안녕하세요 {{ user.username }}! 당신은 {{ user.age }}세입니다.
<!-- 새로운 코드 끝 -->
<form action="{% url 'logout' %}" method="post">
  {% csrf_token %}
  <button type="submit">로그 아웃</button>
</form>
{% else %}
<p>로그인하지 않았습니다</p><a href="{% url
'login' %}">로그인</a> | <a href="{% url 'signup'
%}">회원가입</a>{% endif %}

{% endblock content %}
```

파일을 저장하고 홈페이지를 새로 고치세요.

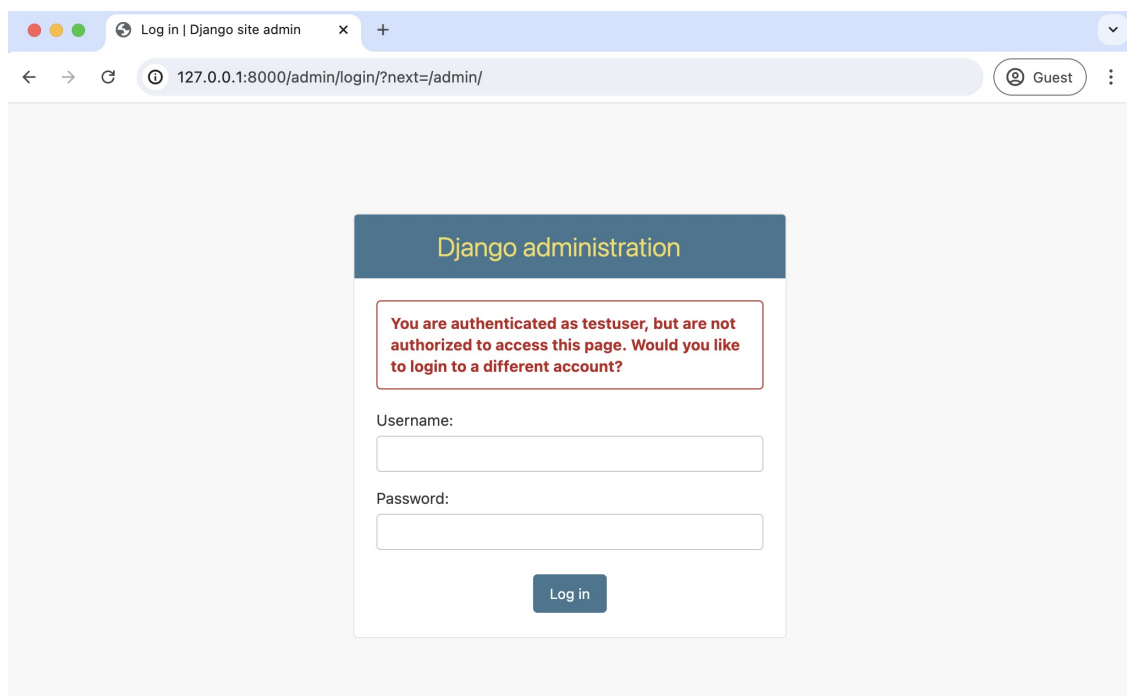


Homepage for testuser

Everything works as expected.

Admin

Navigate to the admin at `http://127.0.0.1:8000/admin` in your web browser and log in to view the two user accounts.



Wrong Admin Login

What's this? Why can't we log in? We're logged in with our new testuser account, not our superuser account. Only a superuser account has permission to log in to the admin! So, use your superuser account to log in instead.

After you've done that, you should see the normal admin homepage. Click on Users to see our two users: the testuser account we just created and your

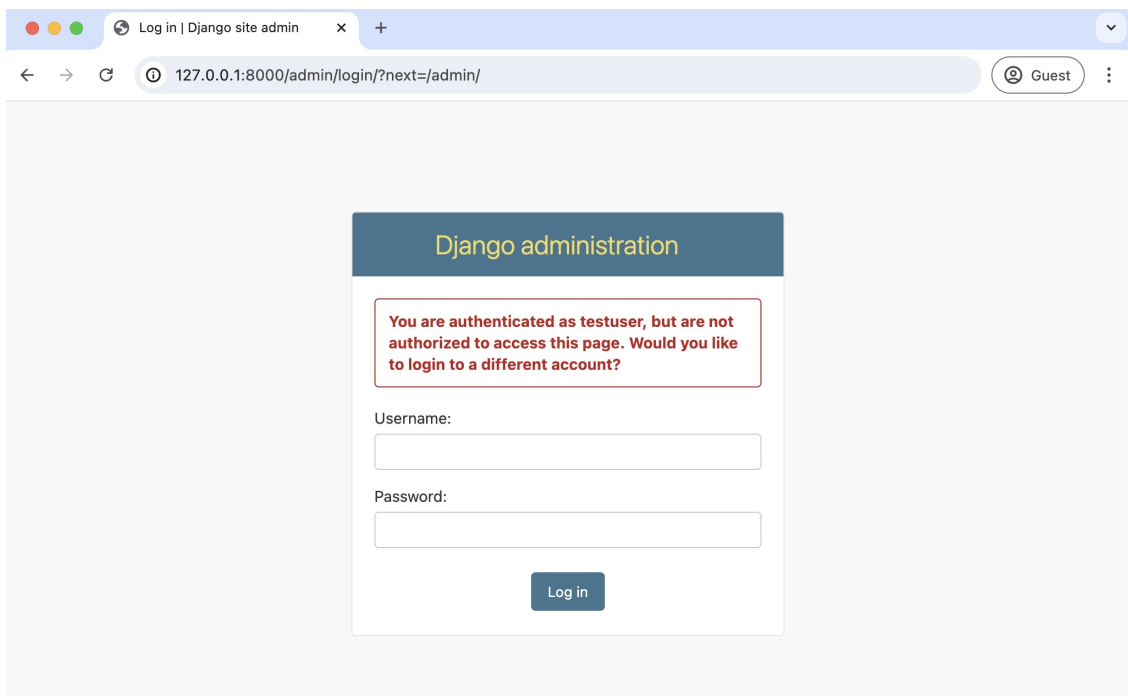


테스트 사용자용 홈페이지

모든 것이 예상대로 작동합니다.

관리자

웹 브라우저에서 <http://127.0.0.1:8000/admin>으로 이동하여 두 개의 사용자 계정을 보려면 로그인하세요.

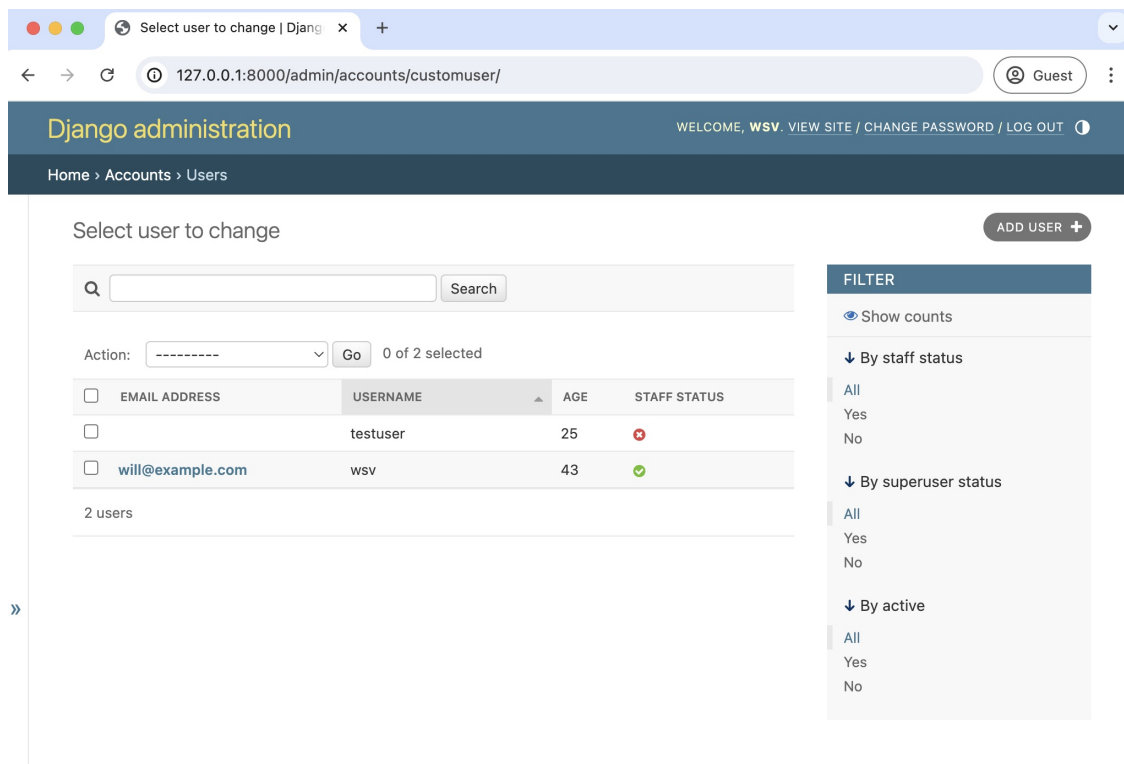


잘못된 관리자 로그인

이게 뭐죠? 왜 로그인할 수 없죠? 우리는 슈퍼유저 계정이 아닌 새로운 테스트 사용자 계정으로 로그인했습니다. 관리자에 로그인할 수 있는 권한은 오직 슈퍼유저 계정만 가지고 있습니다! 그러니 대신 슈퍼유저 계정으로 로그인하세요.

그 작업을 마친 후에는 정상적인 관리자 홈페이지가 표시되어야 합니다. 사용자 클릭하여 방금 생성한 테스트 사용자 계정과 당신의 두 사용자 계정을 확인하세요.

previous superuser name (mine is wsv).



Users in the Admin

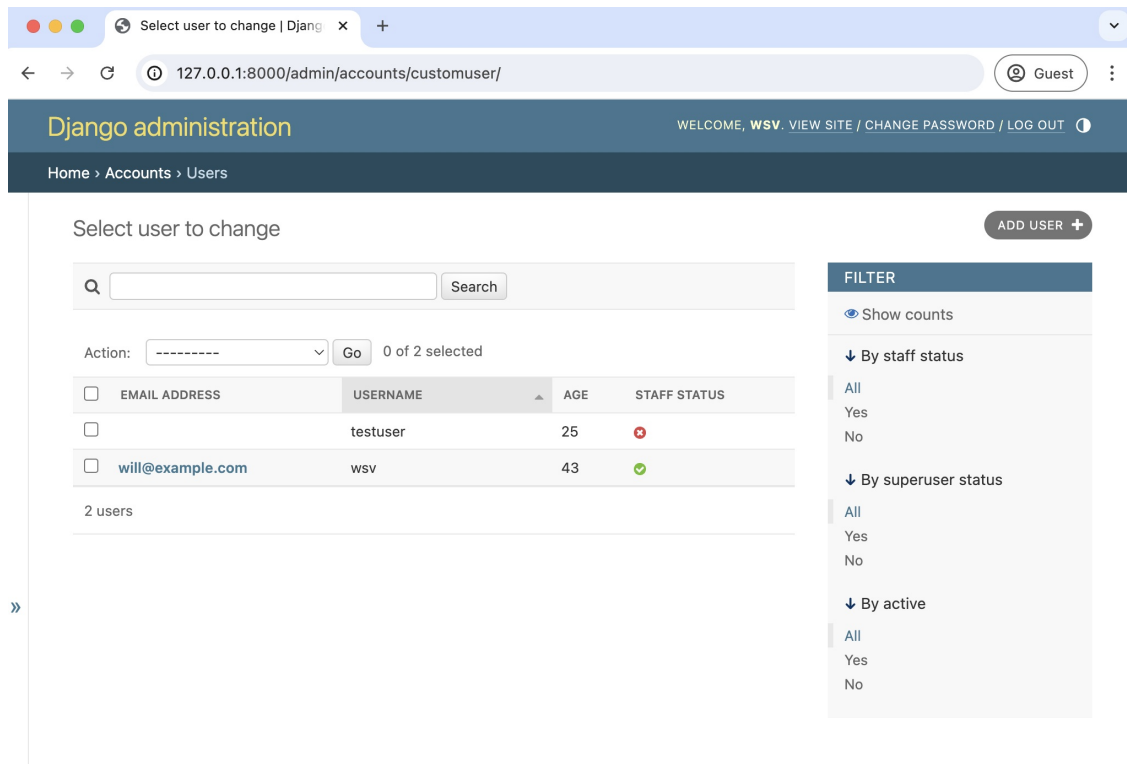
Everything is working, but you may notice no “email address” for our testuser. Why is that? Our signup page has no email field because it was not included in `accounts/forms.py`. This is an important point: just because the user model has a field does not mean it will be included in our custom signup form unless explicitly added. Let’s do so now.

Currently, in `accounts/forms.py` under `fields`, we’re using `Meta.fields`, which displays the default settings of username/password, and then we explicitly added our custom field, age, too. But we can also explicitly set which fields we want to be displayed, so let’s update it to ask for a username/email/age/password by setting it to `( 'username', 'email', 'age', )`. We don’t need to include the password fields because they are required! The other fields can be configured however we choose.

Code

```
# accounts/forms.py
from django.contrib.auth.forms import UserCreationForm, UserChangeForm
from .models import CustomUser
```

이전 슈퍼유저 이름 (내 이름은 wsv입니다).



관리자의 사용자들

모든 것이 작동하고 있지만, 테스트 사용자에게 대한 "이메일 주소"가 없다는 것을 알 수 있습니다. 왜 그럴까요? 우리의 가입 페이지에는 이메일 필드가 없는데, 이는 `accounts/forms.py`에 포함되지 않았기 때문입니다. 이것은 중요한 점입니다: 사용자 모델에 필드가 있다고 해서 반드시 우리의 사용자 정의 가입 양식에 포함된다는 의미는 아닙니다. 명시적으로 추가하지 않는 한 포함되지 않습니다. 지금 추가해 봅시다.

현재 `accounts/forms.py`의 필드 아래에서 우리는 `Meta.fields`를 사용하고 있으며, 이는 기본 사용자 이름/비밀번호 설정을 표시합니다. 그리고 우리는 사용자 정의 필드인 나이도 명시적으로 추가했습니다. 하지만 어떤 필드를 표시할지 명시적으로 설정할 수도 있으므로, 사용자 이름/이메일/나이/비밀번호를 요청하도록 ('username', 'email', 'age',)로 업데이트합니다. 비밀번호 필드는 필수이므로 포함할 필요가 없습니다! 다른 필드는 우리가 원하는 대로 구성할 수 있습니다.

코드

```
# accounts/forms.py
from django.contrib.auth.forms import UserCreationForm, UserChangeForm from .models
import CustomUser
```

```

class CustomUserCreationForm(UserCreationForm):
    class Meta:
        model = CustomUser
        fields = (
            "username",
            "email",
            "age",
        ) # new

class CustomUserChangeForm(UserChangeForm):
    class Meta:
        model = CustomUser
        fields = (
            "username",
            "email",
            "age",
        ) # new

```

Log out of your superuser account and try
<http://127.0.0.1:8000/accounts/signup/> again—you can see the additional
 “Email address” field is there. Sign up with a new user account. I’ve named
 mine testuser2 with an age of 18 and an email address of
 testuser2@email.com.

Sign Up

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address:

Age:

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

New Signup Page

Click the “Sign Up” button and continue to log in. You’ll see a personalized greeting on the homepage.

```
class CustomUserCreationForm( UserCreationForm ): class
```

```
Meta:
```

```
    model = CustomUser
```

```
    fields = ( "
                username", "
                email", "
                age",
```

```
    ) # new
```

```
class CustomUserChangeForm
```

```
( UserChangeForm ) : class Meta:
```

```
    model = CustomUser
```

```
    fields = ( "
                username", "
                email", "
                age",
```

```
    ) # new
```

슈퍼유저 계정에서 로그아웃하고

<http://127.0.0.1:8000/accounts/signup/> 를 다시 시도하세요- 추가된 "이메일 주소" 필드가 있습니다. 새로운 사용자 계정으로 가입하세요. 저는 제 계정을 testuser2로 이름 지었고, 나이는 18세이며 이메일 주소는 testuser2@email.com입니다.

Sign Up

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address:

Age:

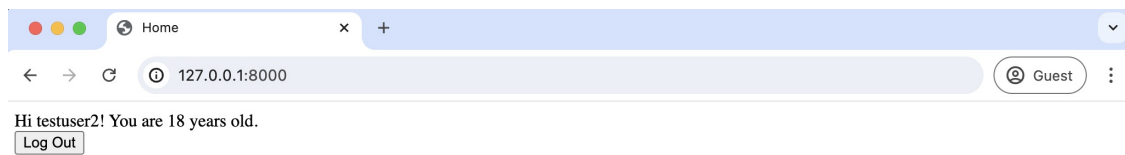
Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

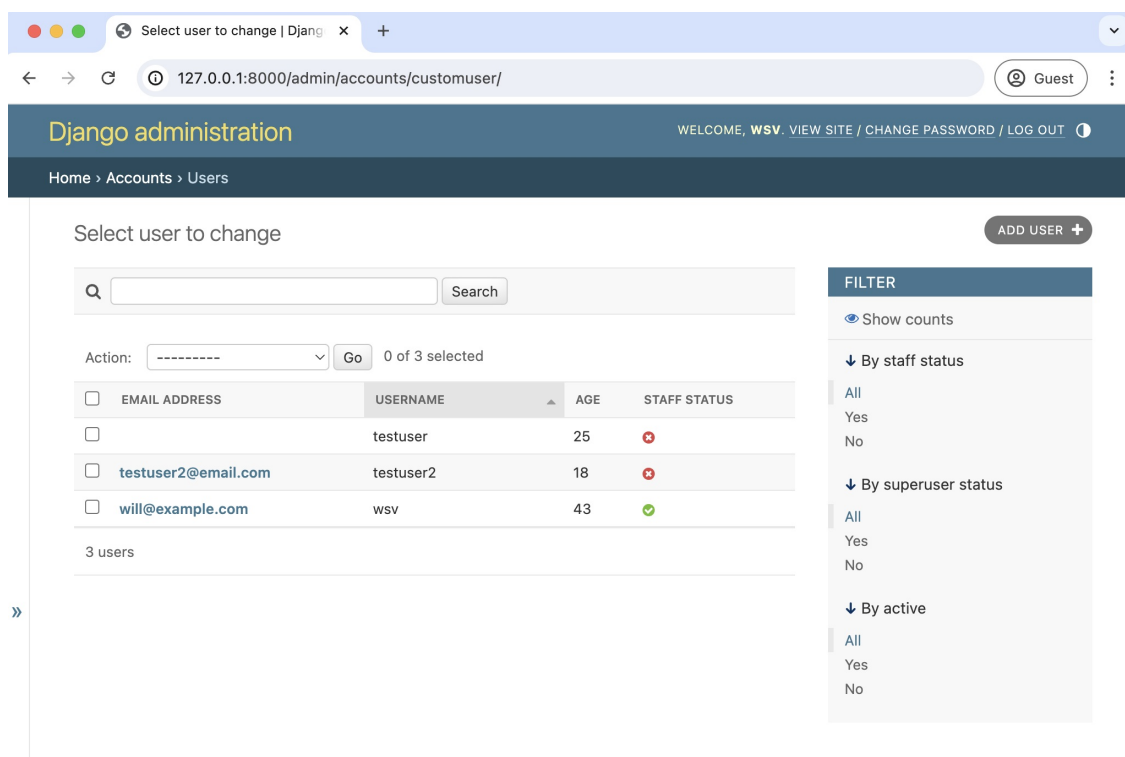
새로운 가입 페이지

"가입하기" 버튼을 클릭하고 로그인 계속 진행하세요. 홈페이지에서 개인화된 인사말을 보게 될 것입니다.



testuser2 Homepage Greeting

Switch back to the admin page, log in using our superuser account, and all three users are on display.



Three Users in the Admin

Django's user authentication flow requires some setup. Still, you should be starting to see that it also provides us incredible flexibility to configure the signup and login process *exactly* how we want it.

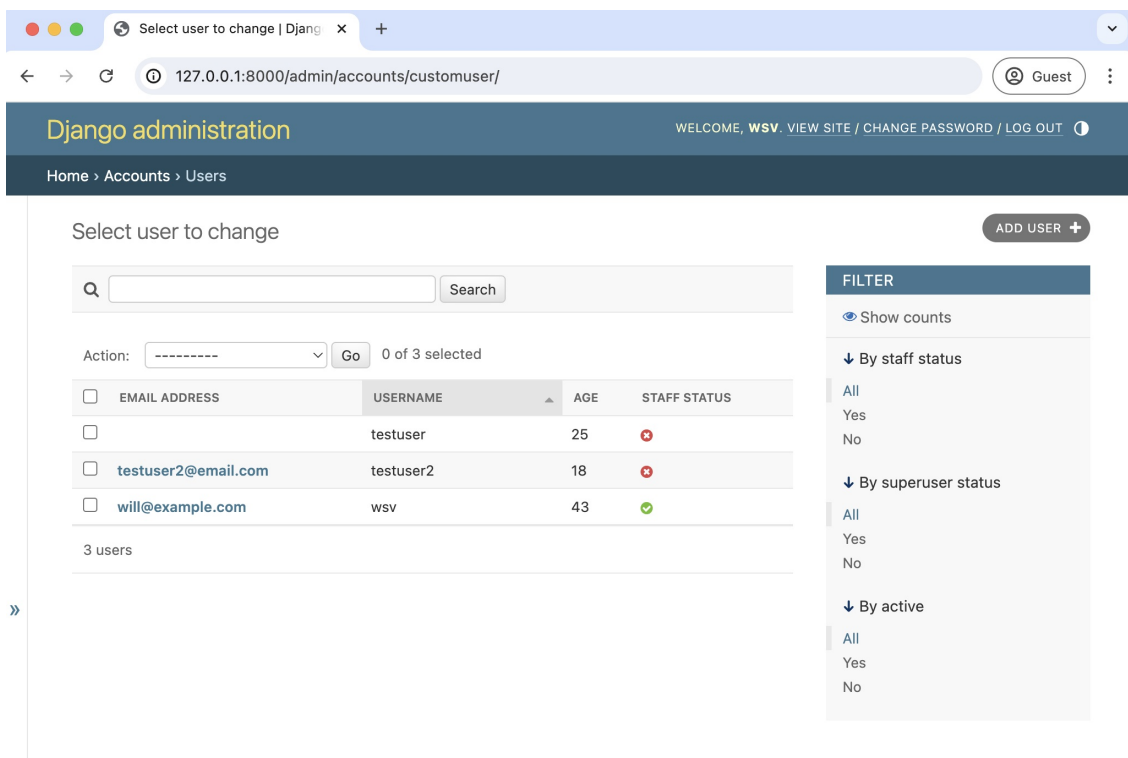
Tests

The new signup page has a view, URL, and template that all should be tested. Open up the `accounts/tests.py` file containing code from the last chapter for



testuser2 홈페이지 인사말

관리자 페이지로 돌아가서 슈퍼유저 계정을 사용하여 로그인하면 세 명의 사용자가 표시됩니다.



관리자에서의 세 명의 사용자

Django의 사용자 인증 흐름은 약간의 설정이 필요합니다. 그럼에도 불구하고 우리는 가입 및 로그인 프로세스를 원하는 대로 구성할 수 있는 놀라운 유연성을 제공한다는 것을 보기 시작해야 합니다.

테스트

새로운 가입 페이지에는 모두 테스트해야 할 뷰, URL 및 템플릿이 있습니다. 마지막 장의 코드를 포함하는 `accounts/tests.py` 파일을 열어보세요.

UsersManagersTests. Below it, add a new class called SignupPageTests that we will review below.

Code

```
# accounts/tests.py
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse # new

class UsersManagersTests(TestCase):
    ...

class SignupPageTests(TestCase): # new
    def test_url_exists_at_correct_location_signupview(self):
        response = self.client.get("/accounts/signup/")
        self.assertEqual(response.status_code, 200)

    def test_signup_view_name(self):
        response = self.client.get(reverse("signup"))
        self.assertEqual(response.status_code, 200)
        self.assertTemplateUsed(response, "registration/signup.html")

    def test_signup_form(self):
        response = self.client.post(
            reverse("signup"),
            {
                "username": "testuser",
                "email": "testuser@email.com",
                "password1": "testpass123",
                "password2": "testpass123",
            },
        )
        self.assertEqual(response.status_code, 302)
        self.assertEqual(get_user_model().objects.all().count(), 1)
        self.assertEqual(get_user_model().objects.all()[0].username, "testuser")
        self.assertEqual(
            get_user_model().objects.all()[0].email, "testuser@email.com"
        )
```

At the top, we import [reverse](#) to verify that the URL and view work properly. Then we create a new class of tests called SignupPageTests. The first test checks that our signup page is at the correct URL and returns a 200 status code. The second test checks the view, reverses signup, which is the URL name, and then confirms a 200 status code and that our signup.html template is being used.

The third test checks our form by sending a post request to fill it out. After the form is submitted, we confirm the expected 302 redirect and confirm that there is now one user in the test database with a matching username and email address. We do not check the password because Django automatically encrypts them by

UsersManagersTests. 아래에 SignupPageTests라는 새로운 클래스를 추가합니다. 아래에서 검토할 것입니다.

코드

```
# accounts/tests.py
from django.contrib.auth import get_user_model from
django.test import TestCase
from django.urls import reverse # new

class UsersManagersTests(TestCase): ...

class SignupPageTests(TestCase): # new
    def test_url_exists_at_correct_location_signupview(self): response =
        self.client.get("/accounts/signup/") self.assertEqual
        (response.status_code, 200)

    def test_signup_view_name(self):
        response = self.client.get(reverse("signup")) self.assertEqual
        (response.status_code, 200) self.assertTemplateUsed(response, "
        registration/signup.html")

    def test_signup_form(self):
        response = self.client.post(reverse
            ("signup"),
            {
                "username": "testuser", "email": "
                testuser@email.com", "password1": "
                testpass123", "password2": "
                testpass123",
            },
        )
        self.assertEqual(response.status_code, 302) self.assertEqual(get_user_model
        ().objects.all().count(), 1) self.assertEqual(get_user_model().objects.all()
        [0].username, "testuser") self.assertEqual(
            get_user_model().objects.all()[0].email, "testuser@email.com")
```

맨 위에서, 우리는 [URL과 뷰](#)가 제대로 작동하는지 확인하기 위해 `reverse`를 가져옵니다. 그런 다음 `SignupPageTests`라는 새로운 테스트 클래스를 만듭니다. 첫 번째 테스트는 우리의 가입 페이지가 올바른 URL에 있고 200 상태 코드를 반환하는지 확인합니다. 두 번째 테스트는 뷰를 확인하고, `signup`을 역전시킵니다. 이는 URL 이름이며, 그런 다음 200 상태 코드와 우리의 `signup.html` 템플릿이 사용되고 있는지 확인합니다.

세 번째 테스트는 양식을 확인하기 위해 게시 요청을 보내어 양식을 작성합니다. 양식이 제출된 후, 우리는 예상되는 302 리디렉션을 확인하고, 테스트 데이터베이스에 일치하는 사용자 이름과 이메일 주소를 가진 사용자가 하나 있는지 확인합니다. 우리는 Django가 자동으로 비밀번호를 암호화하기 때문에 비밀번호를 확인하지 않습니다.

default. That is why if you look in a user's admin view, you can change a password but can't see the current one.

Run the tests with `python manage.py test` to check that everything passes as expected.

Shell

```
(.venv) $ python manage.py test
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...
-----
Ran 5 tests in 0.183s

OK
Destroying test database for alias 'default'...
```

Git

Before moving on to the next chapter, let's record our work with Git and store it on GitHub.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "user authentication"
(.venv) $ git push origin main
```

Conclusion

So far, our *Newspaper* app has a custom user model and working signup, login, and logout pages. But you may have noticed that our site could look better. In the next chapter, we'll add CSS styling with Bootstrap and create a dedicated pages app.

기본값. 그래서 사용자의 관리자 보기에서 비밀번호를 변경할 수 있지만 현재 비밀번호는 볼 수 없습니다.

모든 것이 예상대로 통과하는지 확인하려면 `python manage.py test`로 테스트를 실행하세요.

셸

```
(.venv) $ python manage.py test
5개의 테스트를 찾았습니다.
별칭 'default'에 대한 테스트 데이터베이스를 생성하는 중...
시스템 검사가 문제를 식별하지 않았습니다 (0 무시됨)...
```

```
-----
- 0.183초에 5개의 테스트를 실행했습니다.
```

확인

```
별칭 'default'에 대한 테스트 데이터베이스를 파괴하는 중...
```

Git

다음 장으로 넘어가기 전에 Git으로 작업을 기록하고 GitHub에 저장하십시오.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "사용자 인증"
(.venv) $ git push origin main
```

결론

지금까지 우리의 신문 앱은 사용자 정의 모델과 작동하는 가입, 로그인 및 로그아웃 페이지를 가지고 있습니다. 하지만 사이트가 더 나아 보일 수 있다는 것을 눈치챈 것입니다. 다음 장에서는 Bootstrap을 사용하여 CSS 스타일링을 추가하고 전용 페이지 앱을 만들 것입니다.

Chapter 11: Bootstrap

Web development requires many skills. Not only do you have to program the website correctly, but users also expect it to look good. Adding all the necessary HTML/CSS for a beautiful site can be overwhelming when creating everything from scratch.

While it's possible to hand-code all the required CSS and JavaScript for a modern-looking website, in practice, most developers use a framework like [Bootstrap](#) or [TailwindCSS](#). We'll use Bootstrap for our project, which can be extended and customized as needed.

Pages App

In the previous chapter, we displayed our homepage by including view logic in our `urls.py` file. While this approach works, it feels hackish to me, and it certainly doesn't scale as a website grows over time; it is also confusing to Django newcomers. Instead, we can and should create a dedicated pages app for all of our static pages, such as the homepage, a future about page, etc. This will keep our code nice and organized.

On the command line, use the `startapp` command to create our new pages app. If the server is still running, you must type `Control+c` first to quit.

Shell

```
(.venv) $ python manage.py startapp pages
```

Then, immediately update our `django_project/settings.py` file. I often forget to do this, so it is a good practice to think of creating a new app as a two-step process: run the `startapp` command, then update `INSTALLED_APPS`.

Code

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "accounts",
]
```

11장: 부트스트랩

웹 개발은 많은 기술을 요구합니다. 웹사이트를 올바르게 프로그래밍해야 할 뿐만 아니라, 사용자들은 또한 웹사이트가 보기 좋기를 기대합니다. 아름다운 사이트를 만들기 위해 필요한 모든 HTML/CSS를 추가하는 것은 처음부터 모든 것을 만드는 경우 압도적일 수 있습니다.

현대적인 웹사이트를 위해 필요한 모든 CSS와 JavaScript를 수작업으로 코딩하는 것이 가능하지만, 실제로 대부분의 개발자는 Bootstrap이나 TailwindCSS와 같은 프레임워크를 사용합니다. 우리는 프로젝트에 Bootstrap을 사용할 것이며, 필요에 따라 확장하고 사용자 정의할 수 있습니다.

페이지 앱

이전 장에서는 urls.py 파일에 뷰 로직을 포함하여 홈페이지를 표시했습니다. 이 접근 방식이 작동하지만, 저에게는 해킹처럼 느껴지며, 시간이 지남에 따라 웹사이트가 성장할 때 확장되지 않습니다. 또한 Django 초보자에게 혼란스러울 수 있습니다. 대신, 우리는 모든 정적 페이지(예: 홈페이지, 향후 소개 페이지 등)를 위한 전용 페이지 앱을 만들 수 있으며, 그렇게 해야 합니다. 이렇게 하면 우리의 코드가 깔끔하고 조직적으로 유지됩니다.

명령줄에서 startapp 명령을 사용하여 새로운 페이지 앱을 만듭니다. 서버가 여전히 실행 중인 경우, 먼저 Control+c를 입력하여 종료해야 합니다. 셸

```
(.venv) $ python manage.py startapp pages
```

그런 다음 즉시 django_project/settings.py 파일을 업데이트합니다. 저는 종종 이 작업을 잊어버리므로, 새로운 앱을 만드는 것을 두 단계 프로세스로 생각하는 것이 좋은 습관입니다: startapp 명령을 실행한 다음 INSTALLED_APPS를 업데이트합니다.

코드

```
# django_project/settings.py
INSTALLED_APPS = [
    django.contrib.admin",
    django.contrib.auth",
    django.contrib.contenttypes",
    django.contrib.sessions",
    django.contrib.messages",
    django.contrib.staticfiles",
    accounts",
```

```
    "pages", # new
]
```

Now, we can update our `urls.py` file inside the `django_project` directory by adding the `pages` app and removing the import of `TemplateView` and the previous URL path for the older homepage.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")),
    path("accounts/", include("django.contrib.auth.urls")),
    path("", include("pages.urls")), # new
]
```

It's time to add our homepage, which means Django's standard URLs/views/templates dance. We'll start with the `pages/urls.py` file. First, create it with your text editor. Then, import our not-yet-created views, set the route paths, and name each URL, too.

Code

```
# pages/urls.py
from django.urls import path

from .views import HomePageView

urlpatterns = [
    path("", HomePageView.as_view(), name="home"),
]
```

The `views.py` code should look familiar at this point. We're using Django's `TemplateView` generic class-based view, meaning we only need to specify our `template_name` to use it.

Code

```
# pages/views.py
from django.views.generic import TemplateView

class HomePageView(TemplateView):
    template_name = "home.html"
```

We already have an existing `home.html` template. Let's confirm it still works as expected with our new URL and view. Start the local server `python manage.py`

```
] "pages", # 새로 만들  
]
```

이제, 우리는 django_project 디렉토리 안의 urls.py 파일을 업데이트하여 pages 앱을 추가하고 TemplateView의 임포트를 제거하며 이전 홈페이지의 URL 경로를 삭제할 수 있습니다.

코드

```
# django_project/urls.py  
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("accounts/", include("accounts.urls")), path("accounts/",  
    include("django.contrib.auth.urls")), path("", include("pages.urls")), # new  
]
```

이제 홈페이지를 추가할 시간입니다. 이는 Django의 표준 URL/뷰/템플릿 작업을 의미합니다. 우리는 pages/urls.py 파일부터 시작할 것입니다. 먼저, 텍스트 편집기로 파일을 생성합니다. 그런 다음, 아직 생성되지 않은 뷰를 가져오고, 경로를 설정하며, 각 URL의 이름도 지정합니다.

코드

```
# pages/urls.py  
from django.urls import path  
  
from .views import HomePageView  
  
urlpatterns = [  
    path("", HomePageView.as_view(), name="home"),  
],
```

이 시점에서 views.py 코드는 익숙하게 보일 것입니다. 우리는 Django의 TemplateView 제네릭 클래스 기반 뷰를 사용하고 있으며, 이를 사용하기 위해 template_name만 지정하면 됩니다.

코드

```
# pages/views.py  
from django.views.generic import TemplateView  
  
class HomePageView  
    (TemplateView):  
    template_name = "home.html"
```

우리는 이미 기존의 home.html 템플릿을 가지고 있습니다. 새로운 URL과 뷰로 여전히 예상대로 작동하는지 확인해 봅시다. 로컬 서버를 시작하세요 python manage.py

runserver and navigate to the homepage at `http://127.0.0.1:8000/` to confirm it remains unchanged. It should show the name and age of your logged-in superuser account, which we used at the end of the last chapter.

Tests

We've added new code and functionality, so it is time for tests. You can never have enough tests in your projects. Even though they take some upfront time to write, they always save you time down the road and provide you with confidence as a project grows in complexity.

Let's add tests to ensure our new homepage works properly. The code should look like this in your `pages/tests.py` file.

Code

```
# pages/tests.py
from django.test import SimpleTestCase
from django.urls import reverse

class HomePageTests(SimpleTestCase):
    def test_url_exists_at_correct_location_homepageview(self):
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_homepage_view(self):
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
        self.assertTemplateUsed(response, "home.html")
        self.assertContains(response, "Home")
```

On the top line, we import `SimpleTestCase` since our homepage does not rely on the database. If it did, we'd have to use `TestCase` instead. Then, we import `reverse` to test our URL and view.

Our test class, `HomePageTests`, has two tests that check the homepage URL returns a 200 status code and that it uses our expected URL name, template, and contains "Home" in the response.

Quit the local server with `Control+c` and then run our tests to confirm everything passes.

Shell

```
(.venv) $ python manage.py test
Found 7 test(s).
Creating test database for alias 'default'...
```


runserver를 실행하고 http://127.0.0.1:8000/의 홈페이지로 이동하여 변경되지 않았는지 확인하세요. 마지막 장의 끝에서 사용했던 로그인된 슈퍼유저 계정의 이름과 나이를 보여야 합니다.

테스트

우리는 새로운 코드와 기능을 추가했으므로 이제 테스트를 할 시간입니다. 프로젝트에 충분한 테스트가 없을 수는 없습니다. 처음 작성하는 데 시간이 걸리더라도, 나중에 시간을 절약해 주고 프로젝트가 복잡해질수록 자신감을 제공합니다.

새로운 홈페이지가 제대로 작동하는지 확인하기 위해 테스트를 추가해 봅시다. 코드는 당신의 pages/tests.py 파일에서 이렇게 보여야 합니다.

```
# pages/tests.py
from django.test import SimpleTestCase
from django.urls import reverse

class HomePageTests(SimpleTestCase):
    def test_url_exists_at_correct_location_homepageview(self):
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_homepage_view(self):
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
        self.assertTemplateUsed(response, "home.html")
        self.assertContains(response, "Home")
```

첫 번째 줄에서는 데이터베이스에 의존하지 않기 때문에 SimpleTestCase를 가져옵니다. 만약 데이터베이스에 의존했다면 TestCase를 사용해야 했을 것입니다. 그런 다음, URL과 뷰를 테스트하기 위해 reverse를 가져옵니다.

우리의 테스트 클래스인 HomePageTests는 홈페이지 URL이 200 상태 코드를 반환하는지, 예상 URL 이름, 템플릿을 사용하고 응답에 "Home"이 포함되어 있는지를 확인하는 두 개의 테스트를 가지고 있습니다.

Control+c로 로컬 서버를 종료한 후 모든 것이 통과하는지 확인하기 위해 테스트를 실행합니다.

Shell

```
(.venv) $ python manage.py test
테스트 7개를 찾았습니다.
'default'에 대한 테스트 데이터베이스를 생성하는 중...
```

```
System check identified no issues (0 silenced).
```

```
.....
```

```
-----  
Ran 7 tests in 0.185s
```

```
OK
```

```
Destroying test database for alias 'default'...
```

Testing Philosophy

There's no limit to what you can test in an application. For example, we *could* also add tests now based on logged-in or logged-out behavior and whether the template displays the proper content. But the 80/20 rule of 80% of consequences coming from 20% of causes applies to testing and most other things in life.

There's no sense in making as many unit tests as possible to test things that will likely never fail, at least for a web application. If we were working on a nuclear reactor, having as many tests as possible would make sense, but the stakes are a bit lower for most websites.

So, while you always want to add tests around new features, it's ok to only have partial test coverage from the beginning. As errors inevitably arise on new Git branches and features, add a test for each so they don't fail again. This approach is known as *regression testing*, where tests are re-run each time there's a new change to ensure that previously developed and tested software performs as expected.

Django's testing suite is well-suited for many unit tests and automatic regression tests, so developers can be confident in the consistency of their projects.

Bootstrap

It's now time to add some style to our application. If you've never used Bootstrap, you're in for a real treat. Much like Django, it accomplishes so much in so little code.

There are two ways to add Bootstrap to a project: download and serve all the files locally or rely on a Content Delivery Network (CDN). The second approach is simpler to implement, provided you have a consistent internet connection, so we'll use it here.

시스템 점검에서 문제가 발견되지 않았습니다 (0개의 무시됨).

-----7개의 테스트를 0.185초에 실행했습니다

확인

'default' 별칭에 대한 테스트 데이터베이스를 삭제하는 중...

테스트 철학

애플리케이션에서 테스트할 수 있는 것에는 한계가 없습니다. 예를 들어, 로그인된 상태나 로그아웃된 상태의 동작 및 템플릿이 적절한 콘텐츠를 표시하는지에 따라 지금 테스트를 추가할 수 있습니다. 그러나 20%의 원인에서 80%의 결과가 발생하는 80/20 법칙은 테스트와 인생의 대부분의 다른 것들에도 적용됩니다. 웹 애플리케이션의 경우, 결코 실패하지 않을 것 같은 것들을 테스트하기 위해 가능한 한 많은 단위 테스트를 만드는 것은 의미가 없습니다. 만약 우리가 원자력 발전소에서 작업하고 있다면 가능한 한 많은 테스트를 하는 것이 의미가 있겠지만, 대부분의 웹사이트의 경우 위험이 조금 더 낮습니다.

따라서 새로운 기능 주위에 테스트를 추가하고 싶지만, 처음부터 부분적인 테스트 커버리지만 있어도 괜찮습니다. 새로운 Git 브랜치와 기능에서 오류가 불가피하게 발생하므로, 다시 실패하지 않도록 각 테스트를 추가하세요. 이 접근 방식은 회귀 테스트로 알려져 있으며, 새로운 변경 사항이 있을 때마다 테스트를 다시 실행하여 이전에 개발되고 테스트된 소프트웨어가 예상대로 작동하는지 확인합니다.

장고의 테스트 스위트는 많은 단위 테스트와 자동 회귀 테스트에 잘 적합하므로 개발자는 프로젝트의 일관성에 대해 확신할 수 있습니다.

부트스트랩

이제 애플리케이션에 스타일을 추가할 시간입니다. 부트스트랩을 사용해본 적이 없다면, 진정한 즐거움을 경험하게 될 것입니다. 장고와 마찬가지로, 매우 적은 코드로 많은 것을 이룰 수 있습니다.

부트스트랩을 프로젝트에 추가하는 방법은 두 가지가 있습니다: 모든 파일을 로컬로 다운로드하여 제공하거나 콘텐츠 전송 네트워크(CDN)에 의존하는 것입니다. 두 번째 접근 방식은 일관된 인터넷 연결이 있는 경우 구현하기가 더 간단하므로 여기에서는 이를 사용하겠습니다.

Our template will mimic the “Starter template” provided on the [Bootstrap introduction](#) page and involves adding the following:

- meta name="viewport" and content information at the top within <head>
- Bootstrap CSS link within <head>
- Bootstrap JavaScript bundle at the bottom of the <body> section

Typing out all code yourself is recommended, but adding the Bootstrap CDN is an exception since it is lengthy and easy to mistype. I recommend copying and pasting the Bootstrap CSS and JavaScript Bundle links from the Bootstrap website into the base.html file.

Code

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Newspaper App{% endblock title %}</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
    bootstrap.min.css" rel="stylesheet" integrity="sha384..."
    crossorigin="anonymous">
</head>
<body>
  <main>
    {% block content %}
    {% endblock content %}
  </main>
  <!-- Bootstrap JavaScript Bundle -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/
    bootstrap.bundle.min.js" integrity="sha384..." crossorigin="anonymous">
  </script>
</body>
</html>
```

This code snippet **does not** include the full links for Bootstrap CSS and JavaScript; it is abbreviated. Copy and paste the full links for Bootstrap 5.3 from the [quick start docs](#).

If you start the server again with `python manage.py runserver` and refresh the homepage at `http://127.0.0.1:8000/`, you’ll see that the font size and link colors have changed.

Let’s add a navigation bar at the top of the page containing our links for the homepage, login, logout, and signup pages. Notably, we can use the [if/else](#) tags

우리의 템플릿은 Bootstrap 소개 페이지에 제공된 "[시작 템플릿](#)"을 모방하며 다음을 추가하는 것을 포함합니다:

- <meta name="viewport" content 정보는 <head>의 맨 위에 위치하고 <head> 내에
- Bootstrap CSS 링크가 포함됩니다.
- <body> 섹션의 맨 아래에 Bootstrap JavaScript 번들이 위치합니다.

모든 코드를 직접 입력하는 것이 권장되지만, Bootstrap CDN을 추가하는 것은 길고 오타가 나기 쉬우므로 예외입니다. Bootstrap 웹사이트에서 Bootstrap CSS 및 JavaScript 번들 링크를 base.html 파일에 복사하여 붙여넣는 것을 추천합니다.

코드

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1"><title>{% block
title %}신문 앱{% endblock title %}</title>
  <!-- 부트스트랩 CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@
5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384... "
crossorigin="
anonymous"></head>
<body>
  <main>
    {% block content %}
    {% endblock content %}
  </main>
  <!-- 부트스트랩 자바스크립트 번들 -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@
5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384... " crossorigin="
anonymous">
</script>
</body>
</html>
```

이 코드 스니펫은 부트스트랩 CSS 및 자바스크립트의 전체 링크를 포함하지 않으며, 축약되어 있습니다. 빠른 시작 문서에서 부트스트랩 5.3의 전체 링크를 복사하여 붙여넣으세요.

서버를 python manage.py runserver로 다시 시작하고 <http://127.0.0.1:8000/>에서 홈페이지를 새로 고치면 글꼴 크기와 링크 색상이 변경된 것을 볼 수 있습니다.

페이지 상단에 홈페이지, 로그인, 로그아웃 및 가입 페이지에 대한 링크가 포함된 탐색 막대를 추가해 보겠습니다. 특히, if/else 태그를 사용할 수 있습니다.

in the Django templating engine to add some basic logic. We want the “log in” and “sign up” buttons to appear for a logged-out user and the “log out” and “change password” buttons when a user is logged in.

Again, it’s ok to copy/paste here since this book focuses on learning Django, not HTML, CSS, and Bootstrap. If there are any formatting issues, you can view the [official GitHub repository](#) for reference.

Code

```
<!-- templates/base.html -->
...
<body>
  <nav class="navbar navbar-expand-lg bg-body-tertiary">
    <div class="container-fluid">
      <a class="navbar-brand" href="{% url 'home' %}">Newspaper</a>
      <button class="navbar-toggler" type="button"
        data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
        aria-controls="navbarSupportedContent" aria-expanded="false"
        aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarSupportedContent">
        <ul class="navbar-nav me-auto mb-2 mb-lg-0">
          <li class="nav-item">
            <a class="nav-link active" aria-current="page" href="#">Home</a>
          </li>
          {% if user.is_authenticated %}
          <li><a href="#" class="nav-link px-2 link-dark">+ New</a></li>
          </ul>
          <div class="mr-auto">
            <ul class="navbar-nav">
              <li class="nav-item dropdown">
                <a class="nav-link dropdown-toggle" href="#" role="button"
                  data-bs-toggle="dropdown" aria-expanded="false">
                  {{ user.username }}
                </a>
                <ul class="dropdown-menu dropdown-menu-end">
                  <li><a class="dropdown-item" href="{% url 'password_change' %}">
                    Change password</a></li>
                  <li>
                    <hr class="dropdown-divider">
                  </li>
                  <li>
                    <form method="post" action="{% url 'logout' %}"
                      style="display:inline;">
                      {% csrf_token %}
                      <button type="submit" class="btn btn-link nav-link"
                        style="display:inline; cursor:pointer;">Logout</button>
                    </form>
                  </li>
                </ul>
              </li>
            </ul>
          </div>
          {% else %}
        </ul>
      </div>
    <div class="mr-auto">
```

장고 템플릿 엔진에서 기본 로직을 추가합니다. 우리는 로그아웃된 사용자에게는 "로그인" 및 "회원가입" 버튼이 나타나고, 사용자가 로그인했을 때는 "로그아웃" 및 "비밀번호 변경" 버튼이 나타나기를 원합니다.

다시 말하지만, 이 책은 Django 학습에 중점을 두고 있으므로 여기서 복사/붙여넣기를 해도 괜찮습니다. HTML, CSS 및 Bootstrap에 대한 것이 아닙니다. 형식에 문제가 있는 경우, 참조를 위해 공식 GitHub 리포지토리를 확인할 수 있습니다.

코드

```
<!-- templates/base.html -->
...
<body>
  <nav class="navbar navbar-expand-lg bg-body-tertiary"><div class="
    container-fluid">
      <a class="navbar-brand" href="{% url 'home' %}">신문</a>
      <button class="navbar-toggler" type="button"
        data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent" aria-
        controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
        navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarSupportedContent">
        <ul class="navbar-nav me-auto mb-2 mb-lg-0"><li
          class="nav-item">
            <a class="nav-link active" aria-current="page" href="#">홈</a></li>

            {% if user.is_authenticated %}
            <li><a href="#" class="nav-link px-2 link-dark">+ 새로 만들기</a></li></ul>

            <div class="mr-auto">
              <ul class="navbar-nav"><li class="
                nav-item dropdown">
                  <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-
                    toggle="dropdown" aria-expanded="false">{{ user.username }}

                  </a>
                  <ul class="dropdown-menu dropdown-menu-end"><li><a class="dropdown-item"
                    href="{% url 'password_change' %}">
                      비밀번호 변경</a></li></ul>

                  <hr class="dropdown-divider"></li>

                  <li>
                    <form method="post" action="{% url 'logout' %}"
                      style="display: inline; ">{%
                      csrf_token %}
                    <button type="submit" class="btn btn-link nav-link" style="display: inline;
                      cursor: pointer; ">로그아웃</button>
                  </form>
                ></li>
              </ul>
            </li>{% else %}
            {%}
          </ul>
        </div>
      </div>
    </div>
  <div class="mr-auto">
```

```

        <form class="form d-flex">
          <a href="{% url 'login' %}" class="btn btn-outline-secondary">Log in</a>
          <a href="{% url 'signup' %}" class="btn btn-primary ms-2">Sign up</a>
        </form>
      </div>
    {% endif %}
  </div>
</div>
</nav>
<main>
  <div class="container">
    {% block content %}
    {% endblock content %}
  </div>
</main>
...

```

If you refresh the homepage at `http://127.0.0.1:8000/`, our new navbar has magically appeared! Note that there are no actual links yet for new articles “+ New”; a placeholder is represented in the code by `href="#"`. We will add that later on. Also, note that our logged-in username is now in the upper right corner, along with a dropdown arrow. If you click on it, there are links for “Change password” and “Log Out.”



Homepage with Bootstrap Nav Logged In

If you click “Log Out” in the dropdown, the navbar changes to button links for either “Log In” or “Sign Up” and the “+ New” link disappears. There is no sense in letting logged out users create articles.


```

<form class="form d-flex">
  <a href="{% url 'login' %}" class="btn btn-outline-secondary">로그인</a>
  <a href="{% url 'signup' %}" class="btn btn-primary ms-2">회원가입</a></form>

</div>
{% endif %}
</div>
</div>
</nav>
<main>

<div class="container">{%
  block content %}{%
    endblock content %}
</div>
</main>
...

```

http://127.0.0.1:8000/에서 홈페이지를 새로 고치면, 우리의 새로운 네비게이션 바가 마법처럼 나타났습니다! 새로운 기사 “+새로 만들기”에 대한 실제 링크는 아직 없다는 점에 유의하세요; 코드에서는 href="#"로 자리 표시자가 나타납니다. 우리는 나중에 그것을 추가할 것입니다. 또한, 로그인한 사용자 이름이 이제 오른쪽 상단 모서리에 드롭다운 화살표와 함께 표시됩니다. 클릭하면 “비밀번호 변경” 및 “로그아웃” 링크가 있습니다.



부트스트랩 네비게이션 로그인된 홈페이지

드롭다운에서 "로그 아웃"을 클릭하면 네비게이션 바가 "로그인" 또는 "회원가입" 버튼 링크로 변경되고 "+ 새로 만들기" 링크가 사라집니다. 로그아웃된 사용자가 기사를 작성하도록 허용하는 것은 의미가 없습니다.



Homepage with Bootstrap Nav Logged Out

If you click on the “Log In” button in the top nav, you can also see that our login page at <http://127.0.0.1:8000/accounts/login/> looks better.



Bootstrap Login

The only thing that looks off is our gray “Log In” button. We can use Bootstrap to add some nice styling, such as making it green and inviting. Change the “button” line in the `templates/registration/login.html` file.

Code

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}

{% block title %}Log In{% endblock title %}

{% block content %}
<h2>Log In</h2>
<form method="post">{% csrf_token %}
  {{ form }}
  <!-- new code here -->
  <button class="btn btn-success ms-2" type="submit">Log In</button>
  <!-- end new code -->
</form>
{% endblock content %}
```

Now refresh the page to see our new button in action.



부트스트랩 내비게이션이 있는 홈페이지 로그아웃 상태

상단 네비게이션의 "로그인" 버튼을 클릭하면 <http://127.0.0.1:8000/accounts/login>에 있는 로그인 페이지가 더 보기 좋다는 것을 알 수 있습니다.



부트스트랩 로그인

유일하게 이상해 보이는 것은 우리의 회색 "로그인" 버튼입니다. Bootstrap을 사용하여 초록색으로 바꾸고 더 매력적으로 만드는 등 멋진 스타일을 추가할 수 있습니다. 변경하세요.

templates/registration/login.html 파일의 "button" 줄. 코드

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}

{% block title %}로그인{% endblock title %}{% block
content %}

<h2>로그인</h2>
<form method="post">{% csrf_token %}{% form
  %}
  <!-- 새로운 코드 여기 -->
  <button class="btn btn-success ms-2" type="submit">로그인</button>
  <!-- 새로운 코드 끝 -->
</form>
{% endblock content %}
```

이제 페이지를 새로 고쳐서 우리의 새로운 버튼이 작동하는 것을 보세요.



Bootstrap Login with New Button

Signup Form

If you click on the link for “Sign Up” you’ll see that the page has Bootstrap stylings and distracting helper text. For example, after “Username” it says “Required. 150 characters or fewer. Letters, digits, and @/./+/-/_ only.” So, where did that text come from? Whenever something feels like “magic” in Django, rest assured that it is decidedly not. If you did not write the code, it exists somewhere within Django.

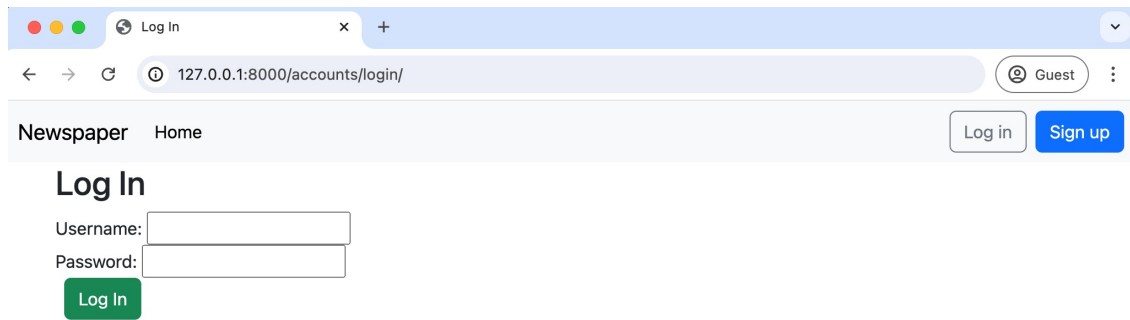
The best method to figure out what’s happening under the hood in Django is to download the source code and take a look yourself. All the code development occurs on GitHub, and you can find the Django repo at <https://github.com/django/django/>. To copy it onto your local computer, open a new tab (Command + t) on your command line and navigate to the desired location. Let’s navigate there now since we’ve been using a code folder on the desktop.

Shell

```
# Windows
$ cd onedrive\desktop\code

# macOS
$ cd ~/desktop/code
```

On the GitHub website, you’ll see a green “<> Code” button. Click it and select “GitHub CLI” to view the command line instructions to download the repo.



부트스트랩 로그인과 새 버튼

가입 양식

“가입하기” 링크를 클릭하면 페이지에 부트스트랩 스타일과 주의가 산만해지는 도움말 텍스트가 표시됩니다. 예를 들어, “사용자 이름” 뒤에는 “필수. 150자 이하. 문자, 숫자 및 @/./+/-/_만 가능합니다.”라고 적혀 있습니다. 그 텍스트는 어디서 왔을까요? Django에서 무언가가 “마법”처럼 느껴질 때, 그것은 확실히 그렇지 않다는 것을 확신하세요. 당신이 코드를 작성하지 않았다면, 그것은 Django의 어딘가에 존재합니다.

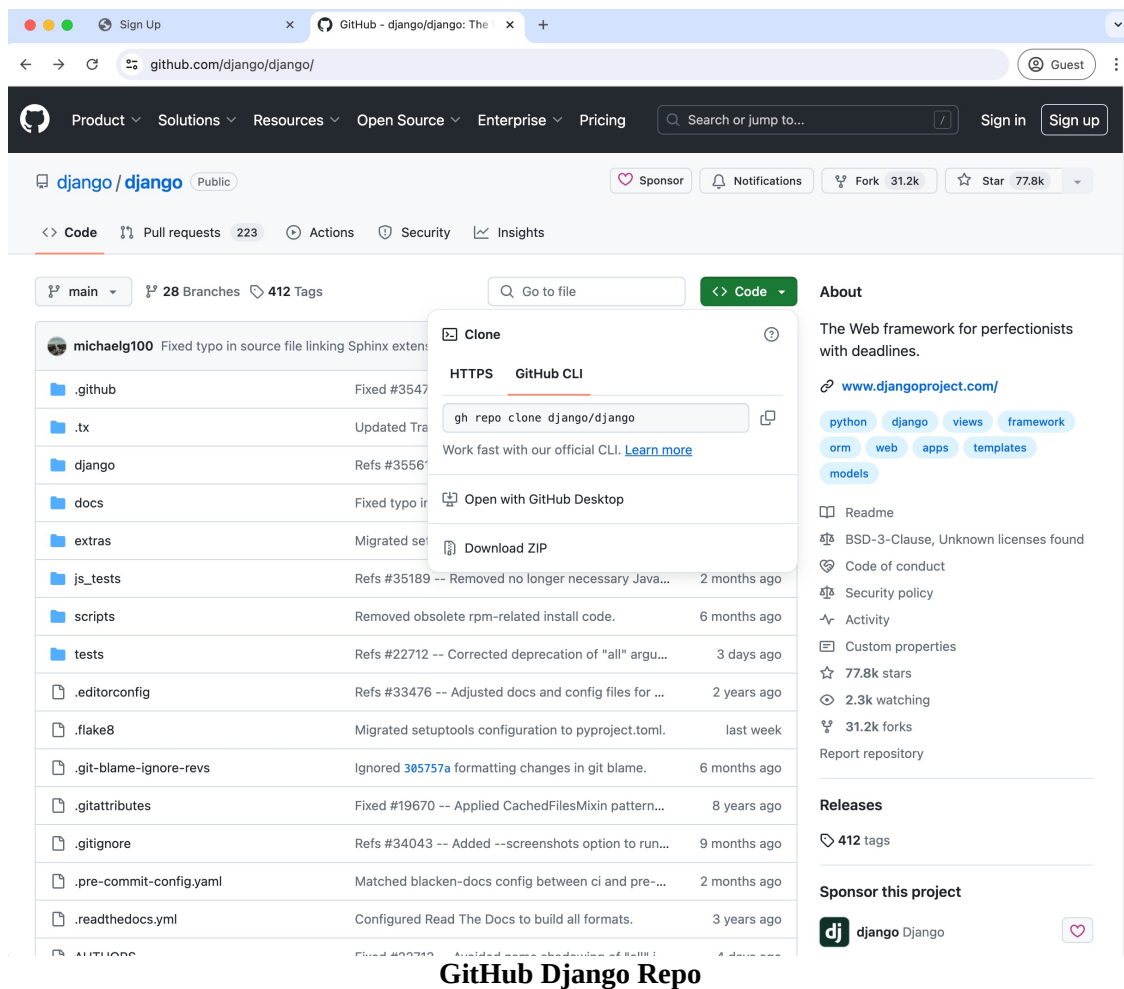
Django에서 내부에서 무슨 일이 일어나고 있는지 알아내는 가장 좋은 방법은 소스 코드를 다운로드하고 직접 살펴보는 것입니다. 모든 코드 개발은 GitHub에서 이루어지며, Django 레포는 <https://github.com/django/django/> 에서 찾을 수 있습니다. 로컬 컴퓨터에 복사하려면, 명령줄에서 새 탭(커맨드 + t)을 열고 원하는 위치로 이동하세요. 우리는 바탕화면의 코드 폴더를 사용하고 있으므로 지금 그곳으로 이동해 보겠습니다.

셸

```
# Windows$ cd onedrive\desktop\code
```

```
# macOS$ cd ~/desktop/code
```

GitHub 웹사이트에서 녹색 “<> 코드” 버튼을 볼 수 있습니다. 클릭하고 “GitHub CLI”를 선택하여 레포를 다운로드하는 명령줄 지침을 확인하세요.



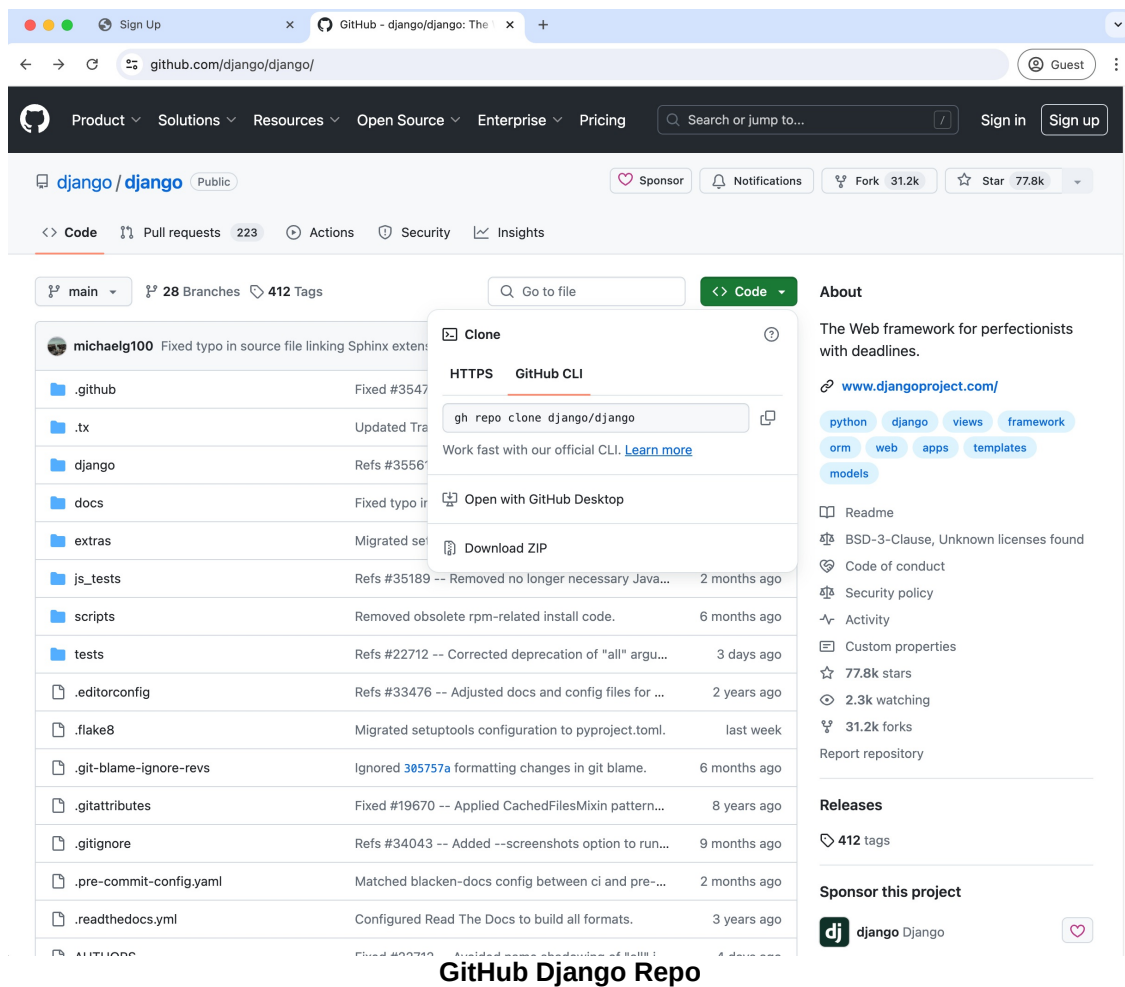
GitHub Django Repo

On the command line, type `gh repo clone django/django` to copy the Django source code onto your computer in a new directory called `django`.

Shell

```
$ gh repo clone django/django
```

Having the Django source code on your computer will require manual updates now and then to stay current. After all, Django undergoes regular updates to fix bugs, improve security, and add new features. Why not just use the built-in GitHub search? Searching on GitHub *sometimes* works, but lately, it has been inconsistent, something GitHub is well aware of and working to improve. Downloading the source code and searching yourself are valuable tools, so it is worth taking the time to learn how to do so, as we are here.



GitHub Django Repo

명령줄에서 `gh repo clone django/django`를 입력하여 Django의 소스 코드를 `django.Shell`이라는 새 디렉토리에 컴퓨터에 복사합니다.

```
$ gh repo clone django/django
```

컴퓨터에 Django 소스 코드가 있으면 최신 상태를 유지하기 위해 수동 업데이트가 필요합니다. 결국, Django는 버그를 수정하고 보안을 개선하며 새로운 기능을 추가하기 위해 정기적으로 업데이트됩니다. 내장된 GitHub 검색을 사용하는 것이 좋지 않을까요? GitHub에서 검색하는 것은 때때로 작동하지만 최근에는 일관성이 없었습니다. GitHub은 이를 잘 알고 있으며 개선하기 위해 노력하고 있습니다. 소스 코드를 다운로드하고 직접 검색하는 것은 유용한 도구이므로, 그렇게 하는 방법을 배우는 데 시간을 투자할 가치가 있습니다. 우리는 여기 있습니다.

In your text editor, open the Django source code to perform searches. For example, press the keys command + shift + f in the VS Code text editor to do a “find” search in all files. Type in a search for “150 characters or fewer” and you’ll find the top link to the page for `django/contrib/auth/models.py`. The specific code is on line 350, and the text is part of the auth app, on the username field for `AbstractUser`.

Note: We now have two command line tabs open: one for our project code and one for the Django source code. Make sure you understand which is which. In the future, we *will* do additional searches on the source code, but all code in this book requires switching *back* to the terminal tab with the project source code. Switch back as we are about to add more code to our project.

We have three options now:

- override the existing `help_text`
- hide the `help_text`
- restyle the `help_text`

We’ll choose the third option since it’s a good way to introduce the excellent 3rd party package [django-crispy-forms](#).

Working with forms is challenging, and `django-crispy-forms` makes writing DRY (Don’t Repeat-Yourself) code easier. First, stop the local server with `Control+c`. Then use `pip` to install the package in our project. We’ll also install the [Bootstrap5 template pack](#).

Shell

```
(.venv) $ python -m pip install django-crispy-forms==2.2
(.venv) $ python -m pip install crispy-bootstrap5==2024.2
```

Add the new apps to our `INSTALLED_APPS` list in the `django_project/settings.py` file. As the number of apps starts to grow, it can be helpful to distinguish between “3rd party” apps and “local” apps. Here’s what the code looks like now.

Code

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
```


텍스트 편집기에서 Django 소스 코드를 열어 검색을 수행하세요. 예를 들어, VS Code 텍스트 편집기에서 `command + shift + f` 키를 눌러 모든 파일에서 "찾기" 검색을 할 수 있습니다. "150 characters or fewer"를 검색하면 `django/contrib/auth/models.py` 페이지에 대한 상위 링크를 찾을 수 있습니다. 해당 코드는 350번째 줄에 있으며, 텍스트는 auth 앱의 `AbstractUser`의 `username` 필드에 속해 있습니다.

참고: 현재 두 개의 명령줄 탭이 열려 있습니다: 하나는 우리 프로젝트 코드용이고 다른 하나는 Django 소스 코드용입니다. 각각이 어떤 것인지 확실히 이해하세요. 앞으로 소스 코드에서 추가 검색을 할 예정이지만, 이 책의 모든 코드는 프로젝트 소스 코드가 있는 터미널 탭으로 다시 전환해야 합니다. 프로젝트에 더 많은 코드를 추가할 예정이니 다시 전환하세요.

우리는 지금 세 가지 선택지가 있습니다:

- 기존 `help_text`를 덮어쓰기
- 도움말 숨기기 도움말 스타일 변경
-

우리는 훌륭한 서드파티 패키지 `django-crispy-forms`를 소개하는 좋은 방법이기에 세 번째 옵션을 선택할 것입니다.

폼 작업은 어려운데, `django-crispy-forms`는 DRY(반복하지 않기) 코드를 작성하는 것을 더 쉽게 만들어줍니다. 먼저, `Control+c`로 로컬 서버를 중지하세요. 그런 다음 `pip`를 사용하여 프로젝트에 패키지를 설치합니다. 또한 Bootstrap5 템플릿 팩도 설치할 것입니다.

셀

```
(.venv) $ python -m pip install django-crispy-forms==2.2(.venv) $  
python -m pip install crispy-bootstrap5==2024.2
```

새로운 앱들을 `thedjango_project/settings.py` 파일의 `INSTALLED_APPS` 목록에 추가하세요. 앱의 수가 늘어나기 시작하면 "서드파티" 앱과 "로컬" 앱을 구분하는 것이 도움이 될 수 있습니다. 현재 코드가 어떻게 생겼는지 아래에 나와 있습니다.

코드

```
# django_project/settings.py  
INSTALLED_APPS = [  
    django.contrib.admin", "  
    django.contrib.auth", "  
    django.contrib.contenttypes", "  
    django.contrib.sessions",
```

```
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # 3rd Party
    "crispy_forms", # new
    "crispy_bootstrap5", # new
    # Local
    "accounts",
    "pages",
]
```

And then, at the bottom of the `settings.py` file, add two new lines as well.

Code

```
# django_project/settings.py
CRISPY_ALLOWED_TEMPLATE_PACKS = "bootstrap5" # new
CRISPY_TEMPLATE_PACK = "bootstrap5" # new
```

Now in our `signup.html` template, we can quickly use crispy forms. First, we load `crispy_forms_tags` at the top and then swap out `{{ form }}` for `{{ form|crispy }}`. We'll also update the “Sign Up” button to be green with the `btn-success` styling.

Code

```
<!-- templates/registration/signup.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block title %}Sign Up{% endblock title %}

{% block content %}
<h2>Sign Up</h2>
<form method="post">{% csrf_token %}
    {{ form|crispy }}
    <button class="btn btn-success" type="submit">Sign Up</button>
</form>
{% endblock content %}
```

We can see the new changes if you start the server again with `python manage.py runserver` and refresh the signup page.

```
"django.contrib.messages", "  
django.contrib.staticfiles",  
# 3rd Party"crispy_forms", #  
new"crispy_bootstrap5", # new#  
Local  
  
"accounts", "  
pages",  
]
```

그런 다음 settings.py 파일의 맨 아래에 두 개의 새로운 줄을 추가합니다.코드

```
# django_project/settings.py  
CRISPY_ALLOWED_TEMPLATE_PACKS = "bootstrap5" # new  
CRISPY_TEMPLATE_PACK = "bootstrap5" # new
```

이제 signup.html 템플릿에서 crispy forms를 빠르게 사용할 수 있습니다. 먼저, 상단에서 crispy_forms_tags를 로드한 다음 {{ form }}을 {{form|crispy }}로 교체합니다. “가입” 버튼을 btn-success 스타일로 초록색으로 업데이트할 것입니다.

코드

```
<!-- templates/registration/signup.html -->  
{% extends "base.html" %}{% load  
crispy_forms_tags %}  
  
{% block title %}회원 가입{% endblock title %}{% block  
content %}  
  
<h2>회원 가입</h2>  
<form method="post">{% csrf_token %}  
  {{ form|crispy }}  
  <button class="btn btn-success" type="submit">회원 가입</button></form>  
  
{% endblock content %}
```

서버를 다시 시작하고 python manage.pyrunserver로 새로 고침하면 새로운 변경 사항을 볼 수 있습니다.

Sign Up

Username*

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address

Age

Password*

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation*

Enter the same password as before, for verification.

Sign Up

Crispy Signup Page

We can also add crispy forms to our login page. The process is the same. Here is that updated code:

Code

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block title %}Log In{% endblock title %}

{% block content %}
<h2>Log In</h2>
<form method="post">{% csrf_token %}
  {{ form|crispy }}
  <button class="btn btn-success ms-2" type="submit">Log In</button>
</form>
{% endblock content %}
```

Refresh the login page, and the update will be visible.

Sign Up

Username*

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address

Age

Password*

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation*

Enter the same password as before, for verification.

Sign Up

바삭한 가입 페이지

로그인 페이지에 바삭한 양식을 추가할 수도 있습니다. 과정은 동일합니다. 업데이트된 코드는 다음과 같습니다:

코드

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}{% load
crispy_forms_tags %}

{% block title %}로그인{% endblock title %}{% block
content %}

<h2>로그인</h2>
<form method="post">{% csrf_token %}
  {{ form|crispy }}
  <button class="btn btn-success ms-2" type="submit">로그인</button></form>

{% endblock content %}
```

로그인 페이지를 새로 고치면 업데이트가 표시됩니다.



Crispy Login Page

Git and requirements.txt

We have now added several packages to our Django project so it is a good time to create a `requirements.txt` file.

Shell

```
(.venv) $ pip freeze > requirements.txt
```

At the moment, this is how mine looks:

Code

```
# requirements.txt
asgiref==3.8.1
black==24.4.2
click==8.1.7
crispy-bootstrap5==2024.2
Django==5.0.6
django-crispy-forms==2.2
mypy-extensions==1.0.0
packaging==24.1
pathspec==0.12.1
platformdirs==4.2.2
sqlparse==0.5.0
```

Then we can add a quick Git commit to save our work in this chapter and store it on GitHub.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "add Bootstrap styling"
(.venv) $ git push origin main
```



크리스피 로그인 페이지

Git 및 requirements.txt

이제 우리는 Django 프로젝트에 여러 패키지를 추가했으므로 좋은 시점입니다 requirements.txt 파일을

```
(.venv) $ pip freeze > requirements.txt
```

현재 제 모습은 다음과 같습니다:

```
#
requirements.txtasgiref
==3.8.1black==
24.4.2click==8.1.7
crispy-bootstrap5==
2024.2Django==5.0.6
django-crispy-forms==
2.2mypy-extensions==
1.0.0packaging==
24.1pathspec==
0.12.1platformdirs==
4.2.2sqlparse==0.5.0
```

생성하기 위해
shell

그럼 이 장에서 작업을 저장하고 GitHub에 저장하기 위해 빠른 Git 커밋을 추가할 수 있습니다.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "add Bootstrap styling" (.venv) $
git push origin main
```

Conclusion

Our *Newspaper* app looks good. To improve the look of our forms, we added Bootstrap and Django Crispy Forms to our website. The last step of our user authentication flow is configuring password change and reset. Again, Django has taken care of the heavy lifting, requiring a minimal amount of code on our part.

결론

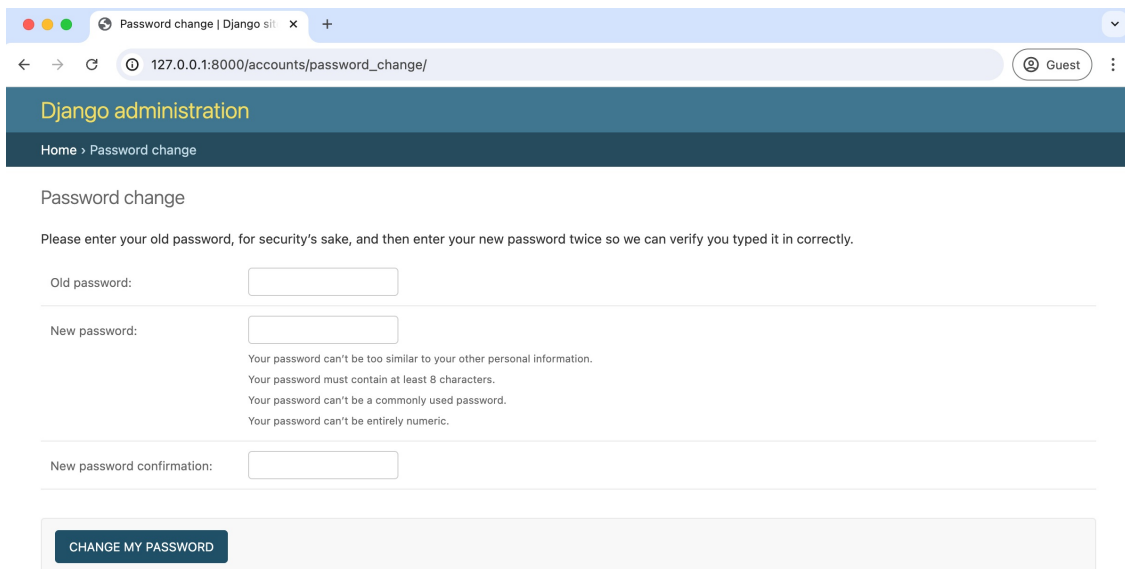
우리 신문 앱은 좋아 보입니다. 양식의 모양을 개선하기 위해 Bootstrap과 Django Crispy Forms를 웹사이트에 추가했습니다. 사용자 인증 흐름의 마지막 단계는 비밀번호 변경 및 재설정 구성입니다. 다시 말해, Django가 많은 작업을 처리하여 우리 쪽에서 최소한의 코드만 필요로 합니다.

Chapter 12: Password Change and Reset

In this chapter, we will complete the authorization flow of our *Newspaper* app by adding password change and password reset functionality. We'll implement Django's built-in views and URLs for password changes and resets before customizing them with our own Bootstrap-powered templates.

Password Change

Many websites allow users to change their passwords, and Django provides a default implementation that already works at this stage. To try it out, click the “Log In” button to ensure you’re logged in. Then, navigate to the “Password change” page at `http://127.0.0.1:8000/accounts/password_change/`.

A screenshot of a web browser showing the Django password change page. The browser's address bar displays the URL `127.0.0.1:8000/accounts/password_change/`. The page has a dark blue header with the text "Django administration" and a breadcrumb trail "Home > Password change". Below the header, the page title is "Password change". A message states: "Please enter your old password, for security's sake, and then enter your new password twice so we can verify you typed it in correctly." There are three input fields: "Old password:", "New password:", and "New password confirmation:". The "New password:" field has four lines of feedback text: "Your password can't be too similar to your other personal information.", "Your password must contain at least 8 characters.", "Your password can't be a commonly used password.", and "Your password can't be entirely numeric." At the bottom, there is a button labeled "CHANGE MY PASSWORD".

Password Change

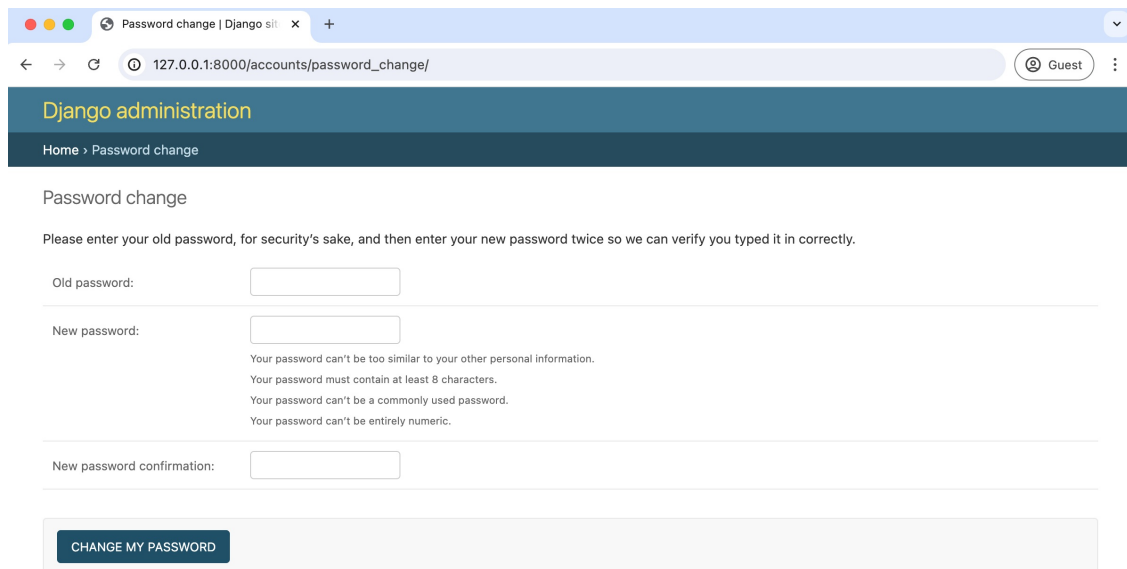
Enter your old password and a new one. Then click the “Change My Password” button, and you will be redirected to the “Password change successful” page.

12장: 비밀번호 변경 및 재설정

이 장에서는 비밀번호 변경 및 비밀번호 재설정 기능을 추가하여 신문 앱의 인증 흐름을 완료합니다. 비밀번호 변경 및 재설정을 위한 Django의 내장 뷰와 URL을 구현한 후, 이를 Bootstrap 기반의 템플릿으로 사용자 정의할 것입니다.

비밀번호 변경

많은 웹사이트에서 사용자가 비밀번호를 변경할 수 있도록 허용하며, Django는 이 단계에서 이미 작동하는 기본 구현을 제공합니다. 사용해 보려면 "로그인" 버튼을 클릭하여 로그인되어 있는지 확인하세요. 그런 다음 http://127.0.0.1:8000/accounts/password_change/에서 "비밀번호 변경" 페이지로 이동합니다.



Password change | Django site x +

127.0.0.1:8000/accounts/password_change/ Guest

Django administration

Home > Password change

Password change

Please enter your old password, for security's sake, and then enter your new password twice so we can verify you typed it in correctly.

Old password:

New password:

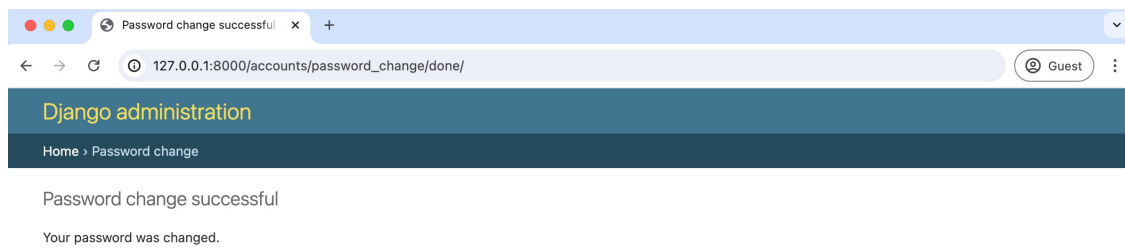
Your password can't be too similar to your other personal information.
Your password must contain at least 8 characters.
Your password can't be a commonly used password.
Your password can't be entirely numeric.

New password confirmation:

CHANGE MY PASSWORD

비밀번호 변경

이전 비밀번호와 새 비밀번호를 입력하세요. 그런 다음 "내 비밀번호 변경" 버튼을 클릭하면 "비밀번호 변경 성공" 페이지로 리디렉션됩니다.



Password Change Done

Customizing Password Change

Let's customize these two password change pages to match the look and feel of our *Newspaper* site. Because Django already has created the views and URLs for us, we only need to change the templates; however, we must use the names `password_change_form.html` and `password_change_done.html`. In your text editor, create two new template files in the registration directory:

- `templates/registration/password_change_form.html`
- `templates/registration/password_change_done.html`

Update `password_change_form.html` with the following code. At the top, we extend `base.html`, load crispy forms, and set our page meta title, which appears in the tab of a web browser but not on the visible webpage itself. The form uses POST since we send data, a `csrf_token` for security reasons, and `{{ form|crispy }}` to use crispy forms styling. As a final tweak, we include a submit button that uses Bootstrap's `btn btn-success` styling to make it green.

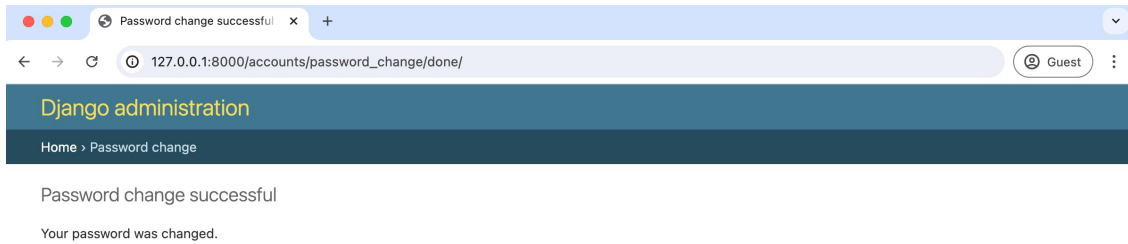
Code

```
<!-- templates/registration/password_change_form.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block title %}Password Change{% endblock title %}

{% block content %}
<h1>Password change</h1>
<p>Please enter your old password, for security's sake, and then enter
your new password twice so we can verify you typed it in correctly.</p>

<form method="POST">{% csrf_token %}
  {{ form|crispy }}
  <input class="btn btn-success" type="submit"
    value="Change my password">
</form>
{% endblock content %}
```



비밀번호 변경 완료

비밀번호 변경 사용자 정의

신문 사이트의 모양과 느낌에 맞게 이 두 비밀번호 변경 페이지를 사용자 정의해 보겠습니다. Django가 이미 우리를 위해 뷰와 URL을 생성했기 때문에, 우리는 템플릿만 변경하면 됩니다. 그러나 `password_change_form.html` 및 `password_change_done.html`이라는 이름을 사용해야 합니다. 텍스트 편집기에서 `registration` 디렉토리에 두 개의 새로운 템플릿 파일을 만드세요:

- `templates/registration/password_change_form.html`
- `templates/registration/password_change_done.html`

다음 코드를 사용하여 `password_change_form.html`을 업데이트합니다. 상단에서 `base.html`을 확장하고, `crispy forms`를 로드하며, 웹 브라우저의 탭에 표시되지만 웹 페이지 자체에는 표시되지 않는 페이지 메타 제목을 설정합니다. 이 양식은 데이터를 전송하므로 POST를 사용하고, 보안상의 이유로 `csrf_token`을 사용하며, `crispy forms` 스타일링을 사용하기 위해 `{{ form|crispy }}`를 사용합니다. 마지막으로, 우리는 부트스트랩의 `btn btn-success` 스타일링을 사용하여 초록색으로 만들기 위한 제출 버튼을 포함합니다. 코드

```
<!-- templates/registration/password_change_form.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

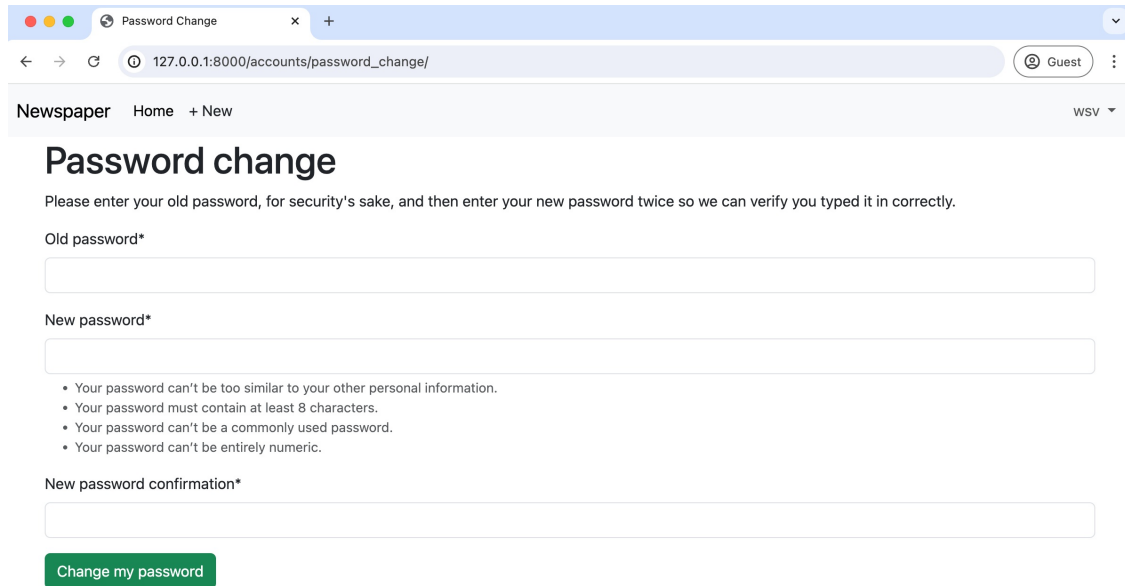
{% block title %}비밀번호 변경{% endblock title %}
{% block content %}

<h1>비밀번호 변경</h1>
<p>보안을 위해 이전 비밀번호를 입력한 다음, 새 비밀번호를 두 번 입력하여 올바르게 입력했는지 확인해 주세요.</p>

<form method="POST">
  {{ csrf_token }}
  {{ form|crispy }}
  <input class="btn btn-success" type="submit" value="비밀번호 변경">
</form>

{% endblock content %}
```

Load the page at http://127.0.0.1:8000/accounts/password_change/ to see our changes.



The screenshot shows a web browser window with the title 'Password Change'. The address bar displays '127.0.0.1:8000/accounts/password_change/'. The page has a header with 'Newspaper', 'Home', and '+ New' links, and a 'Guest' user profile. The main content area is titled 'Password change' and contains a message: 'Please enter your old password, for security's sake, and then enter your new password twice so we can verify you typed it in correctly.' Below this are three input fields: 'Old password*', 'New password*', and 'New password confirmation*'. The 'New password*' field is followed by a list of password requirements: 'Your password can't be too similar to your other personal information.', 'Your password must contain at least 8 characters.', 'Your password can't be a commonly used password.', and 'Your password can't be entirely numeric.' At the bottom is a green button labeled 'Change my password'.

New Password Change Form

Next up is the `password_change_done` template. It also extends `base.html` and includes a new meta title; however, there's no form on the page, just new text.

Code

```
<!-- templates/registration/password_change_done.html -->
{% extends "base.html" %}

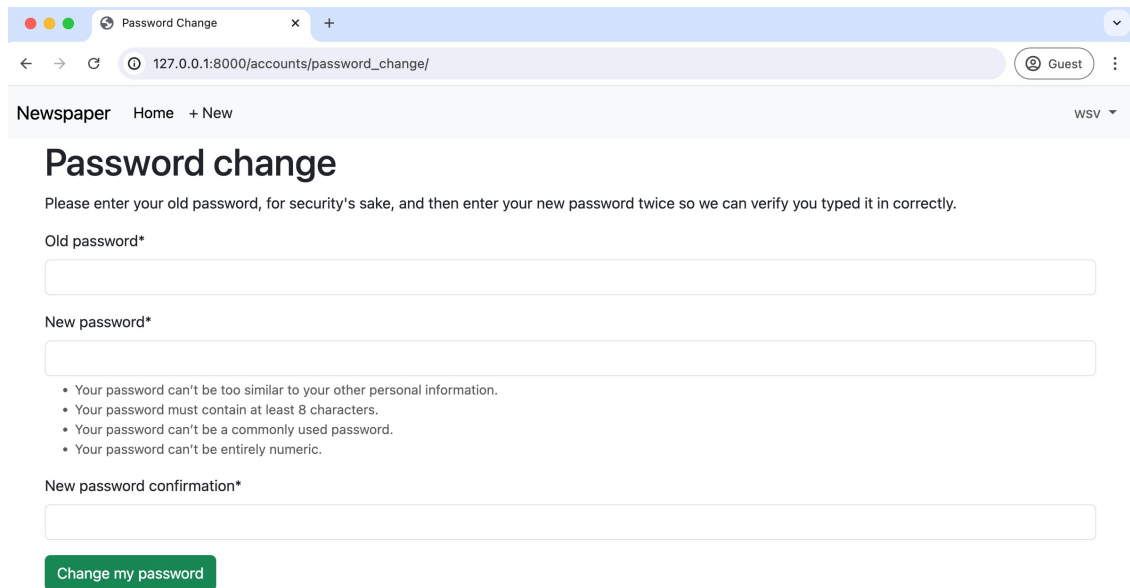
{% block title %}Password Change Successful{% endblock title %}

{% block content %}
<h1>Password change successful</h1>
<p>Your password was changed.</p>
{% endblock content %}
```

This updated page is at:

http://127.0.0.1:8000/accounts/password_change/done/

http://127.0.0.1:8000/accounts/password_change/에서 변경 사항을 확인하세요.



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/accounts/password_change/". The page title is "Password Change". Below the title, there is a message: "Please enter your old password, for security's sake, and then enter your new password twice so we can verify you typed it in correctly." The form consists of three input fields: "Old password*", "New password*", and "New password confirmation*". Below the "New password*" field, there are four bullet points providing password requirements: "Your password can't be too similar to your other personal information.", "Your password must contain at least 8 characters.", "Your password can't be a commonly used password.", and "Your password can't be entirely numeric." At the bottom of the form is a green button labeled "Change my password".

새 비밀번호 변경 양식

다음은 password_change_done 템플릿입니다. 이 템플릿은 base.html을 확장하며 새로운 메타 제목을 포함하지만, 페이지에는 양식이 없고 새로운 텍스트만 있습니다.

코드

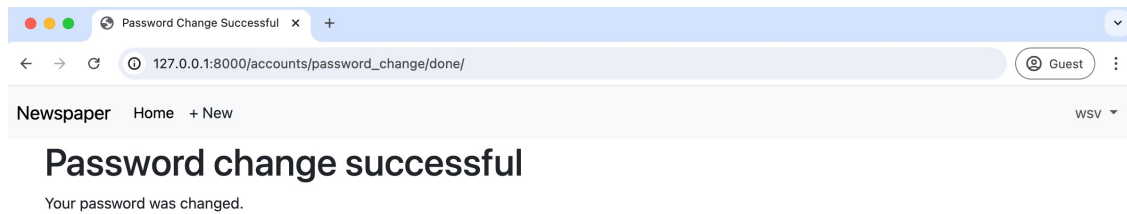
```
<!-- templates/registration/password_change_done.html -->
{% extends "base.html" %}

{% block title %}비밀번호 변경 성공{% endblock title %}

{% block content %}
<h1>비밀번호 변경 성공</h1><p>비밀번호가
변경되었습니다.</p>{% endblock content %}
```

이 업데이트된 페이지는 다음과 같습니다:

http://127.0.0.1:8000/accounts/password_change/done/



New Password Change Done

That wasn't too bad, right? Certainly, it was much less work than creating everything from scratch, especially all the code around securely updating a user's password. Next up is the password reset functionality.

Password Reset

Password reset handles the typical case of users forgetting their passwords. The steps are very similar to configuring password change, as we just did. Django already provides a default implementation that we will use, and then we will customize the templates to match the look and feel of the rest of our site.

The only configuration required is telling Django how to send emails. After all, a user can only reset a password if they can access the email linked to the account. For testing purposes, we can rely on Django's [console backend](#) setting, which outputs the email text to our command-line console instead.

Add the following one-line change at the bottom of the `django_project/settings.py` file.

Code

```
# django_project/settings.py
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend" # new
```

And we're all set! Django will take care of all the rest for us. Let's try it out. Navigate to `http://127.0.0.1:8000/accounts/password_reset/` to view the default password reset page.



새 비밀번호 변경 완료

그렇게 나쁘지 않았죠? 확실히, 모든 것을 처음부터 만드는 것보다 훨씬 적은 작업이었습니다. 특히 사용자의 비밀번호를 안전하게 업데이트하는 코드와 관련된 모든 작업이요. 다음은 비밀번호 재설정 기능입니다.

비밀번호 재설정

비밀번호 재설정은 사용자가 비밀번호를 잊어버리는 일반적인 경우를 처리합니다. 단계는 우리가 방금 했던 비밀번호 변경 구성과 매우 유사합니다. Django는 우리가 사용할 기본 구현을 이미 제공하며, 그 후에 나머지 사이트의 모양과 느낌에 맞게 템플릿을 사용자 정의할 것입니다.

필요한 유일한 구성은 Django에 이메일을 보내는 방법을 알려주는 것입니다. 결국, 사용자는 계정에 연결된 이메일에 접근할 수 있어야만 비밀번호를 재설정할 수 있습니다. 테스트 목적으로, 우리는 [이메일 텍스트를 명령줄 콘솔에 출력하는 Django의 콘솔 백엔드 설정에](#) 의존할 수 있습니다.

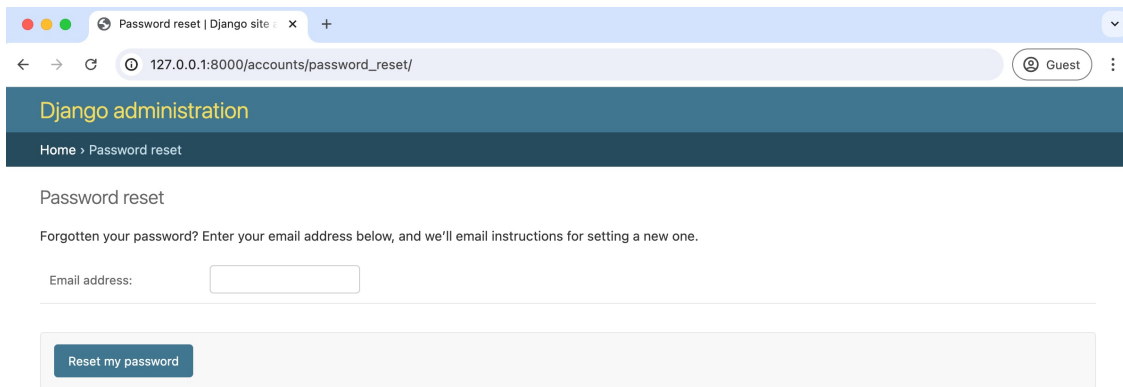
다음의 한 줄 변경을 추가하세요

`django_project/settings.py`

```
# django_project/settings.py
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend" # 새로 추가
```

모두 준비되었습니다! 장고가 나머지를 모두 처리해 줄 것입니다. 한번 시도해 봅시다. http://127.0.0.1:8000/accounts/password_reset/로 이동하여 기본 비밀번호 재설정 페이지를 확인하세요.

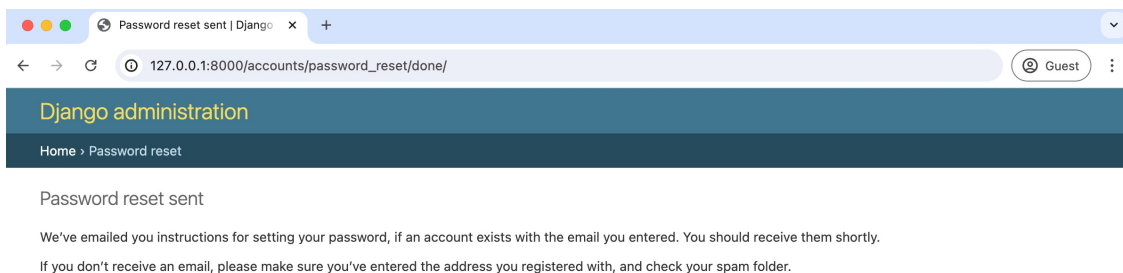
파일의
바라
에
·
코드



Default password reset page

Make sure the email address you enter matches one of your existing user accounts. Recall that `testuser` does not have a linked email account, so you should use `testuser2`, which, if you follow my example, has an email address of `testuser2@email.com`. Upon submission, you'll then be redirected to the password reset done page at:

`http://127.0.0.1:8000/accounts/password_reset/done/`

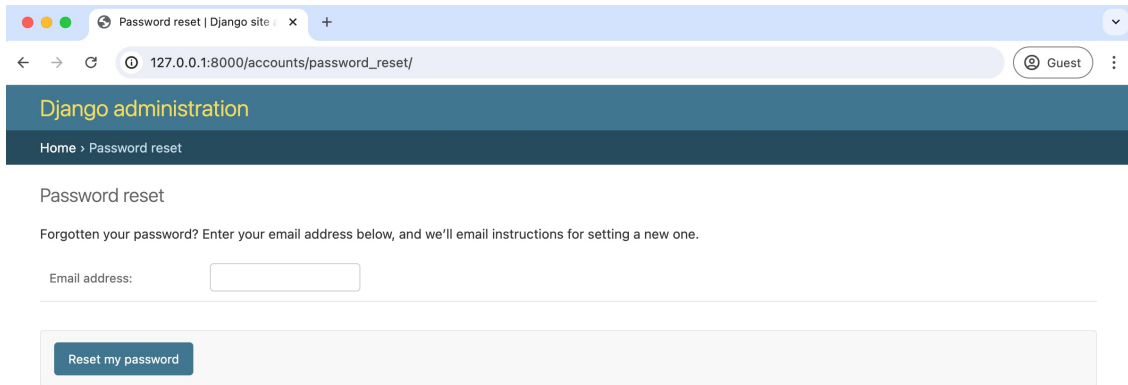


Default Password Reset Done Page

This page says to check our email. Since we've told Django to send emails to the command line console, the email text will now be there. In my console, I see the following:

Shell

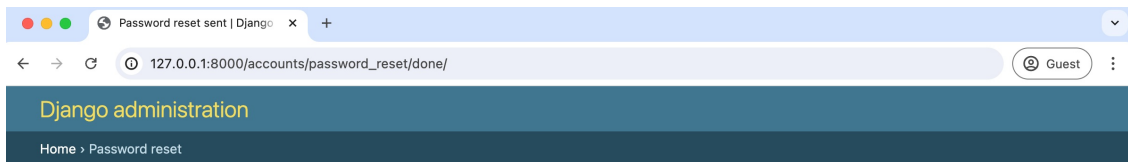
```
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
Subject: Password reset on 127.0.0.1:8000
From: webmaster@localhost
To: testuser2@email.com
```



기본 비밀번호 재설정 페이지

입력하는 이메일 주소가 기존 사용자 계정 중 하나와 일치하는지 확인하세요. testuser는 연결된 이메일 계정이 없으므로, 제 예를 따르면 testuser2를 사용해야 하며, testuser2의 이메일 주소는 testuser2@email.com입니다. 제출 후, 비밀번호 재설정 완료 페이지로 리디렉션됩니다:

http://127.0.0.1:8000/accounts/password_reset/done/



기본 비밀번호 재설정 완료 페이지

이 페이지는 이메일을 확인하라고 합니다. Django에 이메일을 명령줄 콘솔로 보내도록 설정했기 때문에 이메일 텍스트가 이제 그곳에 있을 것입니다. 제 콘솔에서는 다음과 같은 내용을 확인할 수 있습니다:

셸

```
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
Subject: 127.0.0.1:8000에서 비밀번호 재설정
From: webmaster@localhost
To: testuser2@email.com
```

Date: Fri, 28 Jun 2024 15:36:30 -0000

Message-ID:

<168235059067.94136.7639058137157368290@1.0.0.0...0.0.0.ip6.arpa>

You're receiving this email because you requested a password reset for your user account at 127.0.0.1:8000.

Please go to the following page and choose a new password:

<http://127.0.0.1:8000/accounts/reset/Mw/bn63cu-b77b6590a2e38a75c4b2af244c40297e/>

Your username, in case you've forgotten: testuser2

Thanks for using our site!

The 127.0.0.1:8000 team

```
[11/Jun/2024 15:36:30] "POST /accounts/password_reset/ HTTP/1.1" 302 0
[11/Jun/2024 15:36:30] "GET /accounts/password_reset/done/ HTTP/1.1" 200 3014
```

Your email text should be identical except for three lines:

- the “To” on the sixth line contains the email address of the user
- the URL link contains a secure token that Django randomly generates for us and can be used only once
- the username which we’re helpfully reminded of by Django

We will customize all of the default email text shortly, but for now, focus on finding the link provided and entering it into your web browser. In this example, mine is:

<http://127.0.0.1:8000/accounts/reset/Mw/bn63cu-b77b6590a2e38a75c4b2af244c40297e/>

You’ll be redirected to the “Password reset confirmation” page.

날짜: 금, 2024년 6월 28일 15:36:30 -0000메시지-ID:
<168235059067.94136.7639058137157368290@1.0.0.0...0.0.0.ip6.arpa>

이 이메일을 받는 이유는 127.0.0.1:8000에서 사용자 계정의 비밀번호 재설정을 요청했기 때문입니다.

다음 페이지로 이동하여 새 비밀번호를 선택하세요

요: http://127.0.0.1:8000/accounts/reset/Mw/bn63cu-b77b6590a2e38a75c4b2af244c40297e/비밀

번호를 잊어버린 경우 사용자 이름: testuser2

저희 사이트를 이용해 주셔서 감사

합니다!127.0.0.1:8000 팀

- [11/Jun/2024 15:36:30] "POST /accounts/password_reset/ HTTP/1.1" 302 0[11/Jun/2024 15:36:30] "GET /accounts/password_reset/done/ HTTP/1.1" 200 3014

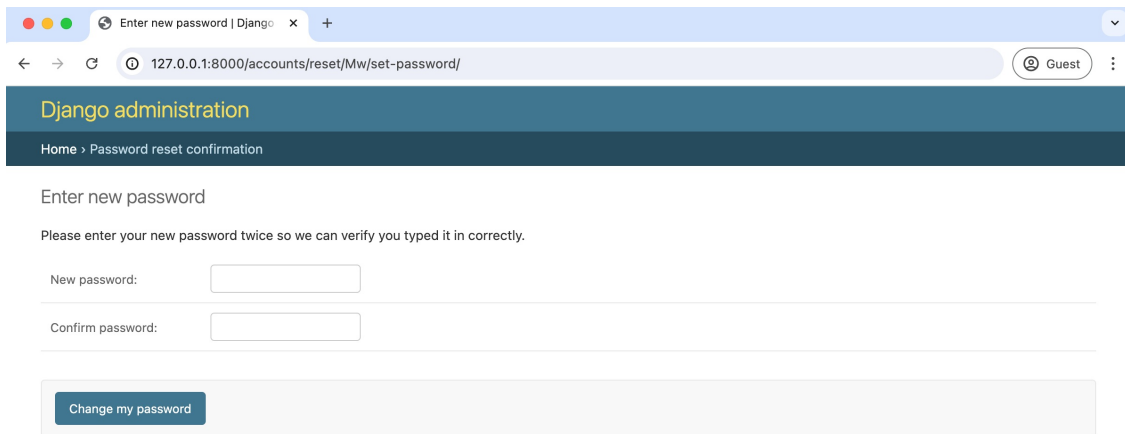
귀하의 이메일 텍스트는 세 줄을 제외하고는 동일해야 합니다:

- 여섯 번째 줄의 “받는 사람”에는 사용자의 이메일 주소가 포함되어 있습니다.
- URL 링크에는 Django가 무작위로 생성한 보안 토큰이 포함되어 있으며 한 번만 사용할 수 있습니다.
- Django가 친절하게 상기시켜주는 사용자 이름

기본 이메일 텍스트는 곧 사용자 정의할 예정이지만, 지금은 제공된 링크를 찾아 웹 브라우저에 입력하는 데 집중하세요. 이 예에서, 제 링크는:

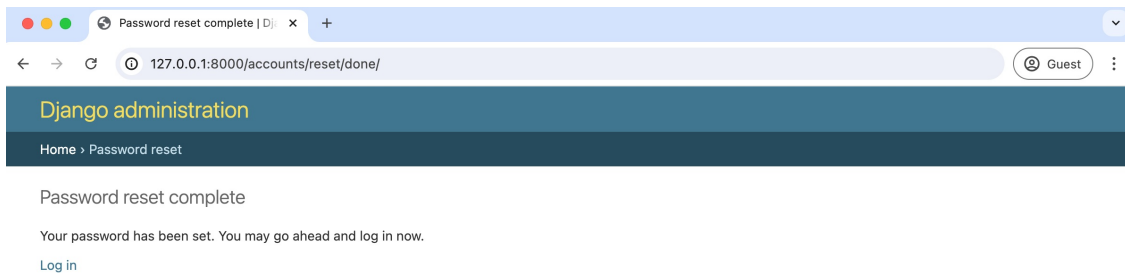
http://127.0.0.1:8000/accounts/reset/Mw/bn63cub77b6590a2e38a75c4b2af244c40297e/

“비밀번호 재설정 확인” 페이지로 리디렉션됩니다.

A screenshot of a web browser showing the Django password reset confirmation page. The browser's address bar displays the URL '127.0.0.1:8000/accounts/reset/Mw/set-password/'. The page has a dark blue header with 'Django administration' in yellow. Below the header, a breadcrumb trail shows 'Home > Password reset confirmation'. The main content area is titled 'Enter new password' and includes a message: 'Please enter your new password twice so we can verify you typed it in correctly.' There are two input fields: 'New password:' and 'Confirm password:'. At the bottom, there is a blue button labeled 'Change my password'.

Default Password Reset Confirmation

Enter a new password and click the “Change my password” button. This final step will redirect you to the “Password reset complete” page.

A screenshot of a web browser showing the Django password reset complete page. The browser's address bar displays the URL '127.0.0.1:8000/accounts/reset/done/'. The page has a dark blue header with 'Django administration' in yellow. Below the header, a breadcrumb trail shows 'Home > Password reset'. The main content area is titled 'Password reset complete' and includes a message: 'Your password has been set. You may go ahead and log in now.' There is a blue link labeled 'Log in'.

Default Password Reset Complete

To confirm everything worked, click the “Log in” link and use your new password. It should work.

Custom Templates

As with the password change pages, we can create new templates to customize the look and feel of the entire password reset flow. If you noticed, four separate templates are used. Create these new files now in your `templates/registration/` directory.

- `templates/registration/password_reset_form.html`
- `templates/registration/password_reset_done.html`
- `templates/registration/password_reset_confirm.html`

A screenshot of a web browser showing the Django administration interface. The browser's address bar displays the URL '127.0.0.1:8000/accounts/reset/Mw/set-password/'. The page title is 'Django administration' and the breadcrumb trail is 'Home > Password reset confirmation'. The main heading is 'Enter new password'. Below it, a message states: 'Please enter your new password twice so we can verify you typed it in correctly.' There are two input fields: 'New password:' and 'Confirm password:'. At the bottom, there is a blue button labeled 'Change my password'.

기본 비밀번호 재설정 확인

새 비밀번호를 입력하고 “비밀번호 변경” 버튼을 클릭하세요. 이 마지막 단계는 “비밀번호 재설정 완료” 페이지로 리디렉션됩니다.

A screenshot of a web browser showing the Django administration interface. The browser's address bar displays the URL '127.0.0.1:8000/accounts/reset/done/'. The page title is 'Django administration' and the breadcrumb trail is 'Home > Password reset'. The main heading is 'Password reset complete'. Below it, a message states: 'Your password has been set. You may go ahead and log in now.' At the bottom, there is a blue link labeled 'Log in'.

기본 비밀번호 재설정 완료

모든 것이 제대로 작동했는지 확인하려면 “로그인” 링크를 클릭하고 새 비밀번호를 사용하세요. 작동해야 합니다.

사용자 정의 템플릿

비밀번호 변경 페이지와 마찬가지로 전체 비밀번호 재설정 흐름의 모양과 느낌을 사용자 정의하기 위해 새로운 템플릿을 만들 수 있습니다. 네 개의 별도의 템플릿이 사용된 것을 보셨다면, 지금 `templates/registration/` 디렉토리에 이 새로운 파일들을 만드세요.

- `templates/registration/password_reset_form.html`
- `templates/registration/password_reset_done.html`
- `templates/registration/password_reset_confirm.html`

- templates/registration/password_reset_complete.html

Start with the password reset form, which is password_reset_form.html. At the top, we extend base.html, load crispy_forms_tags, and set the meta page title. Because we used “block” titles in our base.html file, we can override them here. The form uses POST since we send data, a csrf_token for security reasons, and {{ form|crispy }} for the forms. And we again updated the submit button to green. At this point, updating these template pages should start to feel familiar.

Code

```
<!-- templates/registration/password_reset_form.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block title %}Forgot Your Password?{% endblock title %}

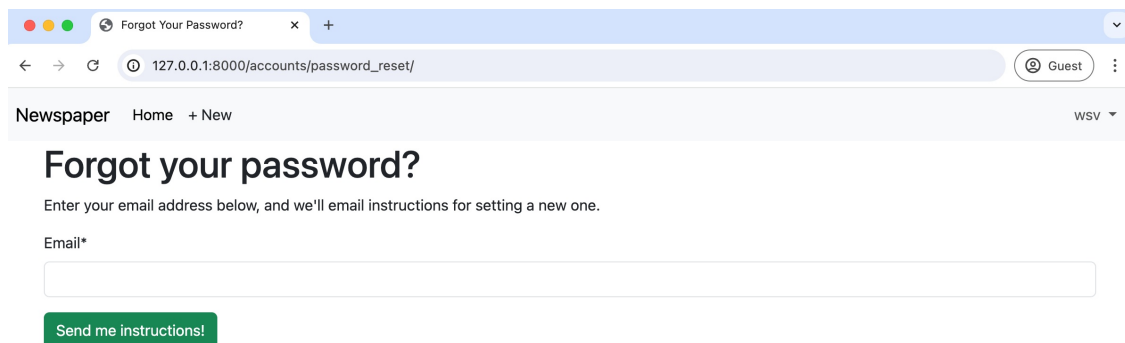
{% block content %}
<h1>Forgot your password?</h1>
<p>Enter your email address below, and we'll email instructions
for setting a new one.</p>

<form method="POST">{% csrf_token %}
  {{ form|crispy }}
  <input class="btn btn-success" type="submit"
    value="Send me instructions!">
</form>
{% endblock content %}
```

Start up the server again with `python manage.py runserver` and navigate to:

`http://127.0.0.1:8000/accounts/password_reset/`

Refresh the page, and you will see our new page.

A screenshot of a web browser showing the 'Forgot Your Password?' page. The browser's address bar shows the URL 'http://127.0.0.1:8000/accounts/password_reset/'. The page has a light gray header with 'Newspaper' on the left and 'WSV' on the right. The main content area has a large heading 'Forgot your password?' followed by the instruction 'Enter your email address below, and we'll email instructions for setting a new one.' Below this is a text input field labeled 'Email*' and a green button labeled 'Send me instructions!'.

New Password Reset

- templates/registration/password_reset_complete.html

비밀번호 재설정 양식인 password_reset_form.html로 시작합니다. 맨 위에서 base.html을 확장하고, crispy_forms_tags를 로드하며, 메타 페이지 제목을 설정합니다. base.html 파일에서 “block” 제목을 사용했기 때문에 여기에서 이를 재정의할 수 있습니다. 양식은 데이터를 전송하므로 POST를 사용하고, 보안상의 이유로 csrf_token을 사용하며, {{ form|crispy }}를 양식에 사용합니다. 그리고 다시 제출 버튼을 초록색으로 업데이트했습니다. 이 시점에서 이러한 템플릿 페이지를 업데이트 하는 것이 익숙해지기 시작해야 합니다.

코드

```
<!-- templates/registration/password_reset_form.html -->
{% extends "base.html" %}{% load
crispy_forms_tags %}

{% block title %}비밀번호를 잊으셨나요?{% endblock title %}{% block content
%}

<h1>비밀번호를 잊으셨나요?</h1>
<p>아래에 이메일 주소를 입력하면 새 비밀번호 설정에 대한 지침을 이메일로 보내드
립니다.</p>

<form method="POST">{% csrf_token %}
  {{ form|crispy }}
  <input class="btn btn-success" type="submit" value="지
침을 보내주세요!">
</form>
{% endblock content %}
```

서버를 다시 시작하려면 python manage.py runserver를 사용하고 다음으로 이동하세

요: http://127.0.0.1:8000/accounts/password_reset/

페이지를 새로 고치면 새로운 페이지가 표시됩니다.

새 비밀번호 재설정

Now we can update the other three pages. Each involves extending `base.html`, setting a new meta title, and adding new content text. When a form is involved, we switch to loading and using `crispy forms`.

Let's begin with the `password_reset_done.html` template.

Code

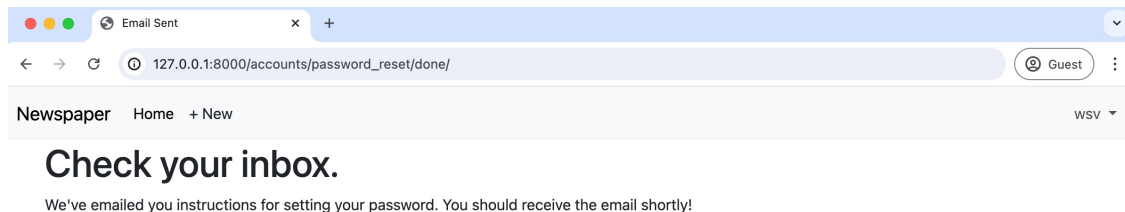
```
<!-- templates/registration/password_reset_done.html -->
{% extends "base.html" %}

{% block title %}Email Sent{% endblock title %}

{% block content %}
<h1>Check your inbox.</h1>
<p>We've emailed you instructions for setting your password.
You should receive the email shortly!</p>
{% endblock content %}
```

Confirm the changes by going to

`http://127.0.0.1:8000/accounts/password_reset/done/`.



New Reset Done

Next up is `password_reset_confirm.html`. Note that it has a form so we'll use `crispy forms` here.

Code

```
<!-- templates/registration/password_reset_confirm.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block title %}Enter new password{% endblock title %}

{% block content %}
<h1>Set a new password!</h1>
<form method="POST">{% csrf_token %}
  {{ form|crispy }}
  <input class="btn btn-success" type="submit" value="Change my password">
</form>
{% endblock content %}
```

이제 다른 세 페이지를 업데이트할 수 있습니다. 각 페이지는 base.html을 확장하고, 새로운 메타 제목을 설정하며, 새로운 콘텐츠 텍스트를 추가하는 것을 포함합니다. 양식이 포함될 때는 crispy forms를 로드하고 사용하도록 전환합니다.

비밀번호 재설정 완료.html 템플릿부터 시작하겠습니다. 코드

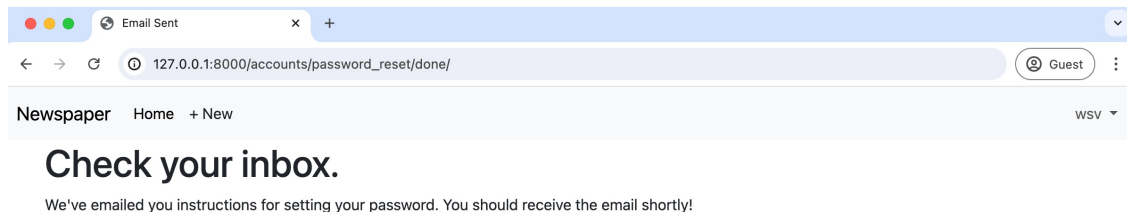
```
<!-- templates/registration/password_reset_done.html -->
{% extends "base.html" %}

{% block title %}이메일 전송됨{% endblock title %}{% block
content %}

<h1>받은 편지함을 확인하세요.</h1>
<p>비밀번호 설정을 위한 지침을 이메일로 보냈습니다. 곧 이메일을 받게 될 것
입니다!</p>
{% endblock content %}
```

변경 사항을 확인하려면

http://127.0.0.1:8000/accounts/password_reset/done/로 이동하세요.



새로운 재설정 완료

다음은 password_reset_confirm.html입니다. 양식이 있으므로 여기서 crispy forms를 사용할 것입니다.

코드

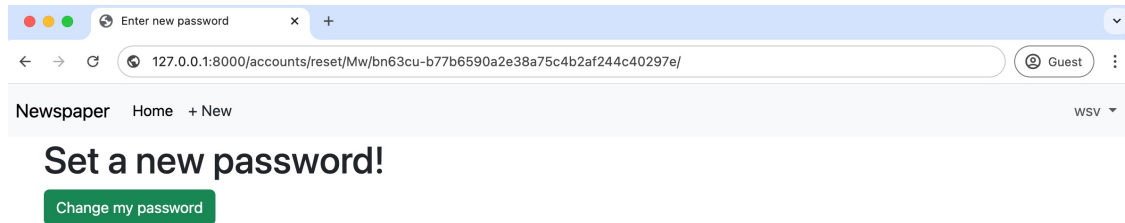
```
<!-- templates/registration/password_reset_confirm.html -->
{% extends "base.html" %}{% load
crispy_forms_tags %}

{% block title %}새 비밀번호 입력{% endblock title %}{% block content %}

<h1>새 비밀번호를 설정하세요!</h1>
<form method="POST">{% csrf_token %}
  {{ form|crispy }}
  <input class="btn btn-success" type="submit" value="비밀번호 변경"></form>

{% endblock content %}
```

In the command line, grab the URL link from the custom email previously outputted to the console, and you'll see the following:



New Set Password

Finally, here is the password reset complete code:

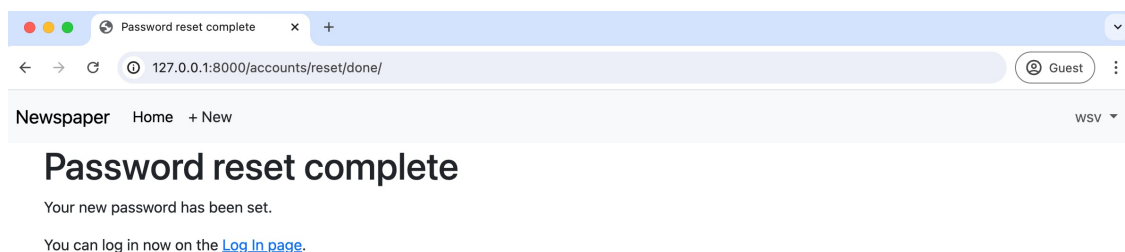
Code

```
<!-- templates/registration/password_reset_complete.html -->
{% extends "base.html" %}

{% block title %}Password reset complete{% endblock title %}

{% block content %}
<h1>Password reset complete</h1>
<p>Your new password has been set.</p>
<p>You can log in now on the
<a href="{% url 'login' %}">Log In page</a>.</p>
{% endblock content %}
```

You can view it at <http://127.0.0.1:8000/accounts/reset/done/>.

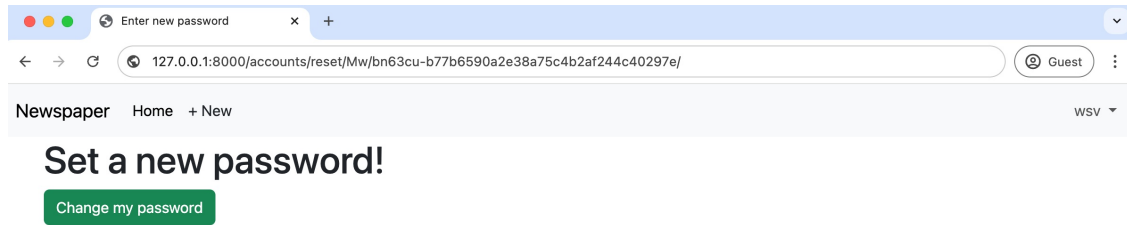


New Password Reset Complete

Try It Out

Let's confirm everything is working by resetting the password for the testuser2 account. Log out of your current account and head to the login page—the logical

명령줄에서 이전에 콘솔에 출력된 사용자 지정 이메일의 URL 링크를 가져오면 다음과 같은 내용이 표시됩니다:



새 비밀번호 설정

마지막으로, 비밀번호 재설정 완료 코드입니다:

코드

```
<!-- templates/registration/password_reset_complete.html -->
```

```
{% extends "base.html" %}
```

```
{% block title %}비밀번호 재설정 완료{% endblock title %}{% block content %}
```

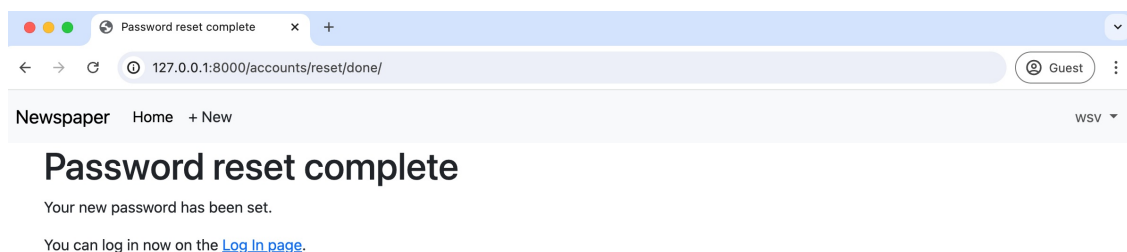
```
<h1>비밀번호 재설정 완료</h1>
```

```
<p>새 비밀번호가 설정되었습니다.</p>
```

```
<p>이제 로그인할 수 있습니다.
```

```
<a href="{% url 'login' %}">로그인 페이지</a>.</p>{% endblock content %}
```

<http://127.0.0.1:8000/accounts/reset/done/>에서 확인할 수 있습니다.



새 비밀번호 재설정 완료

해보세요

테스트 사용자2 계정의 비밀번호를 재설정하여 모든 것이 작동하는지 확인해 보겠습니다. 현재 계정에서 로그아웃하고 로그인 페이지로 이동하세요-논리적인

location for a “Forgot your password?” link that sends a user into the password reset section. Let’s add that link now.

First, we’ll need to add the password reset link to the existing login page since we can’t assume the user will know the correct URL! That goes on the bottom of the form.

Code

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block title %}Log In{% endblock title %}

{% block content %}
<h2>Log In</h2>
<form method="post">{% csrf_token %}
  {{ form|crispy }}
  <button class="btn btn-success ms-2" type="submit">Log In</button>
</form>
<!-- new code here -->
<p><a href="{% url 'password_reset' %}">Forgot your password?</a></p>
<!-- end new code -->
{% endblock content %}
```

Refresh the login webpage to confirm the new “Forgot your password?” link is there.



Forgot Your Password Link

Click on the link to bring up the password reset template and complete the flow using the email for testuser2@email.com. Remember that the unique link will be outputted to your console. Set a new password and use it to log in to the testuser2 account. Everything should work as expected.

"비밀번호를 잊으셨나요?" 링크가 사용자를 비밀번호 재설정 섹션으로 안내하는 위치입니다. 이제 그 링크를 추가해 보겠습니다.

먼저, 사용자가 올바른 URL을 알 것이라고 가정할 수 없기 때문에 기존 로그인 페이지에 비밀번호 재설정 링크를 추가해야 합니다! 그것은 양식의 하단에 위치합니다.

코드

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}{% load
crispy_forms_tags %}

{% block title %}로그인{% endblock title %}{% block
content %}

<h2>로그인</h2>
<form method="post">{% csrf_token %}
  {{ form|crispy }}
  <button class="btn btn-success ms-2" type="submit">로그인</button></form>

<!-- 여기 새로운 코드 -->
<p><a href="{% url 'password_reset' %}">비밀번호를 잊으셨나요?</a></p>
<!-- end new code -->
{% endblock content %}
```

로그인 웹페이지를 새로 고쳐서 새로운 “비밀번호를 잊으셨나요?” 링크가 있는지 확인하세요.



비밀번호를 잊으셨나요 링크

링크를 클릭하여 비밀번호 재설정 템플릿을 불러오고 testuser2@email.com의 이메일을 사용하여 흐름을 완료하세요. 고유 링크는 콘솔에 출력됩니다. 새 비밀번호를 설정하고 이를 사용하여 testuser2 계정에 로그인하세요. 모든 것이 예상대로 작동해야 합니다.

Git

Another chunk of work has been completed. Use Git to save our work before continuing.

Shell

```
(.venv) $ git status  
(.venv) $ git add -A  
(.venv) $ git commit -m "password change and reset"  
(.venv) $ git push origin main
```

Conclusion

In the next chapter, we will build out our actual *Newspaper* app that displays articles.

Git

작업의 또 다른 부분이 완료되었습니다. 계속하기 전에 Git을 사용하여 작업을 저장하세요.

셸

```
(.venv) $ git status  
(.venv) $ git add -A  
(.venv) $ git commit -m "비밀번호 변경 및 재설정"(.venv) $ git  
push origin main
```

결론

다음 장에서는 기사를 표시하는 실제 신문 앱을 구축할 것입니다.

Chapter 13: Articles App

It's time to build out our *Newspaper* app. We will have an articles page where journalists can post articles, set up permissions so only the author of an article can edit or delete it, and finally add the ability for other users to write comments on each article.

Articles App

To start, create an articles app and define the database models. There are no hard and fast rules around what to name your apps except that you can't use the name of a built-in app. If you look at the `INSTALLED_APPS` section of `django_project/settings.py`, you can see which app names are off-limits:

- `admin`
- `auth`
- `contenttypes`
- `sessions`
- `messages`
- `staticfiles`

A general rule of thumb is to use the plural of an app name: posts, payments, users, etc. One exception would be when doing so is obviously wrong, such as blogs. In this case, using the singular blog makes more sense.

Start by creating our new articles app.

Shell

```
(.venv) $ python manage.py startapp articles
```

Then add it to our `INSTALLED_APPS` and update the time zone, `TIME_ZONE`, lower down in the settings since we'll be timestamping our articles. You can find your time zone in [this Wikipedia list](#). For example, I live in Boston, MA, in the Eastern time zone of the United States; therefore, my entry is `America/New_York`.

Code

```
# django_project/settings.py
INSTALLED_APPS = [
```

13장: 기사 앱

이제 신문 앱을 구축할 시간입니다. 기자들이 기사를 게시할 수 있는 기사 페이지를 만들고, 기사의 저자만이 이를 수정하거나 삭제할 수 있도록 권한을 설정하며, 마지막으로 다른 사용자들이 각 기사에 댓글을 작성할 수 있는 기능을 추가할 것입니다.

기사 앱

먼저, 기사 앱을 생성하고 데이터베이스 모델을 정의합니다. 내장 앱의 이름을 사용할 수 없다는 것을 제외하고는 앱 이름을 정하는 데에 대한 엄격한 규칙은 없습니다. `django_project/settings.py`의 `INSTALLED_APPS` 섹션을 보면 어떤 앱 이름이 사용 불가능한지 확인할 수 있습니다:

- admin
- auth
- contenttypes
- sessions
- staticfiles
-

일반적인 규칙은 앱 이름의 복수형을 사용하는 것입니다: `posts`, `payments`, `users` 등. 단, `blogs`와 같이 명백히 잘못된 경우에는 예외가 있습니다. 이 경우, 단수형 `blog`를 사용하는 것이 더 합리적입니다.

새로운 기사 앱을 생성하는 것으로 시작합니다.

셸

```
(.venv) $ python manage.py startapp articles
```

그런 다음 `INSTALLED_APPS`에 추가하고 설정의 하단에 있는 시간대 (`TIME_ZONE`)를 업데이트합니다. 우리는 기사의 타임스탬프를 기록할 것이기 때문입니다. [이 위키피디아 목록에서](#) 시간대를 찾을 수 있습니다. 예를 들어, 저는 미국 동부 시간대에 있는 매사추세츠주 보스턴에 살고 있으므로, 제 항목은 `America/New_York`입니다.

코드

```
# django_project/settings.py
INSTALLED_APPS = [
```

```

    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # 3rd Party
    "crispy_forms",
    "crispy_bootstrap5",
    # Local
    "accounts",
    "pages",
    "articles", # new
]

TIME_ZONE = "America/New_York" # new

```

Next, we define our database model, which contains four fields: title, body, date, and author. We're letting Django automatically set the time and date based on our TIME_ZONE setting. For the author field, we want to [reference our custom user model](#) "accounts.CustomUser" which we set in the django_project/settings.py file as AUTH_USER_MODEL. We will also implement the best practice of defining a get_absolute_url and a __str__ method for viewing the model in our admin interface.

Code

```

# articles/models.py
from django.conf import settings
from django.db import models
from django.urls import reverse

class Article(models.Model):
    title = models.CharField(max_length=255)
    body = models.TextField()
    date = models.DateTimeField(auto_now_add=True)
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )

    def __str__(self):
        return self.title

    def get_absolute_url(self):
        return reverse("article_detail", kwargs={"pk": self.pk})

```

There are two ways to refer to a custom user model: AUTH_USER_MODEL and [get_user_model](#). As general advice:

- AUTH_USER_MODEL makes sense for references within a models.py file

```

"django.contrib.admin", "
django.contrib.auth", "
django.contrib.contenttypes", "
django.contrib.sessions", "
django.contrib.messages", "
django.contrib.staticfiles",
# 3rd Party
"crispy_forms", "
crispy_bootstrap5",
# Local
"accounts", "
pages", "articles", #
new
]

TIME_ZONE = "America/New_York" # new

```

다음으로, 우리는 네 개의 필드(title, body, date, author)를 포함하는 데이터베이스 모델을 정의합니다. 우리는 Django가 TIME_ZONE 설정에 따라 자동으로 시간과 날짜를 설정하도록 하고 있습니다. author 필드의 경우, 우리는 django_project/settings.py 파일에서 AUTH_USER_MODEL로 설정한 사용자 정의 사용자 모델 "accounts.CustomUser"를 참조하고자 합니다. 또한, 우리는 관리 인터페이스에서 모델을 보기 위해 `get_absolute_url` 과 `__str__` 메서드를 정의하는 모범 사례를 구현할 것입니다.

코드

```

# articles/models.py
from django.conf import settings
from django.db import models
from django.urls import reverse

class Article(models.Model):
    title = models.CharField(max_length=255)
    body = models.TextField()
    date = models.DateTimeField(auto_now_add=True)
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,

    def __str__(self):
        return self.title

    def get_absolute_url(self):
        return reverse("article_detail", kwargs={"pk": self.pk})

```

사용자 정의 사용자 모델을 참조하는 두 가지 방법이 있습니다:
AUTH_USER_MODEL 및 `get_user_model`. 일반적인 조언으로는:

- AUTH_USER_MODEL은 models.py 파일 내에서 참조하는 데 의미가 있습니다.

- `get_user_model()` is recommended everywhere else, such as views, tests, etc.

Since we have a new app and model, it's time to make a new migration file and apply it to the database.

Shell

```
(.venv) $ python manage.py makemigrations articles
Migrations for 'articles':
  articles/migrations/0001_initial.py
    - Create model Article
(.venv) $ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying articles.0001_initial... OK
```

At this point, I like to jump into the admin to play around with the model before building out the URLs/views/templates needed to display the data on the website. But first, we need to update `articles/admin.py` so our new app is displayed.

Code

```
# articles/admin.py
from django.contrib import admin
from .models import Article

admin.site.register(Article)
```

Now, we start the server.

Shell

```
(.venv) $ python manage.py runserver
```

Navigate to the admin at `http://127.0.0.1:8000/admin/` and log in.

- `get_user_model()`은 뷰, 테스트 등 다른 모든 곳에서 권장됩니다.

새로운 앱과 모델이 있으므로 새로운 마이그레이션 파일을 만들고 데이터베이스에 적용할 때입니다.

셸

```
(.venv) $ python manage.py makemigrations articles
'articles'에 대한 마이그레이션:
  articles/migrations/0001_initial.py
    - 모델 Article 생성 (.venv) $ python
manage.py migrate 수행할 작업:
```

```
모든 마이그레이션 적용: accounts, admin, articles, auth, contenttypes, sessions 마이그레이션
실행 중:
articles.0001_initial 적용 중... OK
```

이 시점에서, 저는 모델을 가지고 놀기 위해 관리자로 점프하는 것을 좋아합니다. 웹사이트에 데이터를 표시하는 데 필요한 URL/뷰/템플릿을 구축하기 전에 말이죠. 하지만 먼저, 우리의 새로운 앱이 표시되도록 `articles/admin.py`를 업데이트해야 합니다.

코드

```
# articles/admin.py
from django.contrib import admin
from .models import Article

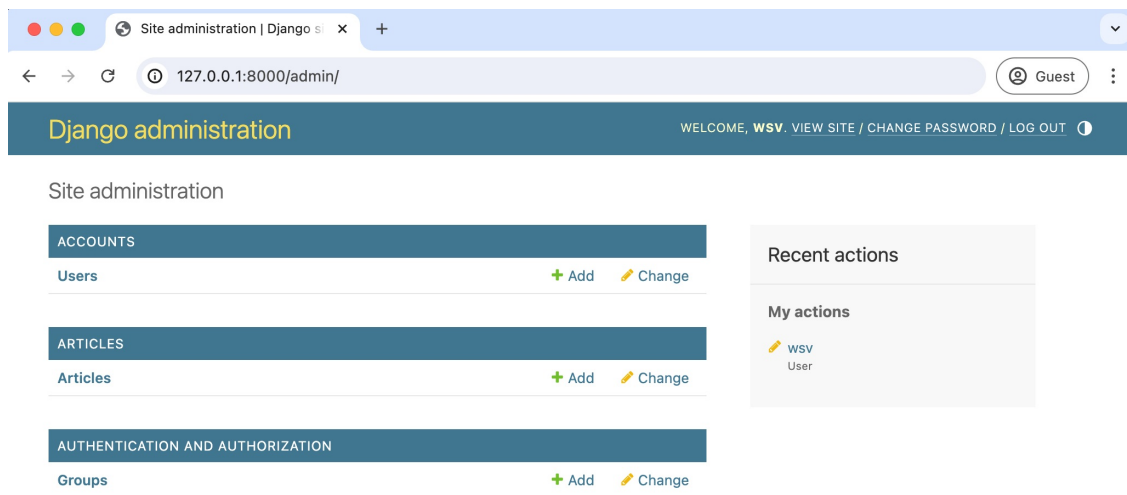
admin.site.register(Article)
```

이제 서버를 시작합니다.

셸

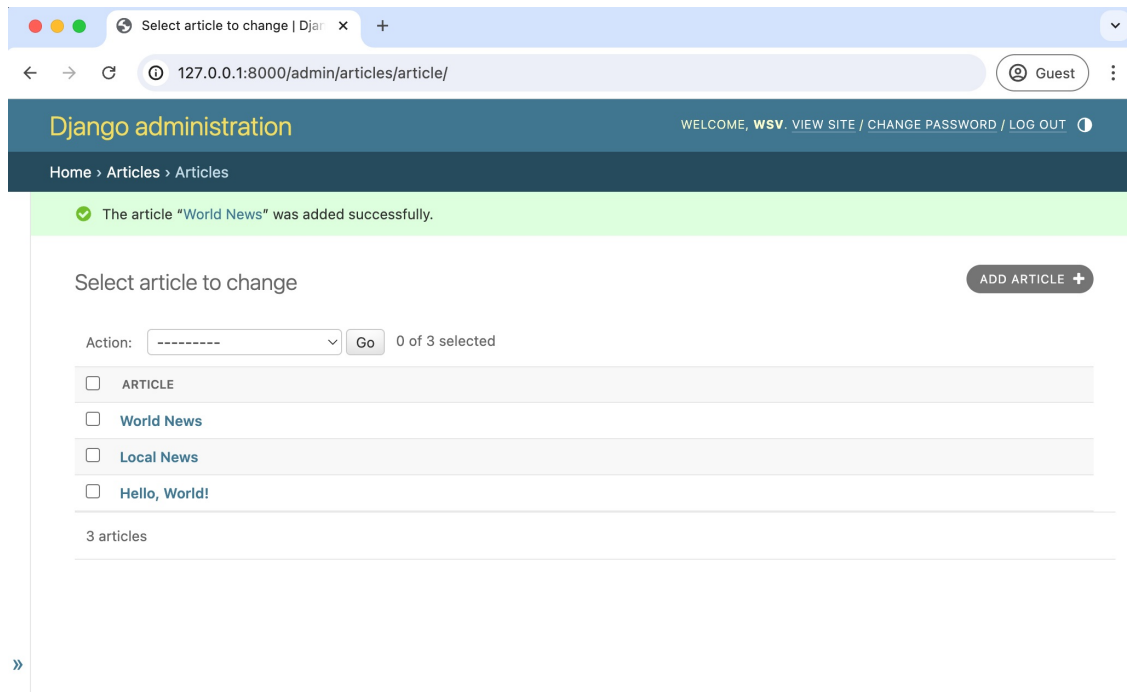
```
(.venv) $ python manage.py runserver
```

`http://127.0.0.1:8000/admin/`에서 관리자로 이동하여 로그인합니다.

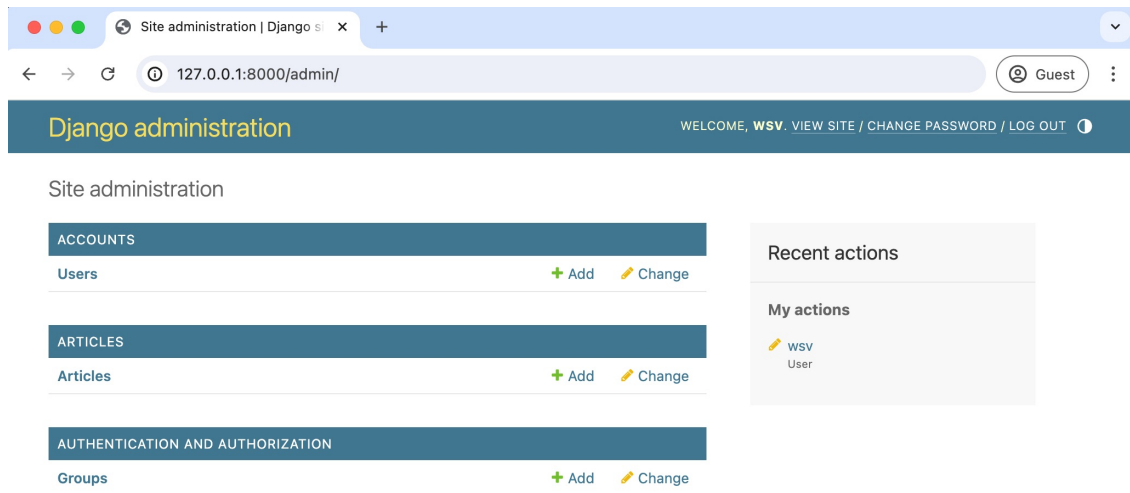


Admin Page

If you click “+ Add” next to “Articles” at the top of the page, we can enter some sample data. You’ll likely have three users available: your superuser, testuser, and testuser2 accounts. Create new articles using your superuser account as the author. I’ve added three new articles, as you can see on the updated Articles page.

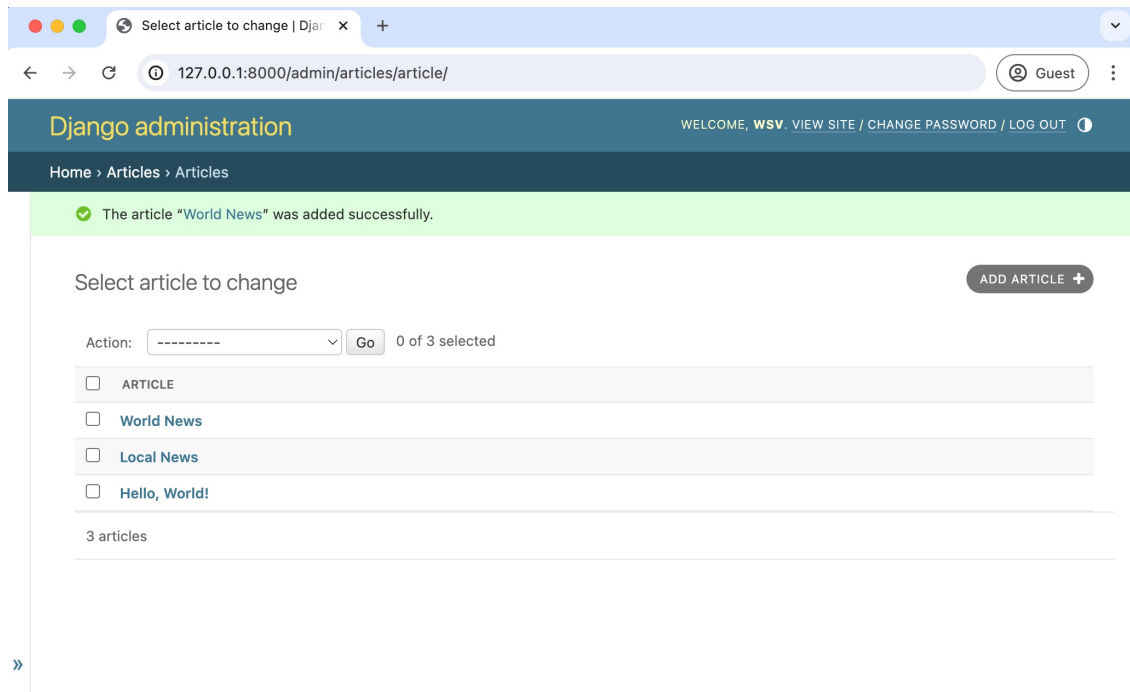


Admin Three Articles



관리자 페이지

페이지 상단의 "기사" 옆에 있는 "+ 추가"를 클릭하면 샘플 데이터를 입력할 수 있습니다. 사용 가능한 세 개의 사용자 계정이 있을 것입니다: 슈퍼유저, testuser, testuser2 계정. 슈퍼유저 계정을 사용하여 새 기사를 작성하세요. 업데이트된 기사 페이지에서 볼 수 있듯이 세 개의 새 기사를 추가했습니다.



관리자 세 개의 기사

But wouldn't it be nice to see a little more information in the admin about each article? We can quickly do that by updating `articles/admin.py` with [list_display](#).

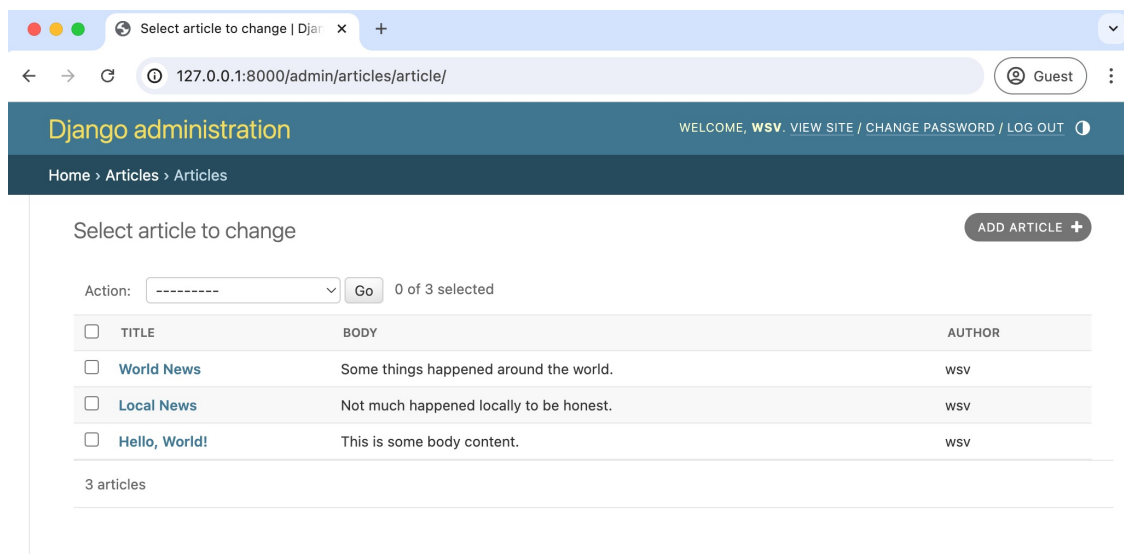
Code

```
# articles/admin.py
from django.contrib import admin
from .models import Article

class ArticleAdmin(admin.ModelAdmin):
    list_display = [
        "title",
        "body",
        "author",
    ]

admin.site.register(Article, ArticleAdmin)
```

We've extended `ModelAdmin`, a class that represents a model in the admin interface, and specified our fields to list with `list_display`. At the bottom of the file we registered `ArticleAdmin` along with the `Article` model we imported at the top. There are many customizations available in the Django admin, so the official docs are worth a close read.



Admin Three Articles with Description

If you click on an individual article, you will see that the `title`, `body`, and `author` are displayed but not the `date`, even though we defined a `date` field in

하지만 각 기사에 대한 정보를 관리자가 조금 더 볼 수 있다면 좋지 않을까요? 우리는 `articles/admin.py`를 업데이트하여 `list_display`로 이를 빠르게 수행할 수 있습니다.

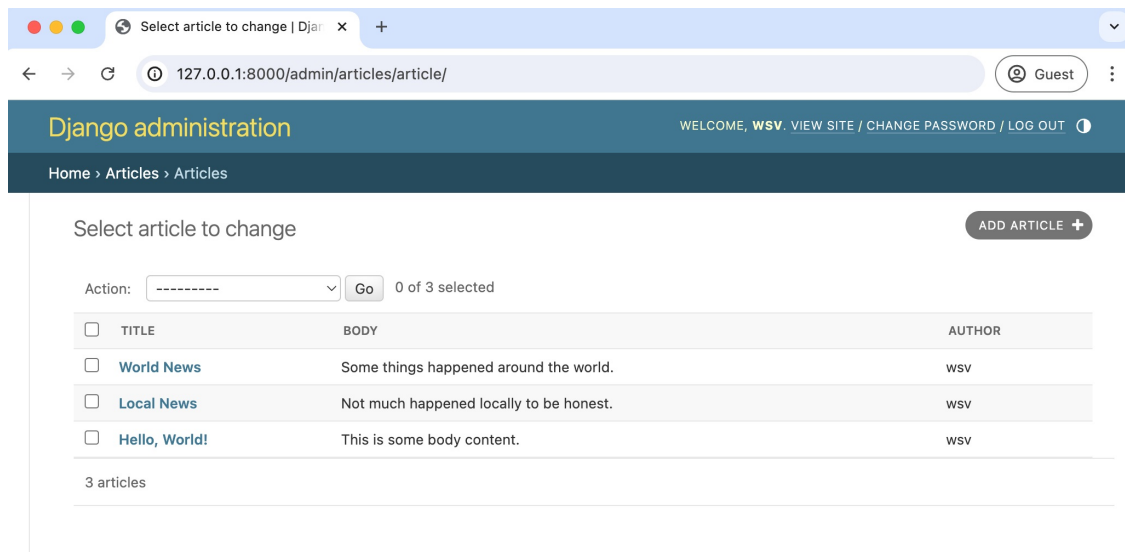
코드

```
# articles/admin.py
from django.contrib import admin
from .models import Article

class ArticleAdmin
    (admin.ModelAdmin): list_display = [
        "title", "
        body", "
        author",
    ]

admin.site.register(Article, ArticleAdmin)
```

우리는 관리 인터페이스에서 모델을 나타내는 클래스인 `ModelAdmin`을 확장하고 `list_display`로 나열할 필드를 지정했습니다. 파일의 맨 아래에서 우리는 상단에서 가져온 `Article` 모델과 함께 `ArticleAdmin`을 등록했습니다. Django 관리에서 사용할 수 있는 많은 사용자 정의가 있으므로 공식 문서를 자세히 읽어볼 가치가 있습니다.



설명이 있는 세 개의 기사 관리

개별 기사를 클릭하면 제목, 본문 및 저자가 표시되지만 날짜는 표시되지 않습니다. 날짜 필드를 정의했음에도 불구하고

our model. That's because the date was automatically added by Django for us and, therefore, can't be changed in the admin. We *could* make the date editable—in more complex apps, it's common to have both a `created_at` and `updated_at` attribute—but to keep things simple, we'll have the date be set upon creation by Django for us for now. Even though date is not displayed here, we can still access it in our templates for display on web pages.

URLs and Views

The next step is to configure our URLs and views. Let's have our articles appear at `articles/`. Add a URL pattern for articles in our `django_project/urls.py` file.

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")),
    path("accounts/", include("django.contrib.auth.urls")),
    path("articles/", include("articles.urls")), # new
    path("", include("pages.urls")),
]
```

Next, we create a new `articles/urls.py` file in the text editor and populate it with our routes. Let's start with the page to list all articles at `articles/`, which will use the view `ArticleListView`.

Code

```
# articles/urls.py
from django.urls import path

from .views import ArticleListView

urlpatterns = [
    path("", ArticleListView.as_view(), name="article_list"),
]
```

Now, create our view using the built-in generic `ListView` from Django. The only two attributes we need to specify are the model `Article` and our template name, which will be `article_list.html`.

Code

```
# articles/views.py
from django.views.generic import ListView
```

우리 모델. 날짜는 Django에 의해 자동으로 추가되었기 때문에 관리자가 변경할 수 없습니다. 날짜를 편집 가능하게 만들 수도 있지만, 더 복잡한 앱에서는 created_at과 updated_at 속성을 모두 갖는 것이 일반적입니다. 하지만 간단하게 유지하기 위해 현재는 Django가 생성 시 날짜를 설정하도록 하겠습니다. 날짜가 여기 표시되지 않더라도, 우리는 여전히 웹 페이지에 표시하기 위해 템플릿에서 접근할 수 있습니다.

URL과 뷰

다음 단계는 URL과 뷰를 구성하는 것입니다. 우리의 기사가 articles/에 나타나도록 합시다. django_project/urls.py 파일에 기사를 위한 URL 패턴을 추가합니다.

코드

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")),
    path("accounts/", include("django.contrib.auth.urls")),
    path("articles/", include("articles.urls")),
    # 새 path("", include("pages.urls")),
],
```

다음으로, 텍스트 편집기에서 새로운 articles/urls.py 파일을 만들고 우리의 경로로 채웁니다. articles/에서 모든 기사를 나열하는 페이지부터 시작하겠습니다. 이 페이지는 ArticleListView 뷰를 사용할 것입니다.

코드

```
# articles/urls.py
from django.urls import path

from .views import ArticleListView

urlpatterns = [
    path("", ArticleListView.as_view(), name="article_list"),
]
```

이제 Django의 내장 제네릭 ListView를 사용하여 우리의 뷰를 만듭니다. 우리가 지정해야 할 두 가지 속성은 모델 Article과 우리의 템플릿 이름인 article_list.html입니다.

코드

```
# articles/views.py
from django.views.generic import ListView
```

```
from .models import Article

class ArticleListView(ListView):
    model = Article
    template_name = "article_list.html"
```

The last step is to create a new template file in the text editor called `templates/article_list.html`. Bootstrap has a built-in component called [Cards](#) that we can customize for our individual articles. Recall that `ListView` returns an object with `<model_name>_list` that we can iterate using a for loop.

We display each article’s title, body, author, and date. We can even provide links to the “detail”, “edit”, and “delete” pages that we haven’t built yet.

Code

```
<!-- templates/article_list.html -->
{% extends "base.html" %}

{% block title %}Articles{% endblock title %}

{% block content %}
{% for article in article_list %}
<div class="card">
  <div class="card-header">
    <span class="fw-bold">
      <a href="#">{{ article.title }}</a>
    </span> &middot;
    <span class="text-muted">by {{ article.author }} |
      {{ article.date }}</span>
  </div>
  <div class="card-body">
    {{ article.body }}
  </div>
  <div class="card-footer text-center text-muted">
    <a href="#">Edit</a> <a href="#">Delete</a>
  </div>
</div>
<br />
{% endfor %}
{% endblock content %}
```

Start the server again and check out our page at `http://127.0.0.1:8000/articles/`.

```
from .models import Article
```

```
class ArticleListView(ListView): model =  
    Article  
    template_name = "article_list.html"
```

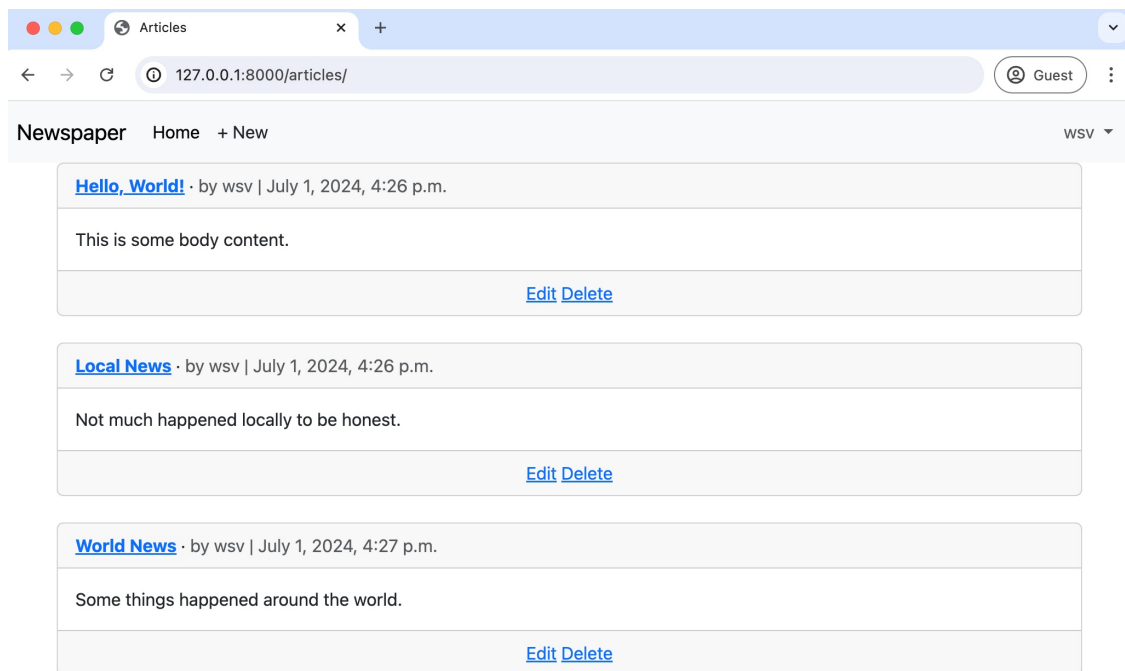
마지막 단계는 텍스트 편집기에서 templates/article_list.html이라는 새 템플릿 파일을 만드는 것입니다. Bootstrap에는 개별 기사를 사용자 정의할 수 있는 Cards라는 내장 구성 요소가 있습니다. ListView는 우리가 for 루프를 사용하여 반복할 수 있는 <model_name>_list가 포함된 객체를 반환한다는 점을 기억하세요.

각 기사의 제목, 본문, 저자 및 날짜를 표시합니다. 아직 만들지 않은 “상세”, “편집” 및 “삭제” 페이지에 대한 링크도 제공할 수 있습니다.

코드

```
<!-- templates/article_list.html -->  
{% extends "base.html" %}  
  
{% block title %}Articles{% endblock title %}{% block content  
%}  
  
{% for article in article_list %}<div class="  
card">  
    <div class="card-header">  
        <span class="fw-bold">  
            <a href="#">{{ article.title }}</a></span>  
            &middot;  
        <span class="text-muted">작성자 {{ article.author }} |  
            {{ article.date }}</span>  
    </div>  
    <div class="card-body">  
        {{ article.body }}  
    </div>  
    <div class="card-footer text-center text-muted"><a href="#">  
        편집</a> <a href="#">삭제</a>  
    </div>  
</div><br />  
{% endfor %}  
  
{% endblock content %}
```

서버를 다시 시작하고 <http://127.0.0.1:8000/articles/>에서 우리의 페이지를 확인하세요.



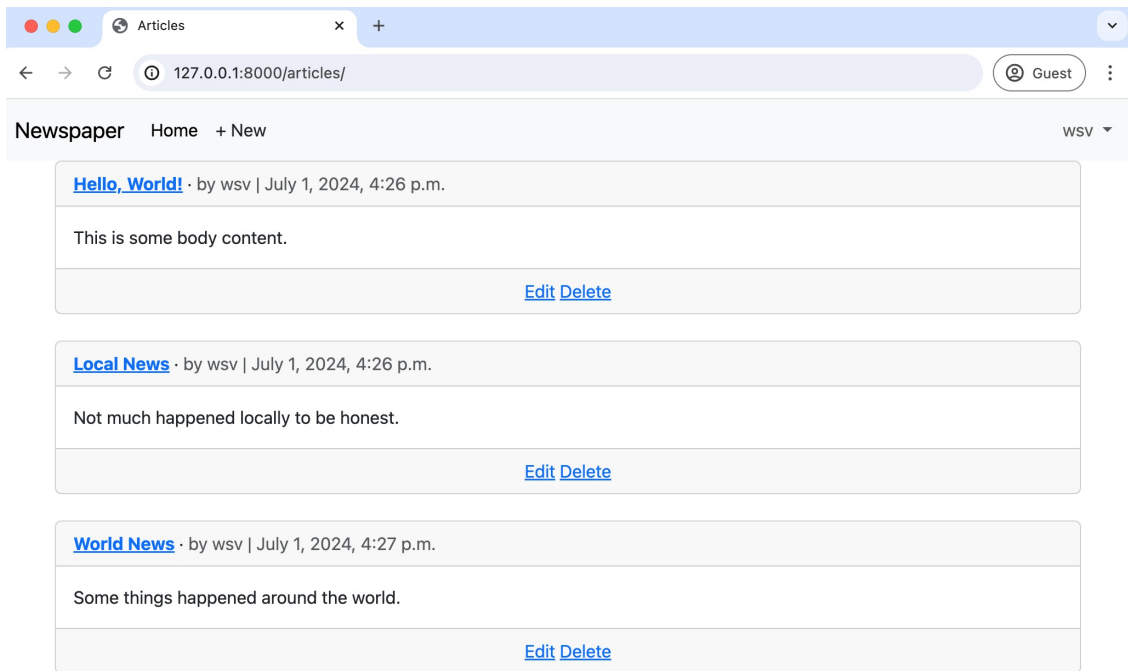
Articles Page

Not bad, eh? If we wanted to get fancy, we could create a [custom template filter](#) so that the date outputted is shown in seconds, minutes, or days. This can be done with some if/else logic and Django's [date options](#), but we won't implement it here.

Detail/Edit/Delete

The next step is to add detail, edit, and delete options for the articles. That means new URLs, views, and templates. Let's start with the URLs. The Django ORM automatically adds a primary key to each database entry, meaning that the first article has a pk value of 1, the second of 2, and so on. We can use this to craft our URL paths.

For our detail page, we want the route to be at `articles/<int:pk>`. The `int` here is a [path converter](#) and essentially tells Django that we want this value to be treated as an integer and not another data type like a string. Therefore, the URL route for the first article will be `articles/1/`. Since we are in the `articles` app, all URL routes will be prefixed with `articles/` because we set that in `django_project/urls.py`. We only need to add the `<int:pk>` part here.



기사 페이지

나쁘지 않죠? 만약 우리가 멋을 부리고 싶다면, 날짜가 초, 분 또는 일로 표시되도록 사용자 정의 템플릿 필터를 만들 수 있습니다. 이는 일부 if/else 논리와 Django의 날짜 옵션을 사용하여 수행할 수 있지만, 여기서는 구현하지 않겠습니다.

세부정보/편집/삭제

다음 단계는 기사에 대한 세부정보, 편집 및 삭제 옵션을 추가하는 것입니다. 즉, 새로운 URL, 뷰 및 템플릿이 필요합니다. URL부터 시작하겠습니다. Django ORM은 각 데이터베이스 항목에 기본 키를 자동으로 추가합니다. 즉, 첫 번째 기사는 pk 값이 1이고, 두 번째 기사는 2이며, 계속해서 증가합니다. 이를 사용하여 URL 경로를 만들 수 있습니다.

세부정보 페이지의 경우, 경로는 `articles/<int:pk>`가 되기를 원합니다. 여기서 `int`는 경로 변환기로, 본질적으로 Django에 이 값을 정수로 처리하라고 알려줍니다. 따라서 첫 번째 기사의 URL 경로는 `articles/1/`가 됩니다. 우리는 `articles` 앱에 있으므로 모든 URL 경로는 `django_project/urls.py`에서 설정한 대로 `articles/`로 접두사가 붙습니다. 여기서는 `<int:pk>` 부분만 추가하면 됩니다.

Next up are the edit and delete routes that will also use the primary key. They will be at the URL routes `articles/1/edit/` and `articles/1/delete/` with the primary key of 1. Here is how the updated `articles/urls.py` file should look.

Code

```
# articles/urls.py
from django.urls import path

from .views import (
    ArticleListView,
    ArticleDetailView, # new
    ArticleUpdateView, # new
    ArticleDeleteView, # new
)

urlpatterns = [
    path("<int:pk>/", ArticleDetailView.as_view(),
        name="article_detail"), # new
    path("<int:pk>/edit/", ArticleUpdateView.as_view(),
        name="article_edit"), # new
    path("<int:pk>/delete/", ArticleDeleteView.as_view(),
        name="article_delete"), # new
    path("", ArticleListView.as_view(),
        name="article_list"),
]
```

We will use Django's generic class-based views for `DetailView`, `UpdateView`, and `DeleteView`. The detail view only requires listing the model and template name. For the update/edit view, we also add the specific attributes—title and body—that can be changed. And for the delete view, we must add a redirect for where to send the user after deleting the entry. That requires importing `reverse_lazy` and specifying the `success_url` along with a corresponding named URL.

Code

```
# articles/views.py
from django.views.generic import ListView, DetailView # new
from django.views.generic.edit import UpdateView, DeleteView # new
from django.urls import reverse_lazy # new
from .models import Article

class ArticleListView(ListView):
    model = Article
    template_name = "article_list.html"

class ArticleDetailView(DetailView): # new
    model = Article
    template_name = "article_detail.html"

class ArticleUpdateView(UpdateView): # new
```

다음은 기본 키를 사용할 편집 및 삭제 경로입니다. 이들은 URL routes `articles/1/edit/` 및 `articles/1/delete/`에 있으며 기본 키는 1입니다. 업데이트된 `articles/urls.py` 파일은 다음과 같아야 합니다.

코드

```
# articles/urls.py
from django.urls import path

from .views import
    (ArticleListView,
     ArticleDetailView, # new
     ArticleUpdateView, # new
     ArticleDeleteView, # new
    )

urlpatterns = [
    path("<int:pk>/", ArticleDetailView.as_view(), name="
        article_detail"), # new
    path("<int:pk>/edit/", ArticleUpdateView.as_view(), name="
        article_edit"), # new
    path("<int:pk>/delete/", ArticleDeleteView.as_view(), name="
        article_delete"), # new
    path("", ArticleListView.as_view(), name="
        article_list"),
```

우리는 `DetailView`, `UpdateView` 및 `DeleteView`를 위해 Django의 일반 클래스 기반 뷰를 사용할 것입니다. 상세 보기는 모델과 템플릿 이름을 나열하는 것만 필요합니다. 업데이트/편집 보기를 위해 변경할 수 있는 특정 속성인 제목과 본문도 추가합니다. 삭제 보기를 위해서는 항목을 삭제한 후 사용자를 보낼 위치에 대한 리디렉션을 추가해야 합니다. 이는 `reverse_lazy`를 가져오고 `success_url`을 지정하며 해당하는 이름이 있는 URL을 함께 지정해야 합니다.

코드

```
# articles/views.py
from django.views.generic import ListView, DetailView # new
from django.views.generic.edit import UpdateView, DeleteView # new
from django.urls import reverse_lazy # new
from .models import Article

class ArticleListView(ListView):
    model = Article
    template_name = "article_list.html"

class ArticleDetailView(DetailView): # new
    model = Article
    template_name = "article_detail.html"

class ArticleUpdateView(UpdateView): # new
```

```
model = Article
fields = (
    "title",
    "body",
)
template_name = "article_edit.html"

class ArticleDeleteView(DeleteView): # new
    model = Article
    template_name = "article_delete.html"
    success_url = reverse_lazy("article_list")
```

If you recall the acronym CRUD (Create-Read-Update-Delete), you'll see that we are implementing three of the four functionalities here. We'll add the fourth, for create, later in this chapter. Almost every website uses CRUD, and this pattern will quickly feel natural when using Django or any other web framework.

The URL paths and views are done, so the final step is to add templates. Create three new template files in your text editor:

- templates/article_detail.html
- templates/article_edit.html
- templates/article_delete.html

We'll start with the details page, which displays the title, date, body, and author and links to edit and delete. It also links back to all articles. `DetailView` automatically names the context object either `object` or the lowercase model name. Recall that the Django templating language's `url` tag wants the URL name, and any arguments are passed in.

The name of our edit route is `article_edit`, and we need to use its primary key, `article.pk`. The delete route name is `article_delete`, and requires a primary key, `article.pk`. Our articles page is a `ListView`, so it does not require any additional arguments passed in.

Code

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="article-entry">
    <h2>{{ object.title }}</h2>
    <p>by {{ object.author }} | {{ object.date }}</p>
    <p>{{ object.body }}</p>
</div>
```

```
model = Article
fields = (
    "title",
    "body",
)
template_name = "article_edit.html"
```

```
class ArticleDeleteView(DeleteView): # new
    model = Article
    template_name = "article_delete.html"
    success_url = reverse_lazy("article_list")
```

CRUD(생성-읽기-업데이트-삭제)라는 약어를 기억하신다면, 여기서 네 가지 기능 중 세 가지를 구현하고 있다는 것을 알 수 있습니다. 이 장 후반부에 생성 기능을 추가할 것입니다. 거의 모든 웹사이트는 CRUD를 사용하며, Django나 다른 웹 프레임워크를 사용할 때 이 패턴은 금방 자연스럽게 느껴질 것입니다.

URL 경로와 뷰는 완료되었으므로 마지막 단계는 템플릿을 추가하는 것입니다. 텍스트 편집기에서 세 개의 새로운 템플릿 파일을 만드세요:

- templates/article_detail.html
- s/article_edit.html
- templates/article_delete.html

세부 정보 페이지부터 시작하겠습니다. 이 페이지는 제목, 날짜, 본문, 저자를 표시하고 편집 및 삭제 링크를 포함합니다. 또한 모든 기사로 돌아가는 링크도 포함되어 있습니다. DetailView는 자동으로 컨텍스트 객체의 이름을 object 또는 소문자 모델 이름으로 지정합니다. Django 템플릿 언어의 url 태그는 URL 이름을 원하며, 모든 인수는 전달됩니다.

편집 경로의 이름은 article_edit이며, 기본 키인 article.pk를 사용해야 합니다. 삭제 경로 이름은 article_delete이며, 기본 키인 article.pk가 필요합니다. 우리의 기사 페이지는 ListView이므로 추가 인수를 전달할 필요가 없습니다.

코드

```
<!-- templates/article_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="article-entry"><h2>
    {{ object.title }}</h2>
    <p>작성자: {{ object.author }} | {{ object.date }}</p>
    <p>{{ object.body }}</p>
</div>
```

```

<div>
  <p><a href="{% url 'article_edit' article.pk %}">Edit</a>
    <a href="{% url 'article_delete' article.pk %}">Delete</a>
  </p>
  <p>Back to <a href="{% url 'article_list' %}">All Articles</a>.</p>
</div>
{% endblock content %}

```

For the edit and delete pages, we can use Bootstrap's [button styling](#) to make the edit button light blue and the delete button red.

Code

```

<!-- templates/article_edit.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block content %}
<h1>Edit</h1>
<form action="" method="post">{% csrf_token %}
  {{ form|crispy }}
  <button class="btn btn-info ms-2" type="submit">Update</button>
</form>
{% endblock content %}

```

Code

```

<!-- templates/article_delete.html -->
{% extends "base.html" %}

{% block content %}
<h1>Delete</h1>
<form action="" method="post">{% csrf_token %}
  <p>Are you sure you want to delete "{{ article.title }}"?</p>
  <button class="btn btn-danger ms-2" type="submit">Confirm</button>
</form>
{% endblock content %}

```

As a final step, in `article_list.html`, we can now add URL routes for detail, edit, and delete pages to replace the existing `<a href="#">` placeholders. We can use the `get_absolute_url` method defined in our model for the detail page. And we can use the [url](#) template tag, the URL name, and the pk of each article for the edit and delete links.

Code

```

<!-- templates/article_list.html -->
{% extends "base.html" %}

{% block title %}Articles{% endblock title %}

{% block content %}
{% for article in article_list %}
<div class="card">
  <div class="card-header">

```

```

<div>
  <p><a href="{% url 'article_edit' article.pk %}">수정</a><a href="{% url
    'article_delete' article.pk %}">삭제</a>
  </p>
  <p>모든 기사로 돌아가기 <a href="{% url 'article_list' %}">All Articles</a> . </p>
</div>
{% endblock content %}

```

수정 및 삭제 페이지에서는 Bootstrap의 버튼 스타일을 사용하여 수정 버튼을 연한 파란색으로, 삭제 버튼을 빨간색으로 만들 수 있습니다.

코드

```

<!-- templates/article_edit.html -->
{% extends "base.html" %}{% load
crispy_forms_tags %}

{% block content %}<h1>
수정</h1>
<form action="" method="post">{% csrf_token %}{% form |
  crispy %}
  <button class="btn btn-info ms-2" type="submit">업데이트</button></form>

{% endblock content %}

```

코드

```

<!-- templates/article_delete.html -->
{% extends "base.html" %}

{% block content %}<h1>
삭제</h1>
<form action="" method="post">{% csrf_token %}
  <p>정말로 "{ article.title }"을 (를) 삭제하시겠습니까?</p>
  <button class="btn btn-danger ms-2" type="submit">확인</button></form>

{% endblock content %}

```

마지막 단계로, article_list.html에서 기존 자리 표시자를 대체하기 위해 세부정보, 편집 및 삭제 페이지에 대한 URL 경로를 추가할 수 있습니다. 세부정보 페이지에는 모델에 정의된 get_absolute_url 메서드를 사용할 수 있습니다. 편집 및 삭제 링크에는 url 템플릿 태그, URL 이름 및 각 기사의 pk를 사용할 수 있습니다.

코드

```

<!-- templates/article_list.html -->
{% extends "base.html" %}

{% block title %}기사{% endblock title %}{% block content %}

{% for article in article_list %}<div class="
card">
  <div class="card-header">

```

```

<span class="fw-bold">
  <!-- add link here! -->
  <a href="{{ article.get_absolute_url }}">{{ article.title }}</a>
</span> &middot;
<span class="text-muted">by {{ article.author }} |
  {{ article.date }}</span>
</div>
<div class="card-body">
  {{ article.body }}
</div>
<div class="card-footer text-center text-muted">
  <!-- new links here! -->
  <a href="{% url 'article_edit' article.pk %}">Edit</a>
  <a href="{% url 'article_delete' article.pk %}">Delete</a>
</div>
</div>
<br />
{% endfor %}
{% endblock content %}

```

Ok, we're ready to view our work. Start the server with `python manage.py runserver` and navigate to the articles list page at `http://127.0.0.1:8000/articles/`. Click the "Edit" link next to the first article, and you'll be redirected to `http://127.0.0.1:8000/articles/1/edit/`.

Newspaper App

127.0.0.1:8000/articles/1/edit/

Guest

Newspaper Home + New WSV

Edit

Title*

Hello, World!

Body*

This is some body content.

Update

Edit Page


```

<span class="fw-bold">
  <!-- 링크를 여기에 추가하세요! -->
  <a href="{{ article.get_absolute_url }}">{{ article.title }}</a></span> &middot;

  <span class="text-muted">작성자 {{ article.author }} |
    {{ article.date }}</span>
</div>
<div class="card-body">
  {{ article.body }}
</div>
<div class="card-footer text-center text-muted">
  <!-- 새로운 링크를 여기에 추가하세요! -->
  <a href="{% url 'article_edit' article.pk %}">편집</a>
  <a href="{% url 'article_delete' article.pk %}">삭제</a>
</div>
</div><br />
{% endfor %}

{% endblock content %}

```

좋습니다, 이제 작업을 볼 준비가 되었습니다. python manage.pyrunserver로 서버를 시작하고 http://127.0.0.1:8000/articles/의 기사 목록 페이지로 이동하세요. 첫 번째 기사 옆에 있는 “편집” 링크를 클릭하면 http://127.0.0.1:8000/articles/1/edit/로 리디렉션됩니다.

Newspaper App

127.0.0.1:8000/articles/1/edit/

Guest

Newspaper Home + New WSV

Edit

Title*

Hello, World!

Body*

This is some body content.

Update

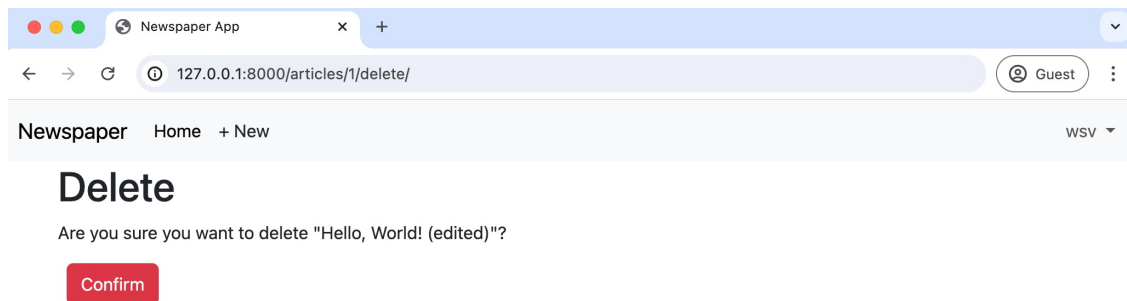
편집 페이지

If you update the “Title” attribute by adding “(edited)” at the end and click “Update”, you’ll be redirected to the detail page, which shows the new change.



Detail Page

You’ll be redirected to the delete page if you click the “Delete” link.



Delete Page

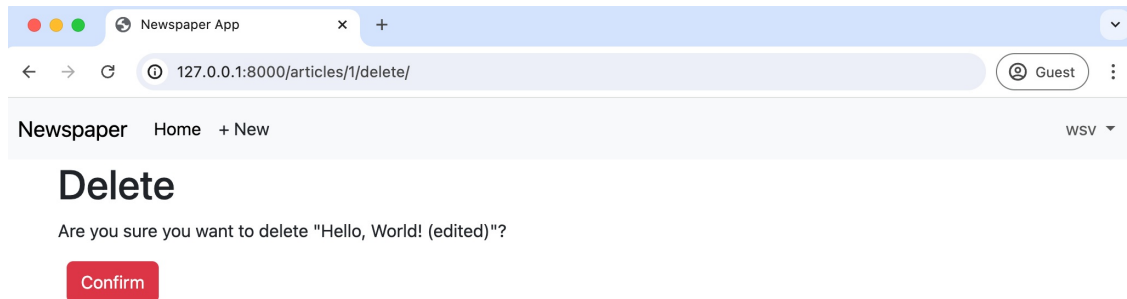
Press the scary red button for “Confirm.” You’ll be redirected to the articles page, which now has only two entries.

"제목" 속성을 끝에 "(편집됨)"을 추가하여 업데이트하고 "업데이트"를 클릭하면 새로운 변경 사항이 표시된 세부 페이지로 리디렉션됩니다.



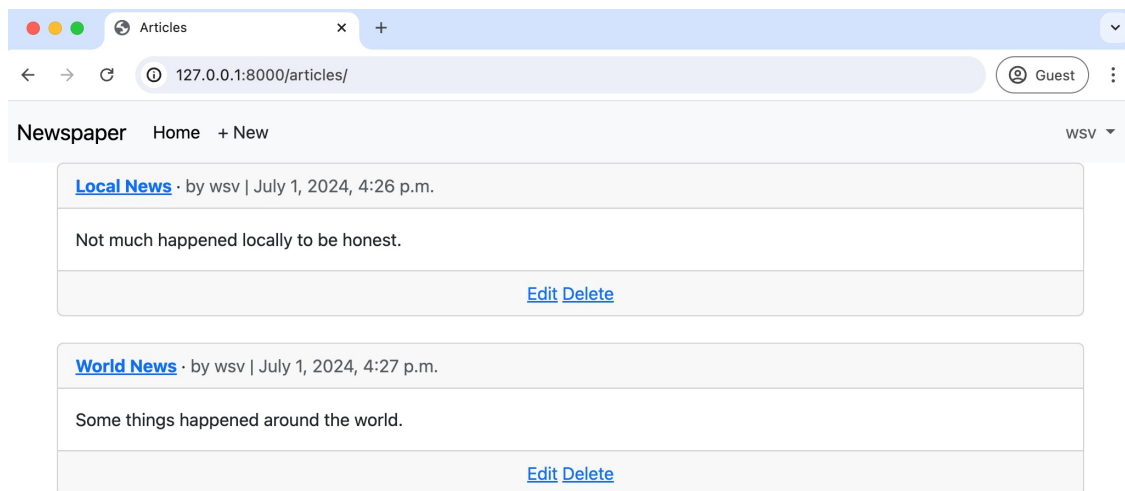
세부 페이지

"삭제" 링크를 클릭하면 삭제 페이지로 리디렉션됩니다.



삭제 페이지

"확인"을 위해 무서운 빨간 버튼을 누르세요. 이제 두 개의 항목만 있는 기사 페이지로 리디렉션됩니다.



Articles Page Two Entries

Create Page

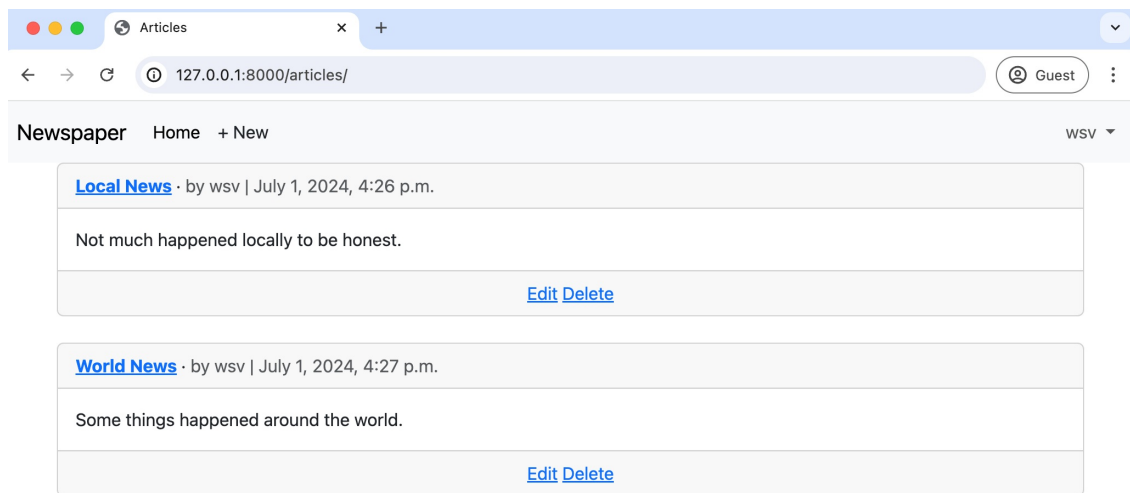
The final step is to create a page for new articles, which we can implement with Django's built-in `CreateView`. Our three steps are to create a view, URL, and template. This flow should feel familiar by now.

In the `articles/views.py` file, add `CreateView` to the imports at the top and create a new class called `ArticleCreateView` at the bottom of the file that specifies our model, template, and the fields available.

Code

```
# articles/views.py
...
from django.views.generic.edit import (
    UpdateView, DeleteView, CreateView # new
)
...
class ArticleCreateView(CreateView): # new
    model = Article
    template_name = "article_new.html"
    fields = (
        "title",
        "body",
        "author",
    )
```

Note that our `fields` attribute has `author` since we want to associate a new article with an author; however, once an article has been created, we do not want



기사 페이지 두 개의 항목

페이지 만들기

마지막 단계는 새로운 기사를 위한 페이지를 만드는 것입니다. 이는 Django의 내장 `CreateView`를 사용하여 구현할 수 있습니다. 우리의 세 단계는 뷰, URL 및 템플릿을 만드는 것입니다. 이 흐름은 이제 익숙할 것입니다.

`articles/views.py` 파일에서 상단의 `import`에 `CreateView`를 추가하고 파일 하단에 우리의 모델, 템플릿 및 사용 가능한 필드를 지정하는 `ArticleCreateView`라는 새 클래스를 만듭니다.

코드

```
# articles/views.py
...
from django.views.generic.edit import
    (UpdateView, DeleteView, CreateView # new
)
...
class ArticleCreateView(CreateView): # new
    model = Article
    template_name = "article_new.html"
    fields = (
        "title",
        "body",
        "author",
    )
```

Note that our fields attribute has `author` since we want to associate a new article with an author; however, once an article has been created, we do not want

a user to be able to change the author, which is why ArticleUpdateView only has the attributes ['title', 'body',].

Now, update the articles/urls.py file with the new route for the view.

Code

```
# articles/urls.py
from django.urls import path

from .views import (
    ArticleListView,
    ArticleDetailView,
    ArticleUpdateView,
    ArticleDeleteView,
    ArticleCreateView, # new
)

urlpatterns = [
    path("<int:pk>/",
        ArticleDetailView.as_view(), name="article_detail"),
    path("<int:pk>/edit/",
        ArticleUpdateView.as_view(), name="article_edit"),
    path("<int:pk>/delete/",
        ArticleDeleteView.as_view(), name="article_delete"),
    path("new/", ArticleCreateView.as_view(), name="article_new"), # new
    path("", ArticleListView.as_view(), name="article_list"),
]
```

To complete the new create functionality, add a template named templates/article_new.html and update it with the following HTML code.

Code

```
<!-- templates/article_new.html -->
{% extends "base.html" %}
{% load crispy_forms_tags %}

{% block content %}
<h1>New article</h1>
<form action="" method="post">{% csrf_token %}
    {{ form|crispy }}
    <button class="btn btn-success ms-2" type="submit">Save</button>
</form>
{% endblock content %}
```

Additional Links

We should add the URL link for creating new articles to our navbar so that logged-in users can access it everywhere on the site.

Code

사용자가 저자를 변경할 수 있도록 하기 위해 ArticleUpdateView에는 ['title', 'body',] 속성만 있습니다.

이제 articles/urls.py 파일을 업데이트하여 뷰에 대한 새로운 경로를 추가하세요. 코드

```
# articles/urls.py
from django.urls import path

from .views import
    (ArticleListView,
     ArticleDetailView,
     ArticleUpdateView,
     ArticleDeleteView,
     ArticleCreateView, # new
    )

urlpatterns = [
    path("<int:pk>/",
        ArticleDetailView.as_view(), name="article_detail"), path("<int:pk>/edit/",
        ArticleUpdateView.as_view(), name="article_edit"), path(
        "<int:pk>/delete/",
        ArticleDeleteView.as_view(), name="article_delete"), path("new/",
        ArticleCreateView.as_view(), name="article_new"), path("",
        ArticleListView.as_view(), name="article_list"),
]
```

새로 만들기 기능을 완료하려면 다음과 같은 템플릿을 추가하세요.
templates/article_new.html 및 다음 HTML 코드로 업데이트합니다.코드

```
<!-- templates/article_new.html -->
{% extends "base.html" %}{% load
crispy_forms_tags %}

{% block content %}<h1>
새 기사</h1>
<form action="" method="post">{% csrf_token %}{% form|
    crispy %}
    <button class="btn btn-success ms-2" type="submit">저장</button></form>

{% endblock content %}
```

추가 링크

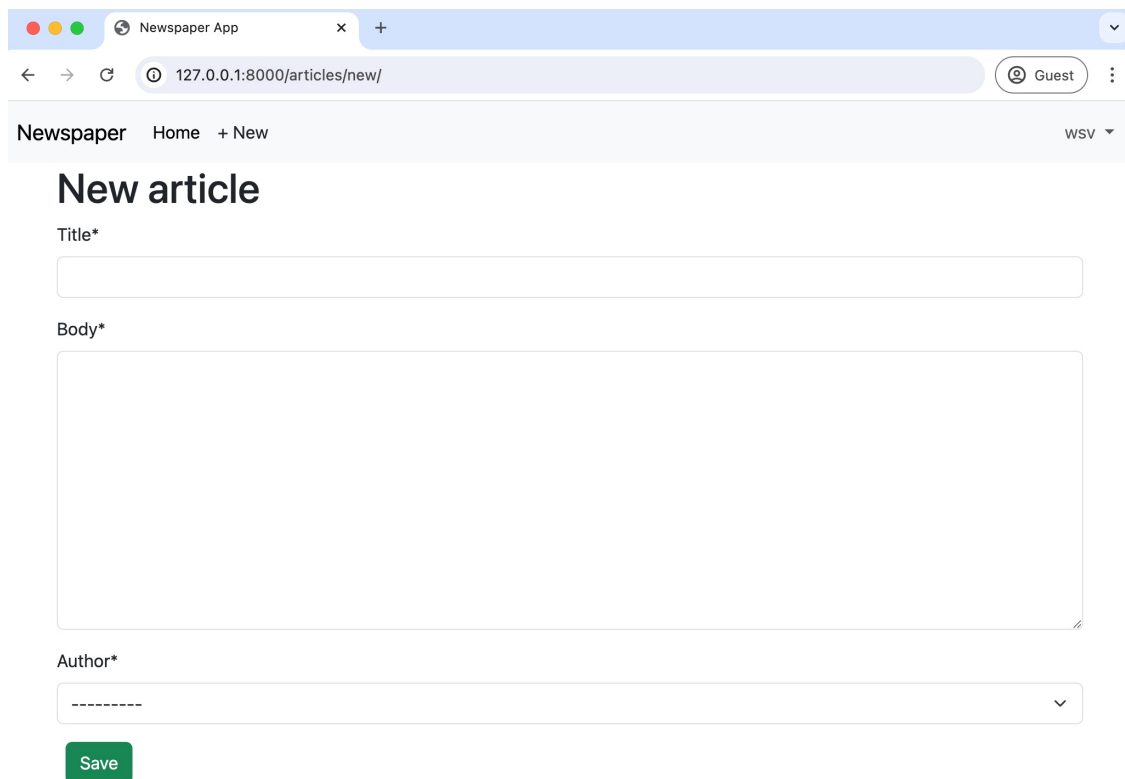
로그인한 사용자가 사이트 어디에서나 접근할 수 있도록 새로운 기사를 작성하는 URL 링크를 내비게이션 바에 추가해야 합니다.

코드

```
<!-- templates/base.html -->
...
{% if user.is_authenticated %}
  <li><a href="{% url 'article_new' %}"
    class="nav-link px-2 link-dark">+ New</a></li>
...

```

Refreshing the webpage and clicking “+ New” will redirect to the create new article page.



Newspaper App

127.0.0.1:8000/articles/new/

Guest

Newspaper Home + New WSV

New article

Title*

Body*

Author*

Save

New Article Page

One final link to add is to make the articles list page accessible from the home page. A user would need to know or guess that it is located at `http://127.0.0.1:8000/articles/`, but we can fix that by adding a button link to the `templates/home.html` file.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block title %}Home{% endblock title %}

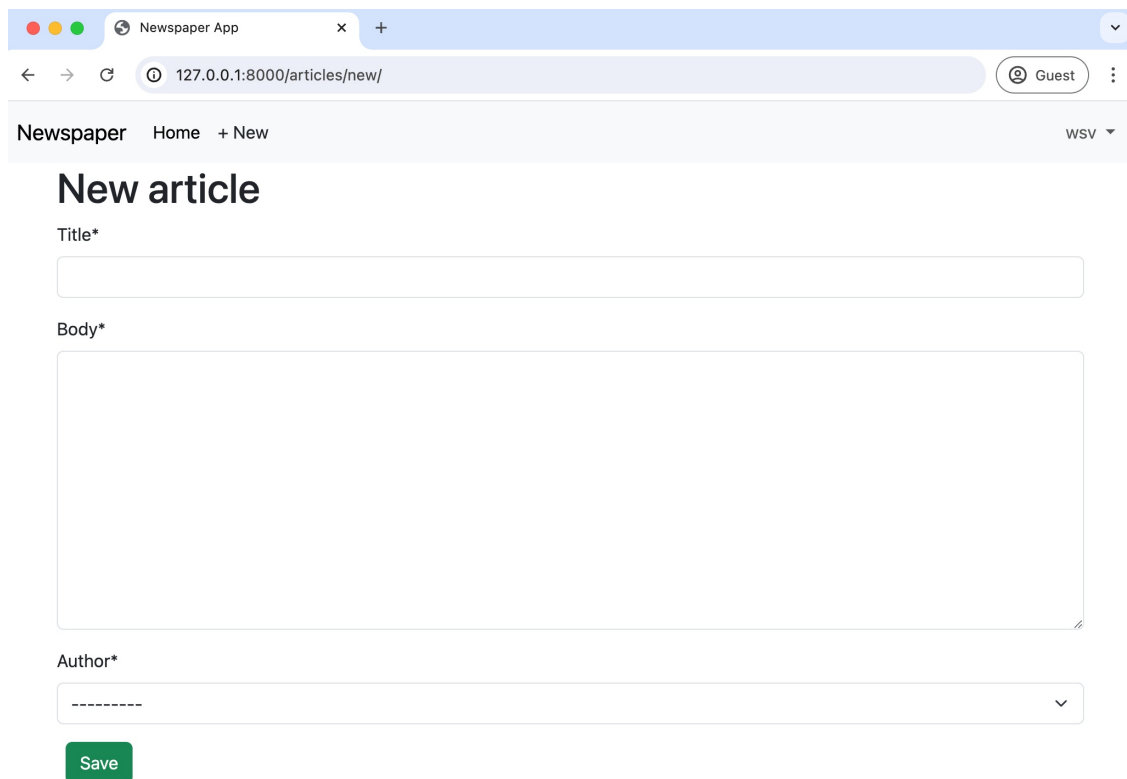
{% block content %}

```



```
<!-- templates/base.html -->
...
{% if user.is_authenticated %}<li><a href="{% url
'article_new' %}"
class="nav-link px-2 link-dark">+ 새로 만들기</a>
</li>...
```

웹페이지를 새로 고치고 “+ 새로 만들기”를 클릭하면 새 기사 작성 페이지로 리디렉션됩니다.



새 기사 페이지

마지막으로 추가할 링크는 기사 목록 페이지를 홈페이지에서 접근할 수 있도록 하는 것입니다. 사용자는 `http://127.0.0.1:8000/articles/`에 위치해 있다는 것을 알아야 하거나 추측해야 하지만, `templates/home.html` 파일에 버튼 링크를 추가하여 이를 해결할 수 있습니다.

코드

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block title %}홈{% endblock title %}{% block content %}
%
```

```
{% if user.is_authenticated %}
Hi {{ user.username }}! You are {{ user.age }} years old.
<form action="{% url 'logout' %}" method="post">
  {% csrf_token %}
  <button type="submit">Log Out</button>
</form>
<br/>
<p><a class="btn btn-primary" href="{% url 'article_list' %}" role="button">
  View all articles</a></p> <!-- new -->
{% else %}
<p>You are not logged in</p>
<a href="{% url 'login' %}">Log In</a> |
<a href="{% url 'signup' %}">Sign Up</a>
{% endif %}
{% endblock content %}
```

Refresh the homepage and the button will appear and work as intended.



Homepage with All Articles Link

If you need help to make sure your HTML file is accurate now, please refer to the [official source code](#).

Git

We added quite a lot of new code in this chapter, so let's save it with Git before proceeding to the next chapter.

Shell

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "newspaper app"
(.venv) $ git push origin main
```

Conclusion

```
{% if user.is_authenticated %}
안녕하세요 {{ user.username }}! 당신은 {{ user.age }}세입니다.
<form action="{% url 'logout' %}" method="post">{%
  csrf_token %}
  <button type="submit">로그아웃</button>
</form>
<br/>
<p><a class="btn btn-primary" href="{% url 'article_list' %}" role="button">
  모든 기사 보기</a></p><!-- new -->{% else %}
  <p>로그인하지 않았습니다</p><a href="{% url
'login' %}">로그인</a> | <a href="{% url 'signup'
%}">회원가입</a>{% endif %}

{% endblock content %}
```

홈페이지를 새로 고치면 버튼이 나타나고 의도한 대로 작동합니다.



모든 기사 링크가 있는 홈페이지

HTML 파일이 정확한지 확인하는 데 도움이 필요하면 공식 소스 코드를 [참조하세요.](#)

Git

이번 장에서는 많은 새로운 코드를 추가했으므로 다음 장으로 진행하기 전에 Git으로 저장합니다.

셸

```
(.venv) $ git status
(.venv) $ git add -A
(.venv) $ git commit -m "newspaper app" (.venv)
$ git push origin main
```

결론

We have now created a dedicated `articles` app with CRUD functionality. Articles can be created, read, updated, deleted, and even viewed as an entire list. But there are no permissions or authorizations yet, which means anyone can do anything! If logged-out users know the correct URLs, they can edit an existing article or delete one, even one that's not theirs! In the next chapter, we will add permissions and authorizations to our project to fix this.

이제 CRUD 기능이 있는 전용 기사 앱을 만들었습니다. 기사는 생성, 읽기, 업데이트, 삭제할 수 있으며 전체 목록으로도 볼 수 있습니다. 하지만 아직 권한이나 인증이 없기 때문에 누구나 무엇이든 할 수 있습니다! 로그인된 사용자가 올바른 URL을 알면 기존 기사를 편집하거나 자신의 것이 아닌 기사를 삭제할 수 있습니다! 다음 장에서는 이 문제를 해결하기 위해 프로젝트에 권한 및 인증을 추가할 것입니다.

Chapter 14: Permissions and Authorization

There are several issues with our current *Newspaper* website. For one thing, we want our newspaper to be financially sustainable. With more time, we could add a dedicated payments app to charge for access. But at a minimum, we want to add rules around permissions and authorization, such as requiring users to log in to view articles. As a mature web framework, Django has built-in authorization functionality that we can use to restrict access to the articles list page and add additional restrictions so that only the author of an article can edit or delete it.

Improved CreateView

Currently, the author on a new article can be set to any existing user. Instead, it should be automatically set to the currently logged-in user. We can modify Django's `CreateView` to achieve this by removing the `author` field and instead setting it automatically via the `form_valid` method.

Code

```
# articles/views.py
class ArticleCreateView(CreateView):
    model = Article
    template_name = "article_new.html"
    fields = ("title", "body") # new

    def form_valid(self, form): # new
        form.instance.author = self.request.user
        return super().form_valid(form)
```

How did I know I could update `CreateView` like this? The answer is that I looked at the source code and used [Classy Class-Based Views](#), an amazing resource that breaks down how each generic class-based view works in Django. Generic class-based views are great, but when you want to customize them, you must roll up your sleeves and understand what's happening under the hood. This is the downside of class-based vs. function-based views: more is hidden, and the inheritance chain must be understood. The more you use and customize built-in views, the more comfortable you will become with making customizations like this. Generally, a specific method, like `form_valid`, can be overridden to achieve your desired result instead of having to rewrite everything from scratch yourself.

14장: 권한 및 인증

현재 우리의 신문 웹사이트에는 여러 가지 문제가 있습니다. 우선, 우리는 우리의 신문이 재정적으로 지속 가능하길 원합니다. 시간이 더 있다면, 접근 요금을 부과하기 위해 전용 결제 앱을 추가할 수 있습니다. 하지만 최소한, 우리는 사용자가 기사를 보기 위해 로그인하도록 요구하는 것과 같은 권한 및 인증에 대한 규칙을 추가하고 싶습니다. 성숙한 웹 프레임워크인 Django는 기사 목록 페이지에 대한 접근을 제한하고 오직 기사의 저자만이 이를 편집하거나 삭제할 수 있도록 추가 제한을 설정하는 데 사용할 수 있는 내장 인증 기능을 가지고 있습니다.

개선된 CreateView

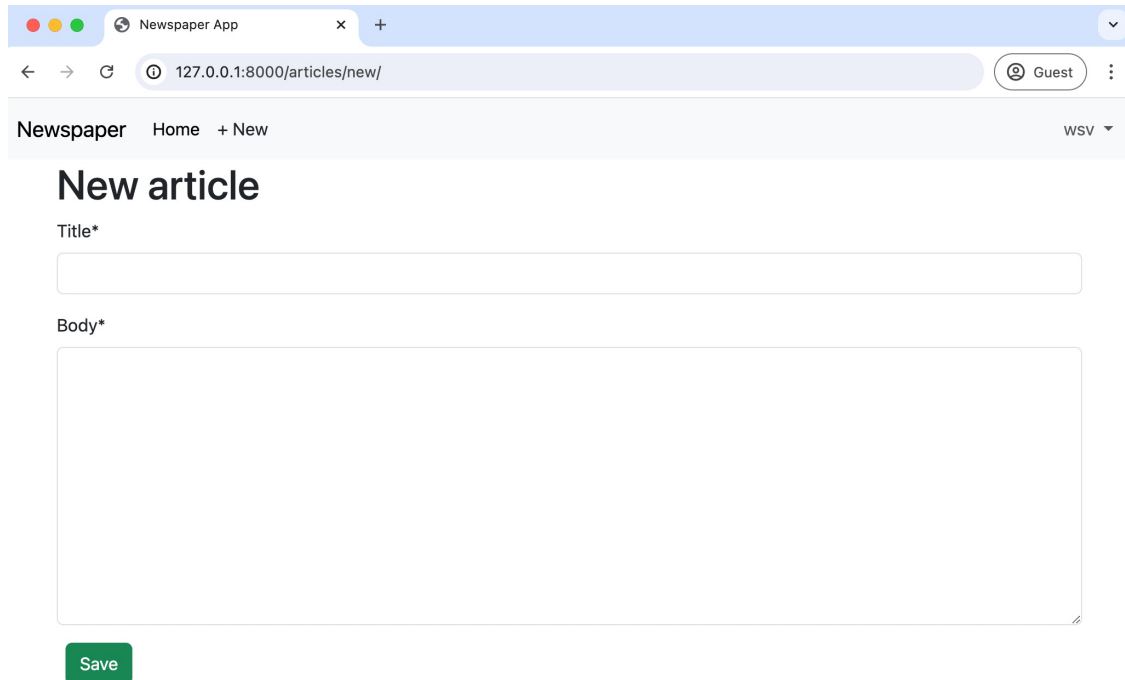
현재 새 기사의 저자는 기존 사용자 중 누구로든 설정할 수 있습니다. 대신, 현재 로그인한 사용자로 자동 설정되어야 합니다. 우리는 Django의 CreateView를 수정하여 저자 필드를 제거하고 대신 form_valid 메서드를 통해 자동으로 설정할 수 있습니다. 코드

```
# articles/views.py
class ArticleCreateView(CreateView): model =
    Article
    template_name = "article_new.html" fields =
    ("title", "body") # 새로 추가

    def form_valid(self, form): # 새로 추가
        form.instance.author = self.request.user
        return super().form_valid(form)
```

내가 CreateView를 이렇게 업데이트할 수 있다는 것을 어떻게 알았을까요? 그 답은 소스 코드를 살펴보고 [Classy Class-Based Views](#)라는 놀라운 리소스를 사용했기 때문입니다. 이 리소스는 Django에서 각 제네릭 클래스 기반 뷰가 어떻게 작동하는지를 설명합니다. 제네릭 클래스 기반 뷰는 훌륭하지만, 이를 사용자 정의하고 싶을 때는 소매를 걷어붙이고 내부에서 무슨 일이 일어나고 있는지를 이해해야 합니다. 이것이 클래스 기반 뷰와 함수 기반 뷰의 단점입니다: 더 많은 것이 숨겨져 있으며, 상속 체인을 이해해야 합니다. 내장 뷰를 더 많이 사용하고 사용자 정의할수록, 이러한 사용자 정의를 만드는 데 더 편안해질 것입니다. 일반적으로 form_valid와 같은 특정 메서드는 모든 것을 처음부터 다시 작성하는 대신 원하는 결과를 얻기 위해 재정의할 수 있습니다.

Now reload the browser and try clicking on the “+ New” link in the top nav. It will redirect to the updated create page where author is no longer a field.



The screenshot shows a web browser window with the title 'Newspaper App' and a single tab. The address bar shows the URL '127.0.0.1:8000/articles/new/'. The user is logged in as 'Guest'. The navigation bar includes 'Newspaper', 'Home', and '+ New' links, with a 'WSV' dropdown menu on the right. The main content area is titled 'New article' and contains two required input fields: 'Title*' and 'Body*'. The 'Body*' field is a large text area. At the bottom left of the form is a green 'Save' button.

New Article Link

If you create a new article and go into the admin, you will see it is automatically set to the currently logged-in user.

Authorizations

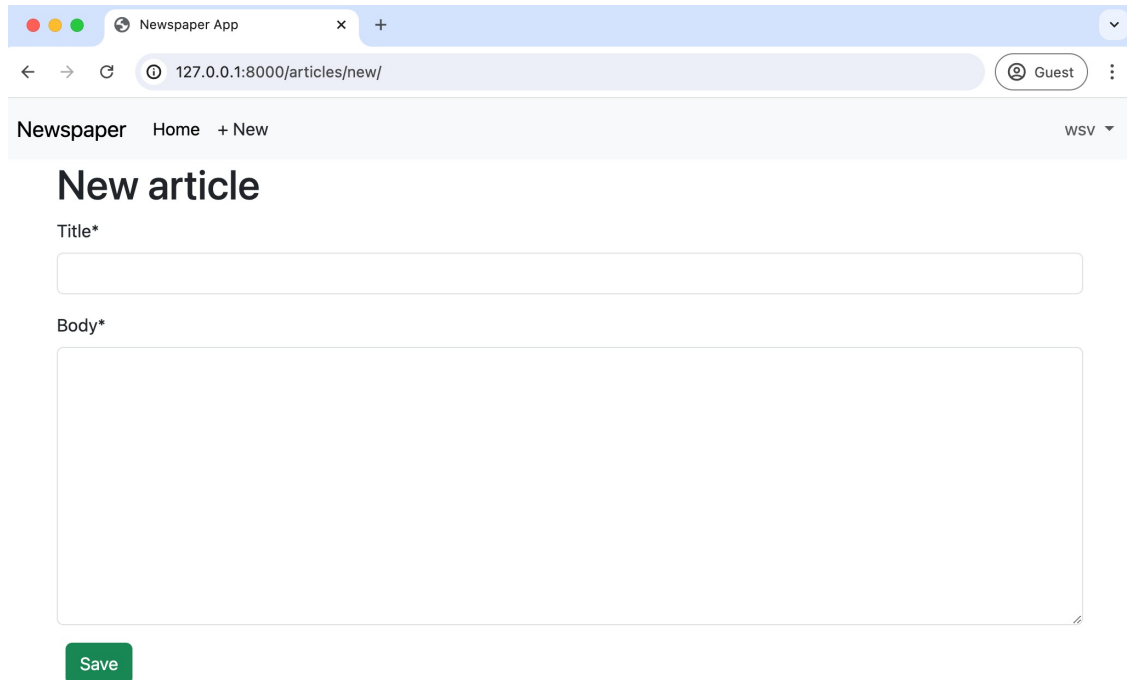
There are multiple issues related to the lack of authorizations for our current project. We want to restrict access to only users so we can one-day charge readers to access our newspaper. But beyond that, any random logged-out user who knows the correct URL can access any part of the site.

Consider what would happen if a logged-out user tried to create a new article. To try it out, click on your username in the upper right corner of the navbar, then select “Log Out” from the dropdown options. The “+ New” link disappears from the navbar, but what happens if you go to it directly:

`http://127.0.0.1:8000/articles/new/`.

The page is still there.

이제 브라우저를 새로 고치고 상단 내비게이션의 “+ 새로 만들기” 링크를 클릭해 보세요. 업데이트된 생성 페이지로 리디렉션되며, 저자는 더 이상 필드가 아닙니다.

A screenshot of a web browser showing the 'Newspaper App' interface. The browser's address bar displays '127.0.0.1:8000/articles/new/'. The page has a header with 'Newspaper', 'Home', and '+ New' links, and a user profile 'Guest'. The main content area is titled 'New article' and contains two input fields: 'Title*' and 'Body*'. Below the 'Body*' field is a green 'Save' button.

새 기사 링크

새 기사를 만들고 관리자로 들어가면 현재 로그인한 사용자로 자동 설정된 것을 볼 수 있습니다.

권한

현재 프로젝트의 권한 부족과 관련된 여러 문제가 있습니다. 우리는 독자들에게 신문에 접근할 수 있도록 언젠가 요금을 부과할 수 있도록 사용자에게만 접근을 제한하고자 합니다. 그러나 그 외에도 올바른 URL을 아는 무작위로 로그아웃된 사용자는 사이트의 어떤 부분에도 접근할 수 있습니다.

로그아웃된 사용자가 새 기사를 만들려고 시도하면 어떤 일이 발생할지 고려해 보세요. 시도해 보려면 내비게이션 바의 오른쪽 상단 모서리에 있는 사용자 이름을 클릭한 다음 드롭다운 옵션에서 “로그 아웃”을 선택하세요. “+ 새로 만들기” 링크는 내비게이션 바에서 사라지지만, 직접 가면 어떻게 될까요: <http://127.0.0.1:8000/articles/new/>.

페이지는 여전히 존재합니다.